

Generative AI using Large Language Models (LLMs)

Lecture 4 | 02/13/25

Instructor: Chen Wang
Fri 4:10 pm - 6:40 pm | 517 Hamilton Hall



Agenda

RAG & Tool Calling

- RAG
 - Semantic Search
 - Keyword Search
 - Embeddings
 - Hybrid Search
 - ReRank
 - Generating Answers
- VectorDB
 - KNN, ANN, HNSW
- GraphRAG
- AgenticRAG
- Functions, Tools and Agents
- LangChain Notebooks



Retrieval Augmented Generation

- What is RAG?
 - “**Retrieval Augmented Generation (RAG)** is a sophisticated approach that enhances the capabilities of large language models (LLMs) by integrating retrieval mechanisms with generative models. This synergy allows the model to access a wealth of external knowledge, significantly improving the relevance and accuracy of generated responses [1]”.
- Why we need RAG?
 - Improves accuracy and relevance of LLM outputs
 - Allows access to current information beyond LLM training data
 - Cost-effective alternative to fine-tuning or retraining LLMs

[1] <https://www.restack.io/p/retrieval-augmented-generation-answer-rag-vs-semantic-cat-ai>



Semantic Search

Traditional Semantic Search vs. RAG

- Semantic Search: Retrieves relevant documents based on meaning and intent
- RAG: Retrieves relevant information AND uses it to generate new, contextual responses

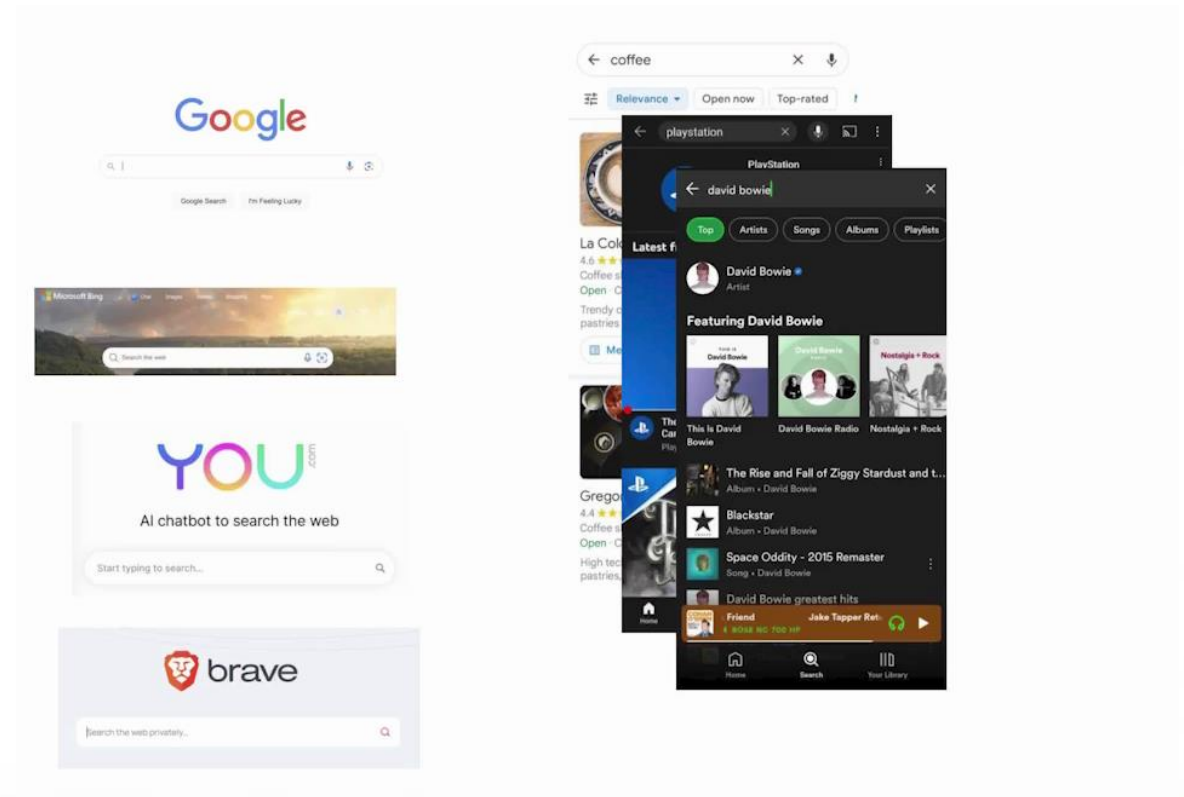
Key Differences

- Semantic Search focuses on retrieval;
- RAG combines retrieval with generation; RAG provides more dynamic and tailored responses



Search is crucial to how we navigate the world

Semantic Search





Keyword Search

Query

What color is the grass?

Responses

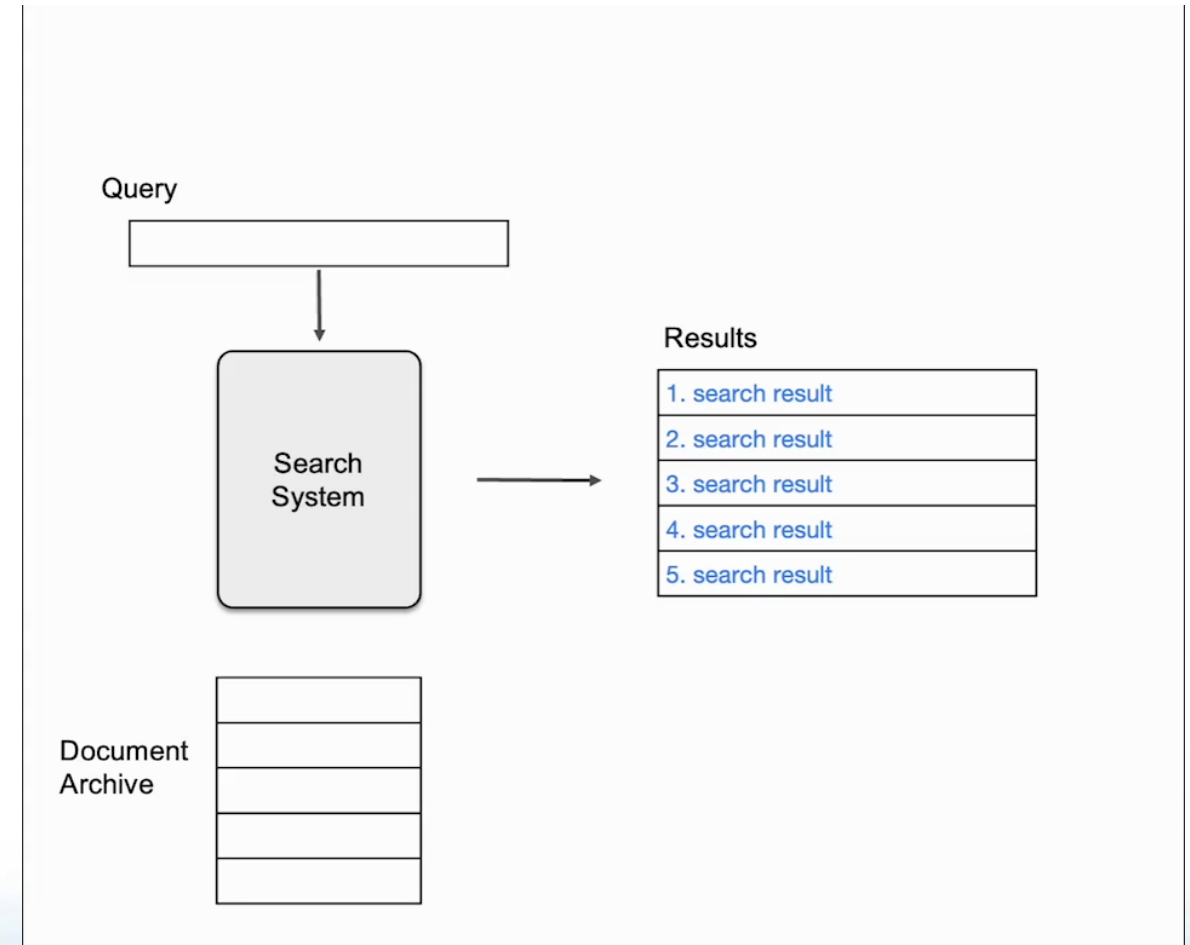
Tomorrow is Saturday
The grass is green
The capital of Canada is Ottawa
The sky is blue
A whale is a mammal

Number of words
in common

1
3
2
2
1

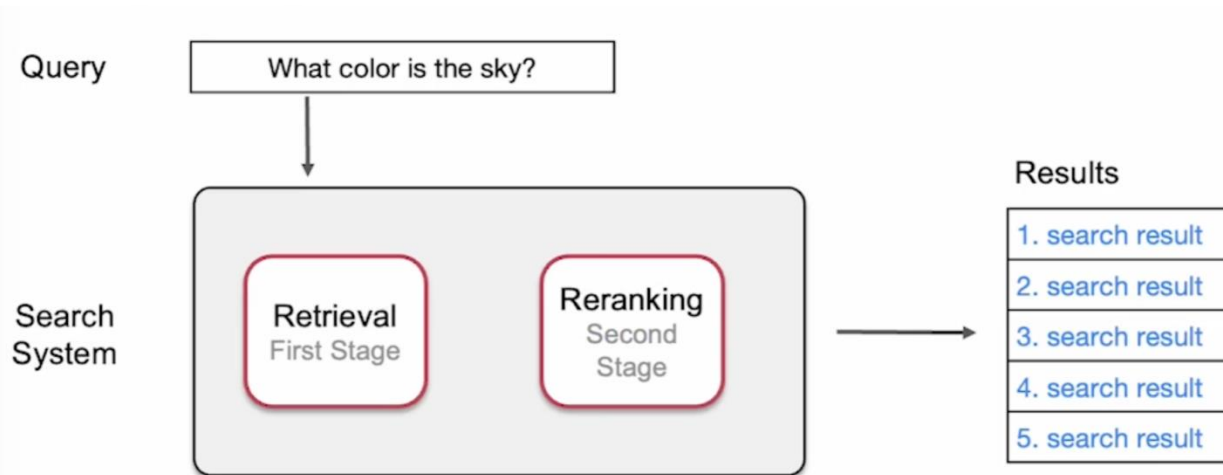


Search at a high level





Keyword Search (Internals)



BM25 Algorithm

Inverted Index

Keyword	Document IDs
abacus	1, 38, 18
...	
Color	23, 804
...	
Sky	804, 922



inverted index Algorithm

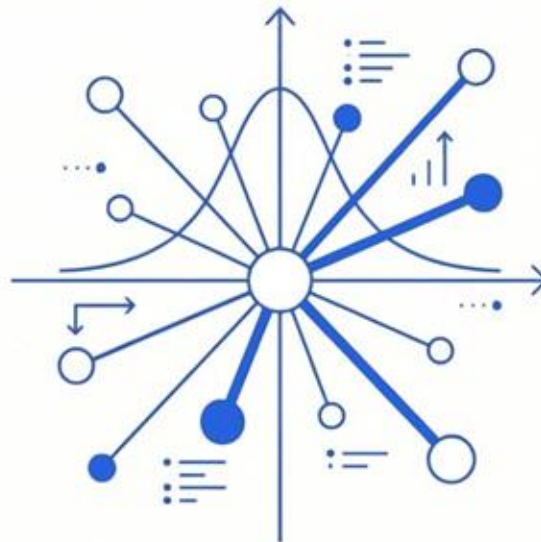


BM25 Algorithm, <https://www.geeksforgeeks.org/keyword-searching-algorithms-for-search-engines/>



BM25: The Algorithm Behind Modern Search Engines

Understanding the Best Matching 25 Ranking Function





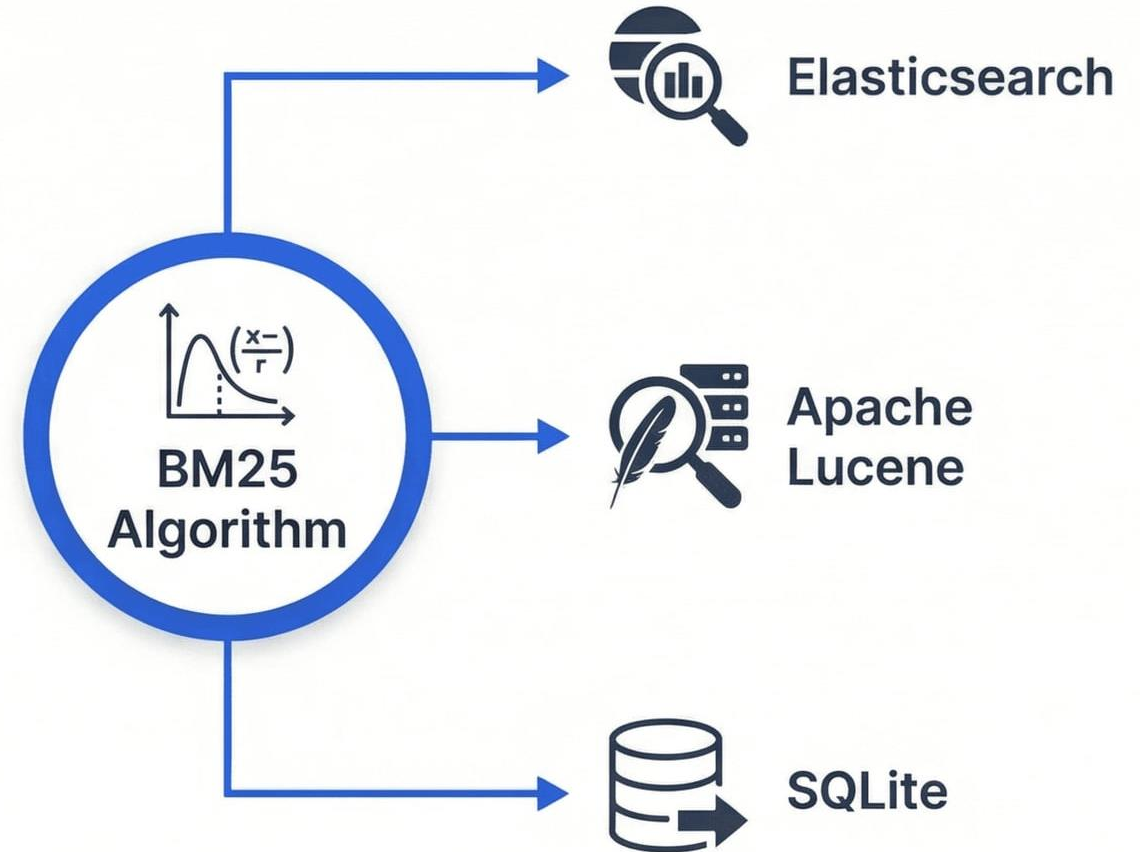
BM25 Powers the Search Systems You Use Every Day

Default ranking algorithm in Elasticsearch, Lucene, & SQLite

Uses probabilistic ranking to estimate relevance

Evolution from classic TF-IDF

Powers web search, enterprise search, & e-commerce

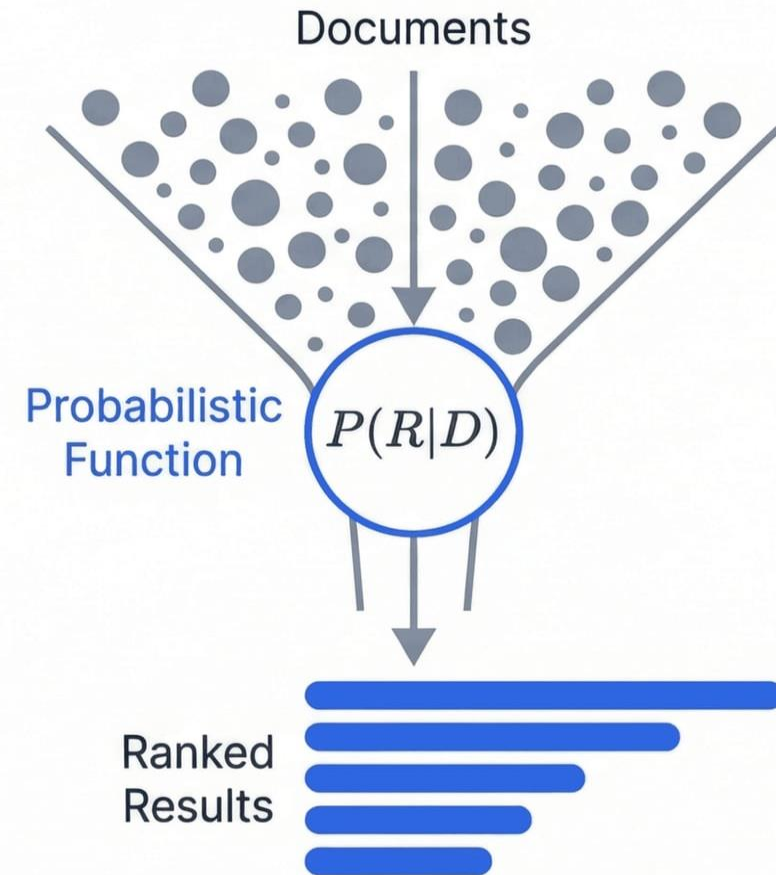




BM25 Ranks Documents by Probability Without Calculating Probability

Key Concepts

- **Foundation:** The Probability Ranking Principle.
- **Method:** Estimates relevance using observable features (Term Frequency, Doc Counts, Lengths).
- **Key Assumption:** Most documents in the collection are irrelevant.
- **Outcome:** Ranks by probability efficiently without computing full distributions.





The Complete BM25 Formula Combines Four Essential Components

2. Term Frequency

3. k_1 (Saturation)

$$\text{Score}(D, Q) = \sum \underbrace{\text{IDF}(q_i)}_{\text{1. IDF (Term Rarity)}} \cdot \frac{\overbrace{f(q_i, D) \cdot (k_1 + 1)}^{\text{2. Term Frequency} \quad \text{3. } k_1 \text{ (Saturation)}}}{\underbrace{f(q_i, D) + k_1 \cdot (1 - b + b \cdot |D|/\text{avg}d_l)}_{\text{4. } b \text{ \& Length Norm}}}$$

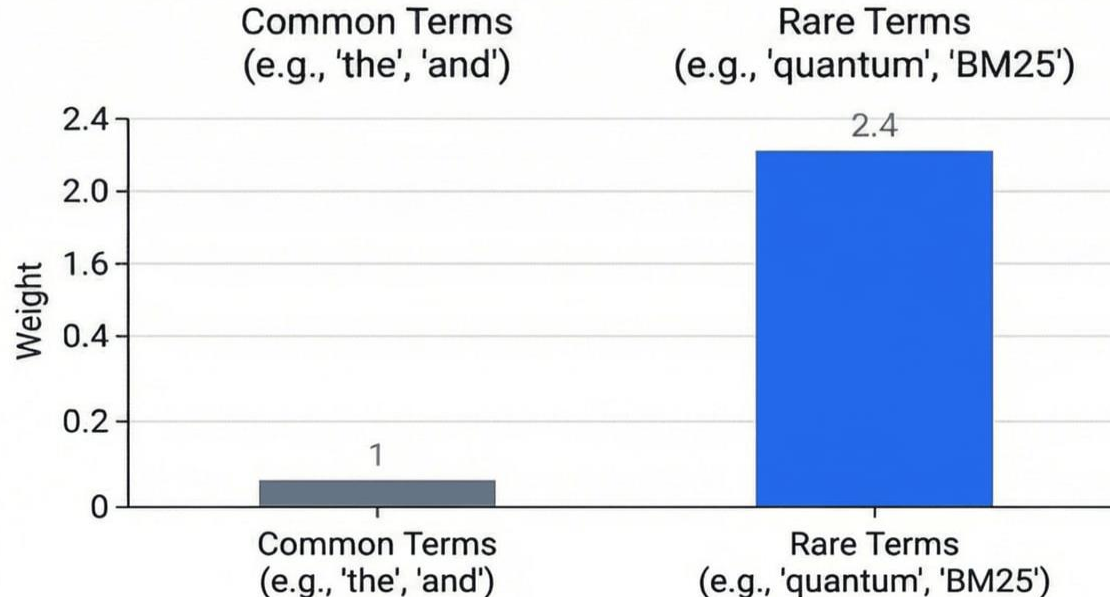
1. IDF (Term Rarity)

4. b & Length Norm



Inverse Document Frequency Boosts Rare Terms Over Civer Common

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$



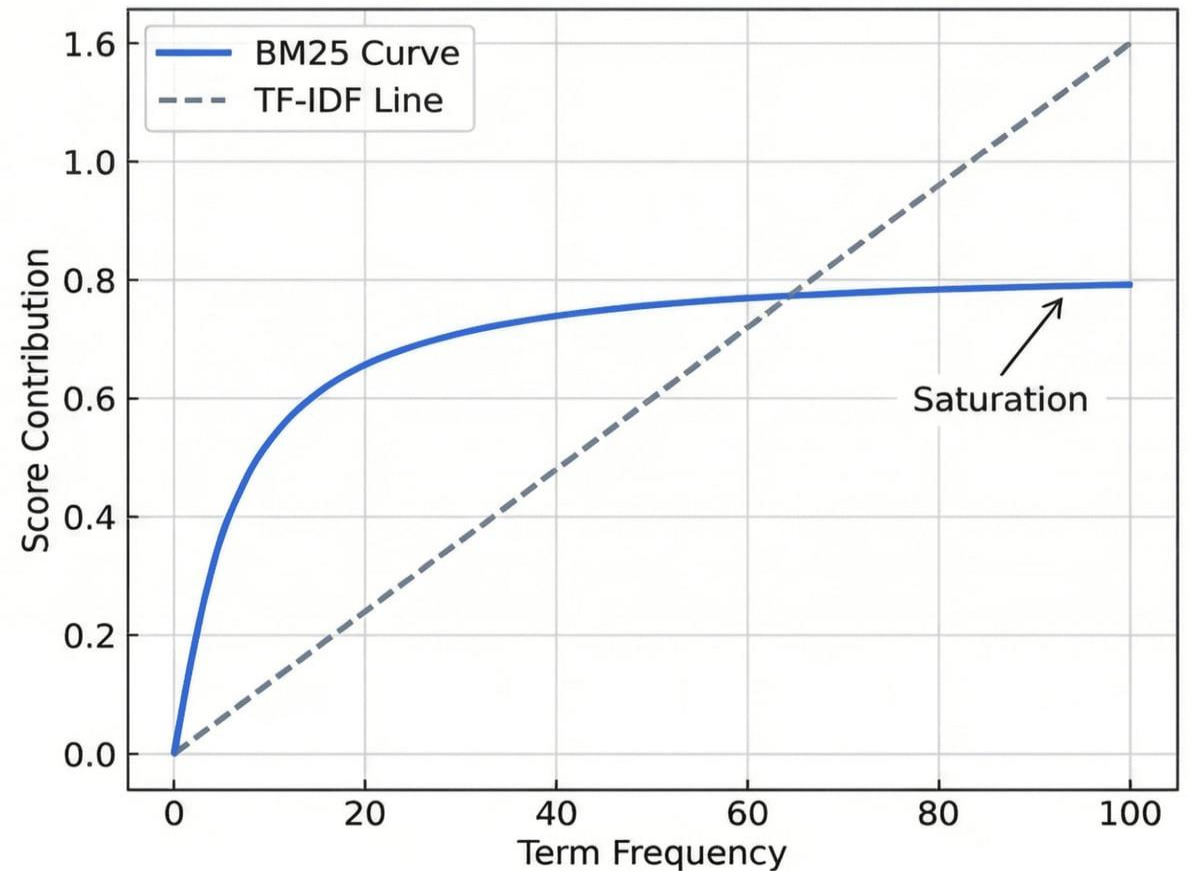
- Rare terms = High Weight (Informative)
- Common terms = Low Weight (Noise)
- Logarithmic scaling smooths values.



Term Frequency Saturation Prevents Over-Rewarding Repetition

- More occurrences increase score, but with diminishing returns.
- Prevents keyword stuffing and gaming the system.
- Parameter k_1 (default 1.2) controls saturation rate.
- Impact: 1st occurrence > 10th occurrence.

$$\text{Score}(\text{TF}) = \frac{\text{TF} * (k_1 + 1)}{\text{TF} + k_1}$$

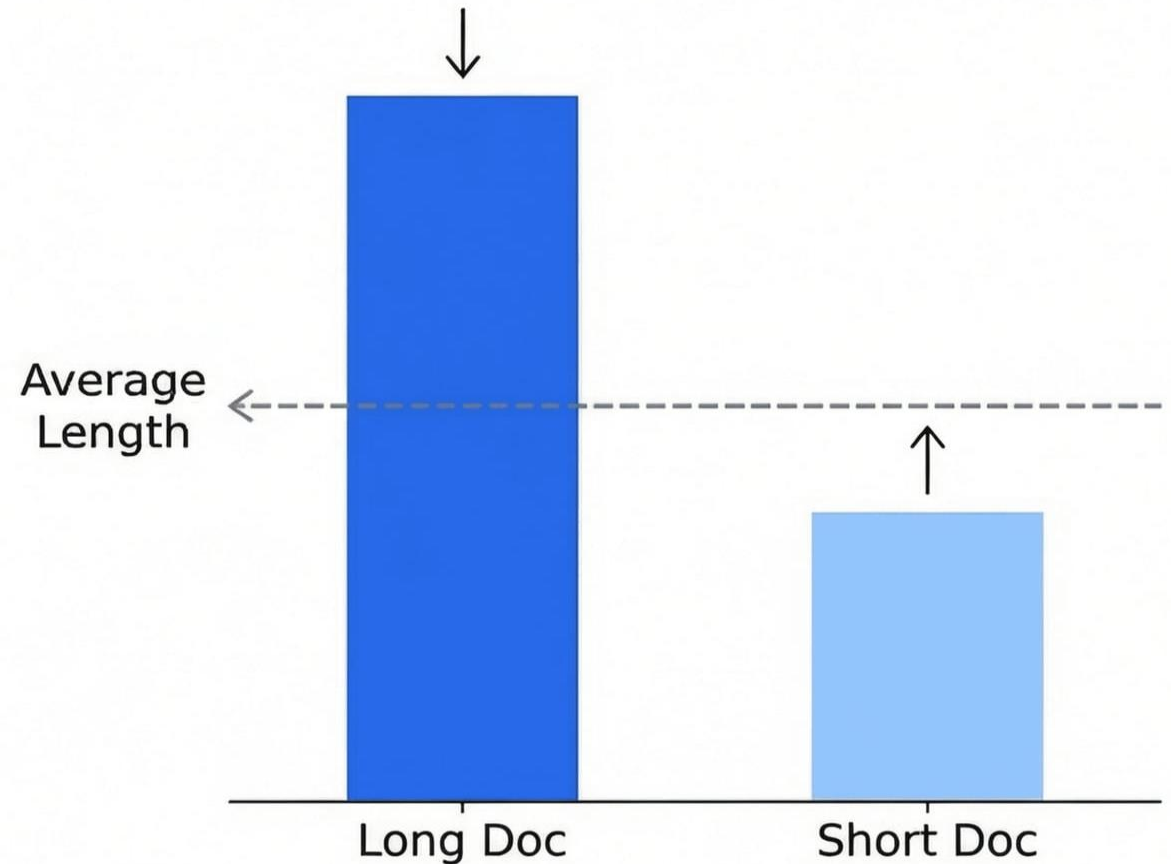




Document Length Normalization Ensures Fair Comparison Across Documents

- Long documents naturally repeat terms more often.
- Normalization prevents unfair scoring advantage.
- Parameter b (default 0.75) controls strength.
- Longer than Avg $\rightarrow$ Score Decreases.
- Shorter than Avg $\rightarrow$ Score Increases.

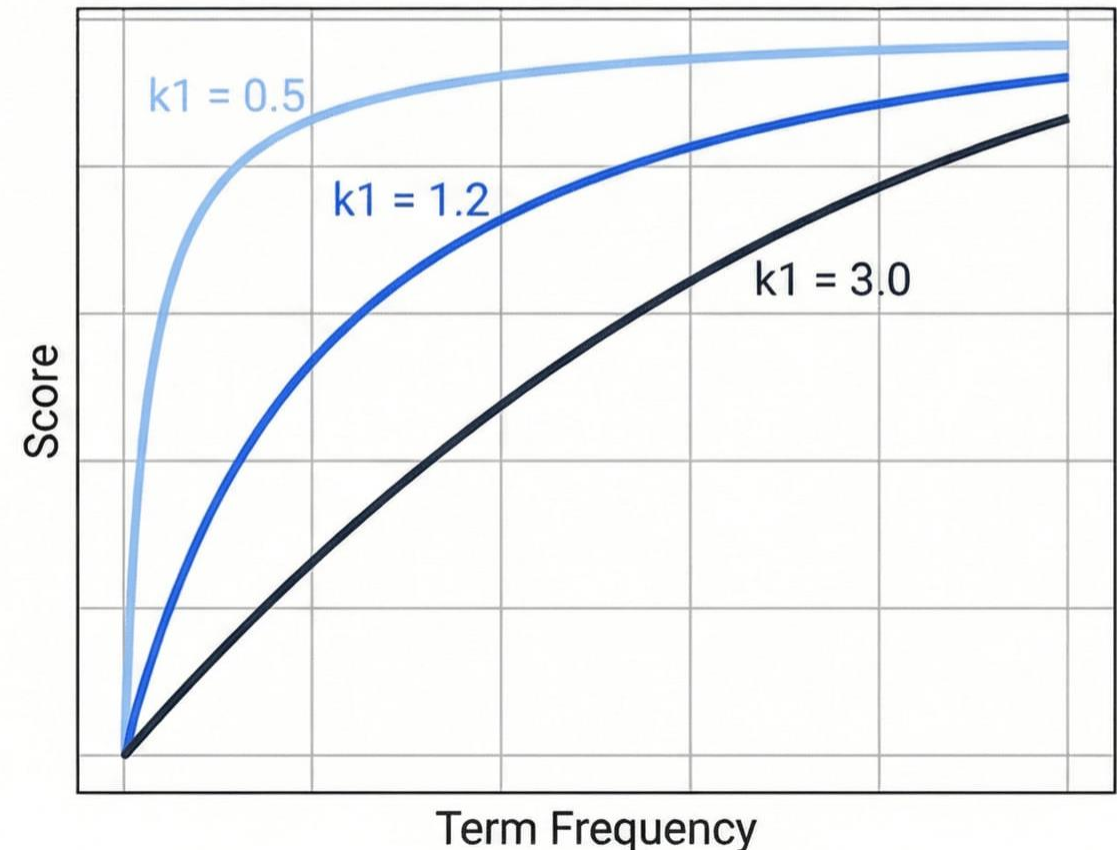
$$(1 - b + b \cdot |D|/\text{avg}d_l)$$





The k_1 Parameter Controls How Much Repetition Matters

- k_1 is a tunable hyperparameter (Default: 1.2).
- Controls how quickly the score saturates.
- $k_1 = 0$: Binary model (only presence matters).
- Higher k_1 : Repetition adds more score (slower saturation).
- Lower k_1 : Repetition adds less score (faster saturation).





The b Parameter Controls Document Length Normalization Strength



$$\text{Length Factor} = 1 - b + b \cdot (|D| / \text{avgdl})$$

Controls how much document length affects the score.

Higher b : Length differences are amplified (Long docs penalized more).

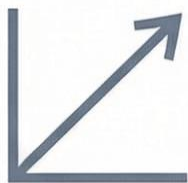
Lower b : Length differences are ignored.

Elasticsearch default: 0.75



BM25 Outperforms TF-IDF Through Saturation and Normalization

TF-IDF



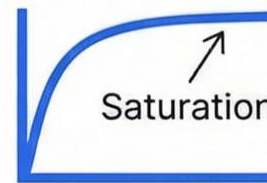
Linear Term Frequency
(Unbounded)

No Built-in Length Normalization

Fixed Formula (No Tuning)

Rewards Excessive Repetition

BM25



Saturating Term Frequency
(Asymptotic)

Explicit Length Normalization

Tunable Parameters (k_1 , b)

Diminishing Returns on Repetition





BM25 Excels at Lexical Matching But Has Limitations



Strengths

- **Robust & Reliable**
(Proven performance across datasets)
- **Efficient & Scalable** (Low computational cost)
- **Tunable** (Adaptable via k_1 & b parameters)
- **Interpretable** (Transparent scoring formula)



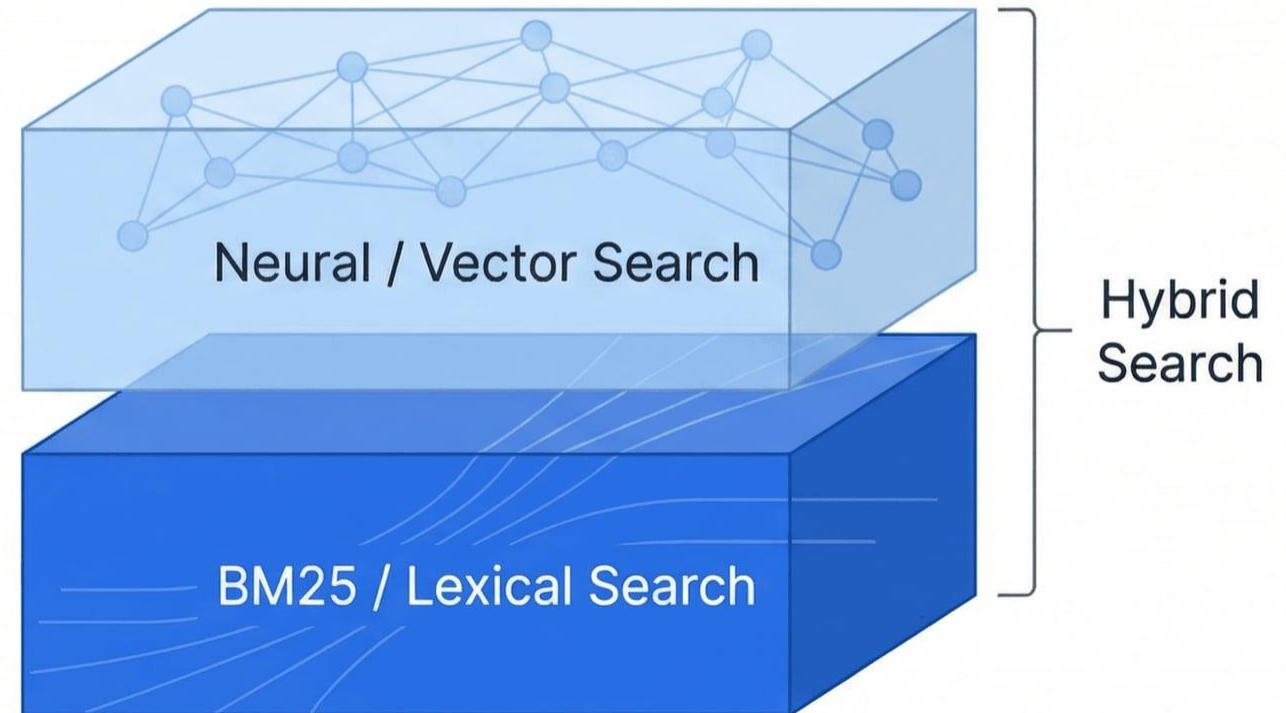
Limitations

- **Lexical Only**
(Matches terms, not concepts)
- **No Synonym Handling** (e.g., Car $\neq$ Auto)
- **Context-Agnostic** (No user personalization)
- **Corpus-Dependent**
Performance



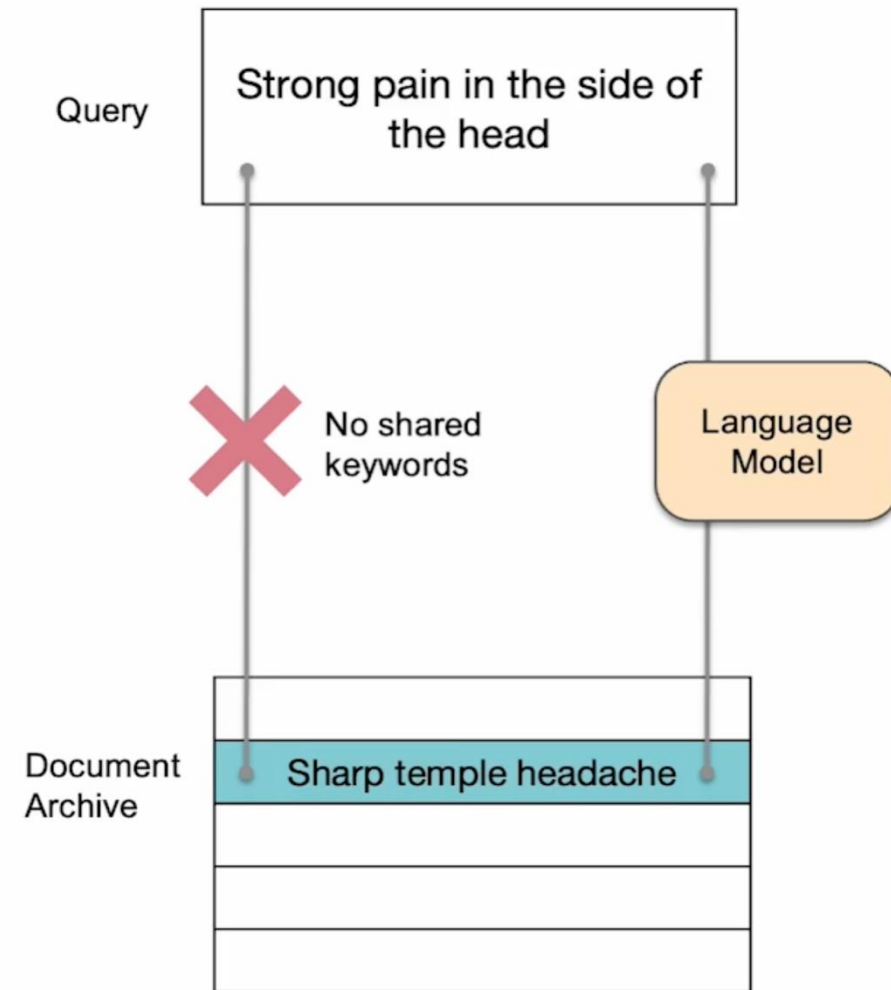
BM25 Remains the Foundation of Modern Information Retrieval

- The Backbone of Production Search Systems.
- Enables Efficient First-Stage Retrieval.
- Perfect Partner for AI: Speed (BM25) + Precision (Vectors).
- Future-Proof: Augmenting, Not Replacing.



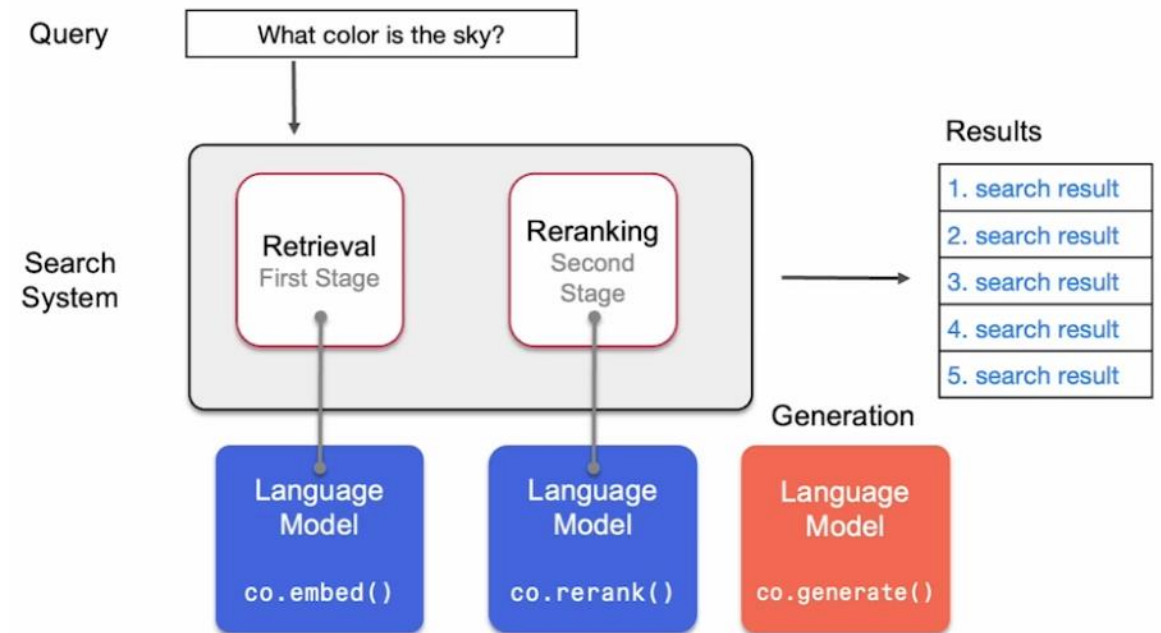


Limitation of keyword matching





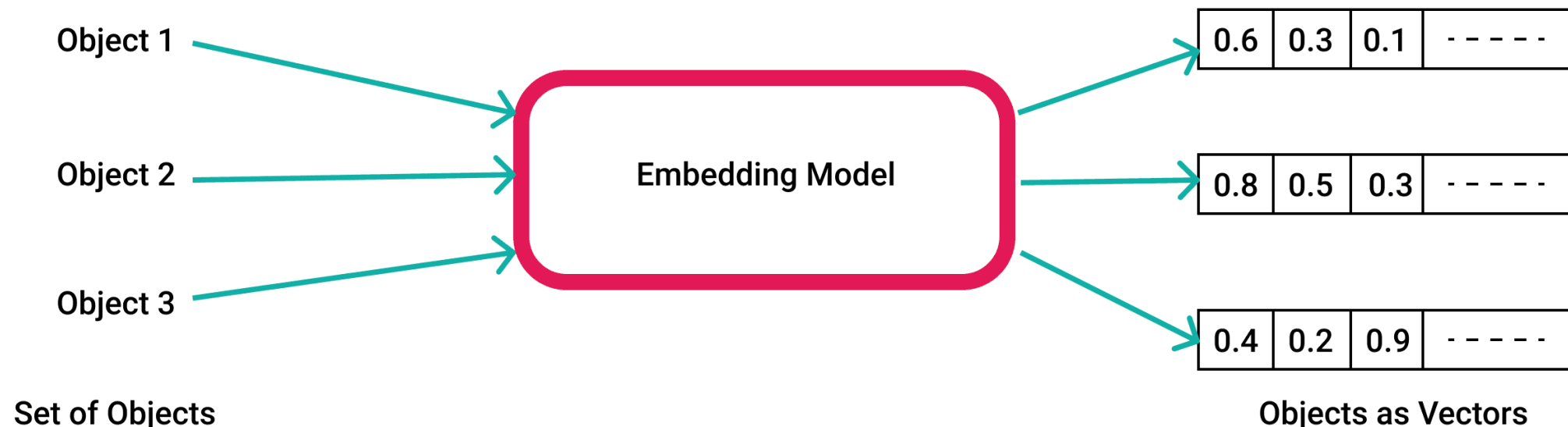
Language Models Can Improve Both Search Stages





Embeddings

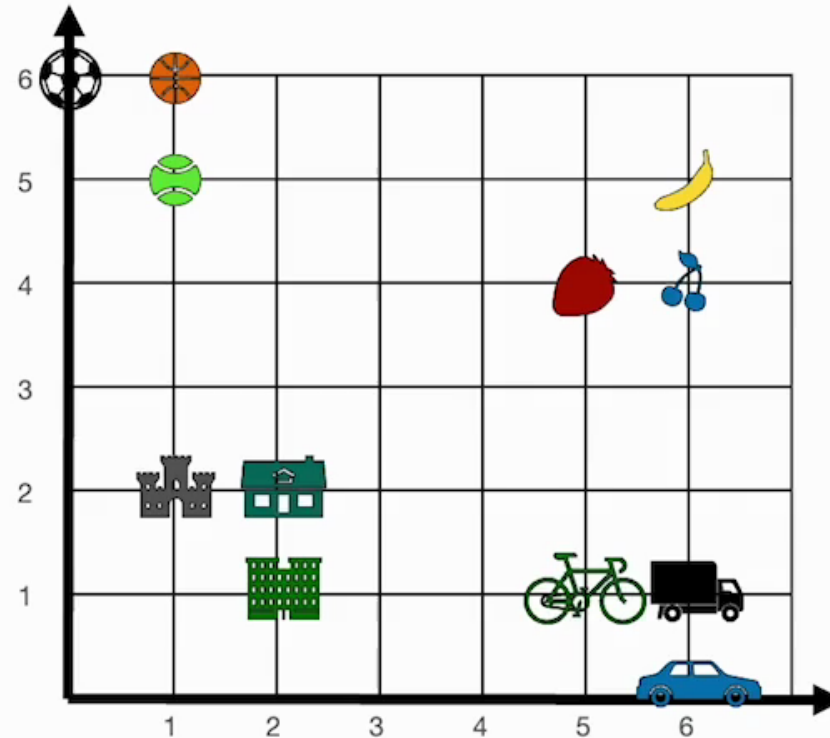
- Embeddings
 - are dense numerical representations of various types of data such as text, images, or audio, transforming them into a low-dimensional space suitable for machine learning.
- Key Points:
 - Dimensionality Reduction: They map high-dimensional data to a lower-dimensional space, capturing important features.
 - Semantic Similarity: Similar items have embeddings close together, allowing tasks like nearest neighbor search.
 - Applications: Used in NLP, computer vision, recommendation systems.
 - Learning from Data: Generated through algorithms like neural networks that learn from input data.





Embeddings

Quiz: Where would you put the word “apple”?



Word	Numbers	
Apple	?	?
Banana	6	5
Strawberry	5	4
Cherry	6	4
Soccer	0	6
Basketball	1	6
Tennis	1	5
Castle	1	2
House	2	2
Building	2	1
Bicycle	5	1
Truck	6	1
Car	6	0



Word Embedding s

Word	Numbers	
Apple	5	5
Soccer	0	6
House	2	2
Car	6	0

Word	Numbers			
A	-0.82	-0.32	...	0.23
Aardvark	0.42	1.28	...	-0.06
...	...	...	...	...
Zygote	-0.74	-1.02	...	1.35

← Hundreds →



Text Embeddings

Sentence	Numbers				
Hello, how are you?	0.39	0.49	...	-1.01	-0.72
I'm going to school today	-0.79	-0.05	...	-0.94	2.71
...	...	...	...	...	...
Once upon a time	3.23	-0.23	...	-1.45	0.82
Hi, how's it going?	0.41	0.48	...	-0.98	-0.66



Query

Responses

The grass is green





Chunking

Transformer (machine learning model)

[Article](#) [Talk](#)

From Wikipedia, the free encyclopedia

A **Transformer** is a [deep learning](#) architecture that relies on the [attention mechanism](#).^[1] It is notable for requiring less training time compared to previous [recurrent neural architectures](#), such as [long short-term memory](#) (LSTM),^[2] and has been prevalently adopted for training [large language models](#) on large (language) datasets, such as the [Wikipedia Corpus](#) and [Common Crawl](#), by virtue of the [parallelized](#) processing of input sequence.^[3] More specifically, the model takes in [tokenized](#) ([byte pair encoding](#)) input tokens, and at each layer, contextualizes each token with other (unmasked) input tokens in parallel via [attention mechanism](#). Though the Transformer model came out in 2017, the core attention mechanism was proposed earlier in 2014 by Bahdanau, Cho, and Bengio for [machine translation](#).^{[4][5]} This architecture is now used not only in [natural language processing](#), [computer vision](#),^[6] but also in [audio](#),^[7] and multi-modal processing. It has also led to the development of [pre-trained systems](#), such as [generative pre-trained transformers](#) (GPTs)^[8] and [BERT](#)^[9] (Bidirectional Encoder Representations from Transformers).

Background [\[edit \]](#)

Before transformers, most state-of-the-art NLP systems relied on gated RNNs, such as [LSTMs](#) and [gated recurrent units](#) (GRUs), with various [attention mechanisms](#) added to them. Unlike RNNs, transformers do not have a recurrent structure. Provided with enough training data, their attention mechanisms alone can match the performance of RNNs with attention added.^[1]

Previous work [\[edit \]](#)

In 1992, [Jürgen Schmidhuber](#) published the fast weight controller as an alternative to RNNs that can learn "internal spotlights of attention,"^[10] and experimented with using it to learn [variable binding](#).^[11]

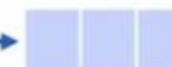
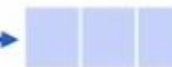
In a fast weight controller, a [feedforward neural network](#) ("slow") learns by [gradient descent](#) to control the weights of another neural network ("fast") through [outer products](#) of self-generated activation patterns called "FROM" and "TO" which corresponds to "key" and "value" in the attention mechanism.^[12] This fast weight is applied to queries. The attention mechanism may be obtained by interposing a [softmax](#) operator and three linear operators (one for each of query, key, and value).^{[12][13]}

Flawed sequential processing [\[edit \]](#)

Models, used before the transformer models prevailed, processed the input [tokens](#) sequentially, maintaining a state vector representing all the tokens up to the current token in the input. To process the n th token, such a model combined the vector representing the sentence up to token $n - 1$ with the information of the new token to create a new state, representing the sentence up to token n . Theoretically, the information from one token can propagate arbitrarily far down the sequence, if at every point the state continues to encode contextual information about the token.

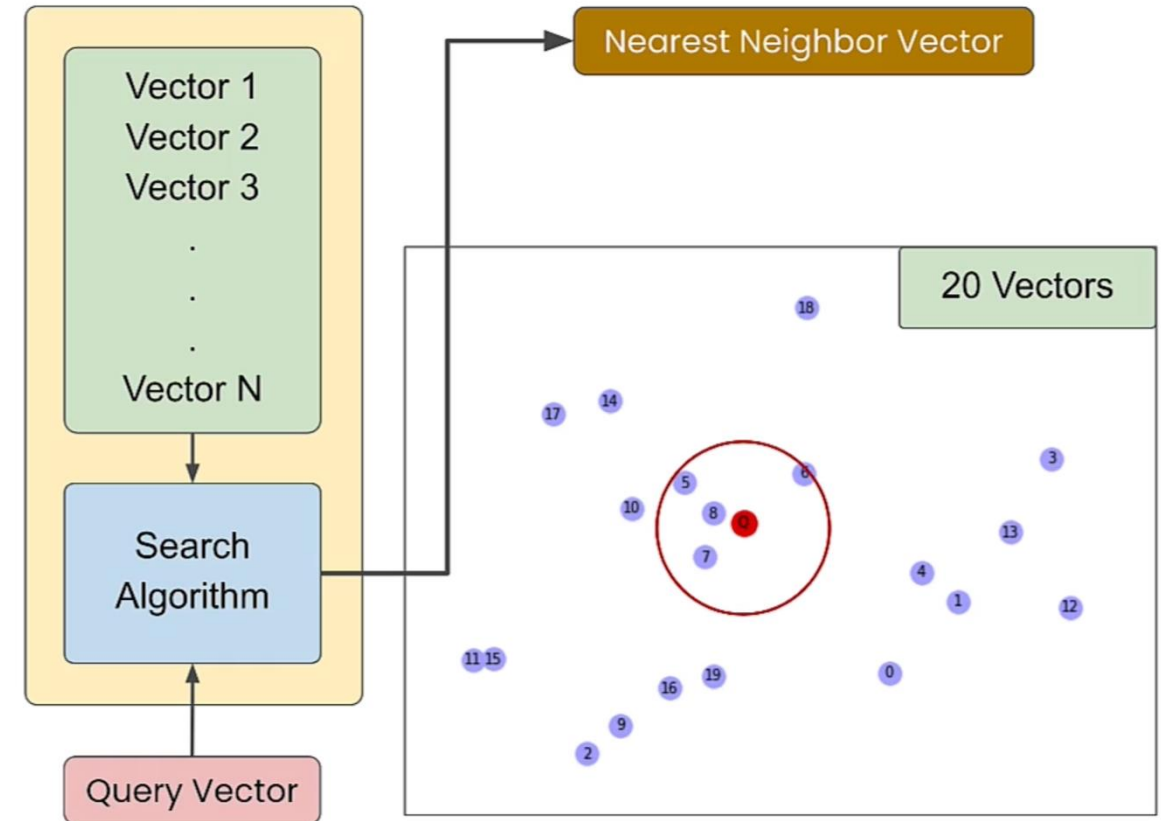
Query

Do transformers have a recurrent structure?





Search for Similar Vectors

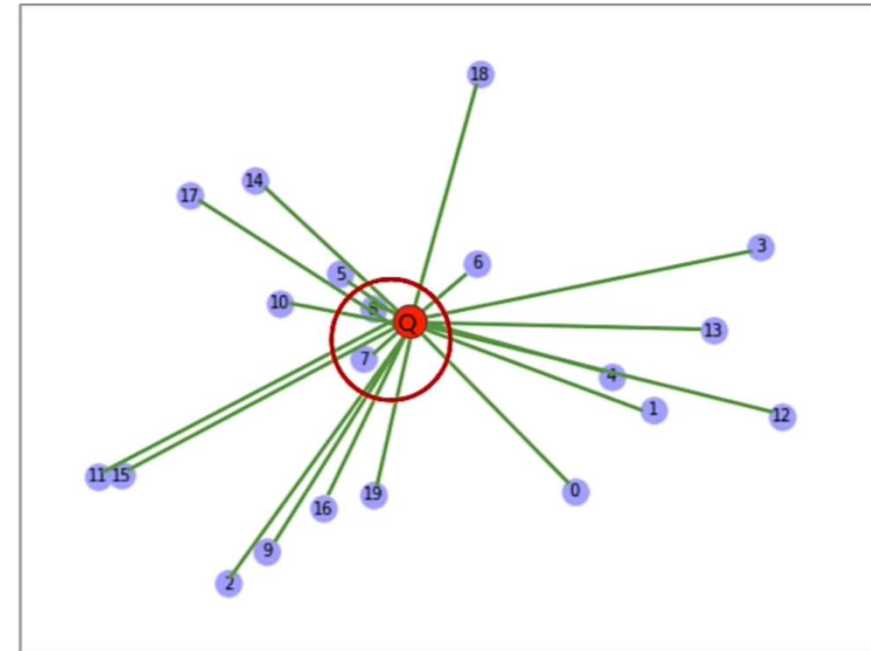


Find "K" nearest neighbors to a search query - K-NN



'brute force' search algorithm

'brute force' search algorithm

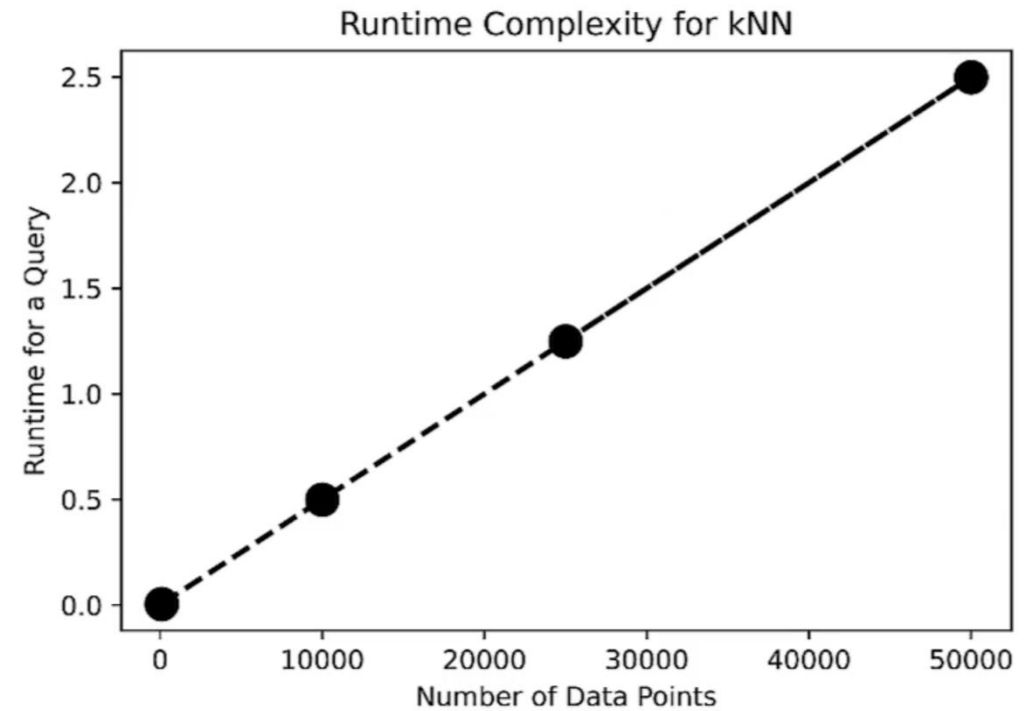


1. Measure the L2 distance between the Query and **each** vector
2. Sort **all** those distances
3. Return the top k matches. These are the most semantically similar points.



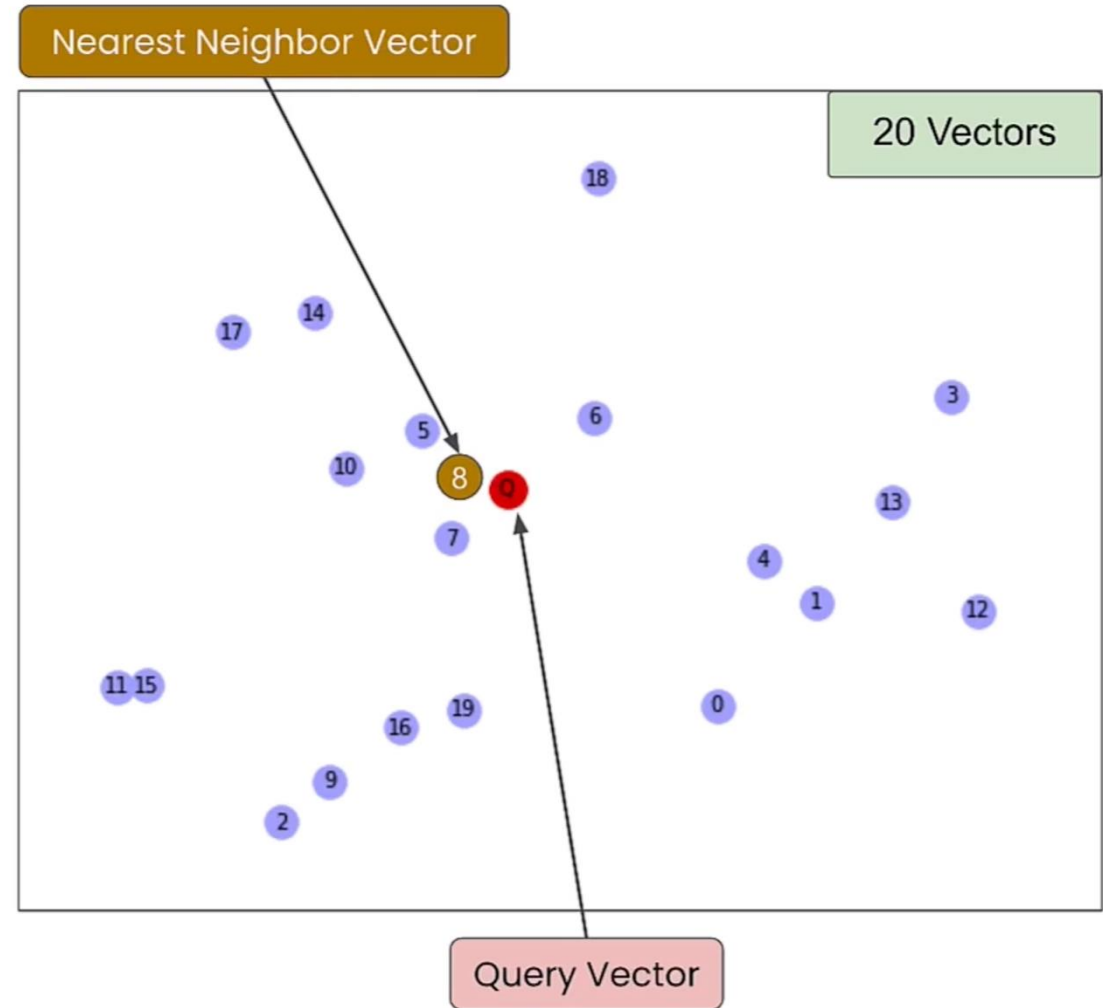
Runtime Complexity of KNN

- $O(dN)$ runtime complexity for search



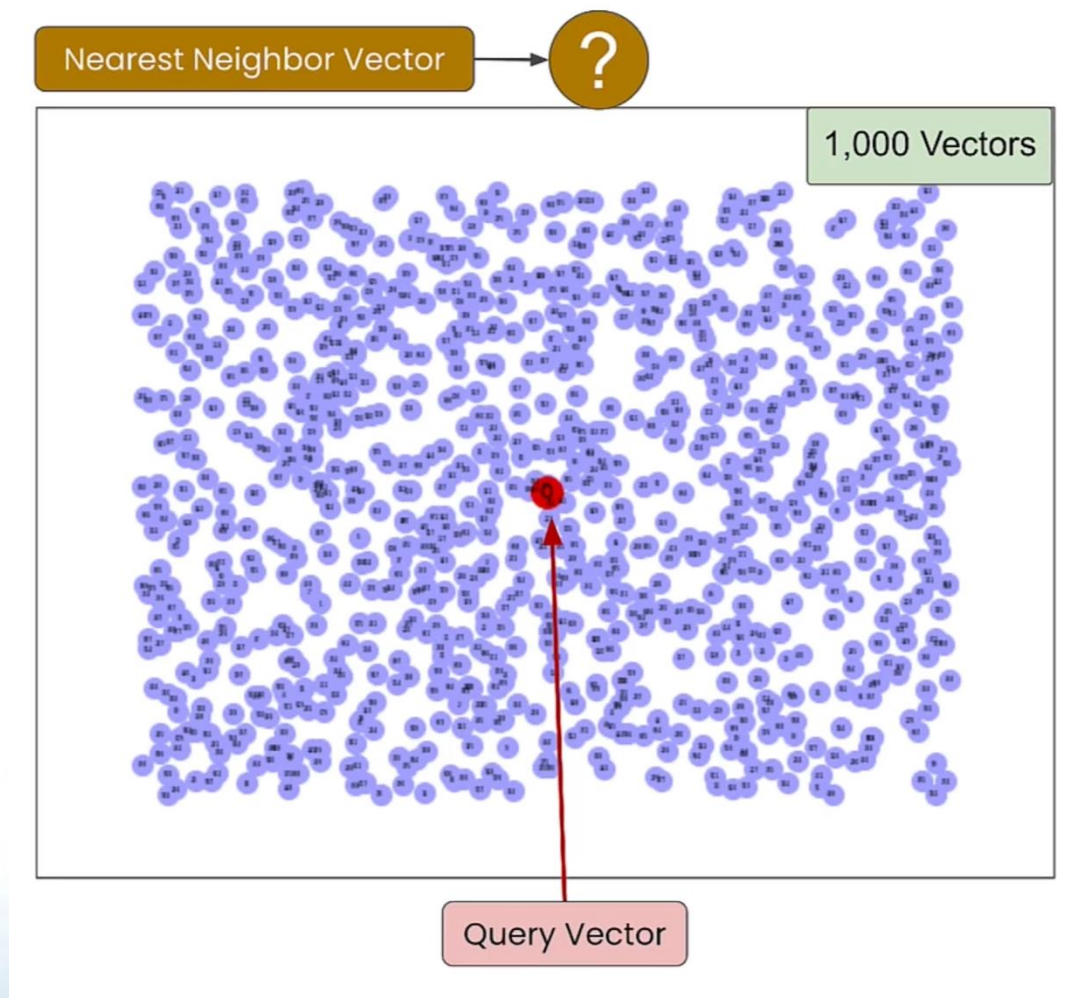


**Not a big
problem at
small scale**





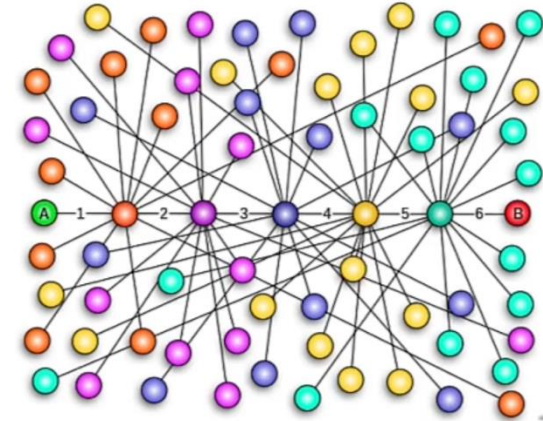
Big
problem at
scale





Small World

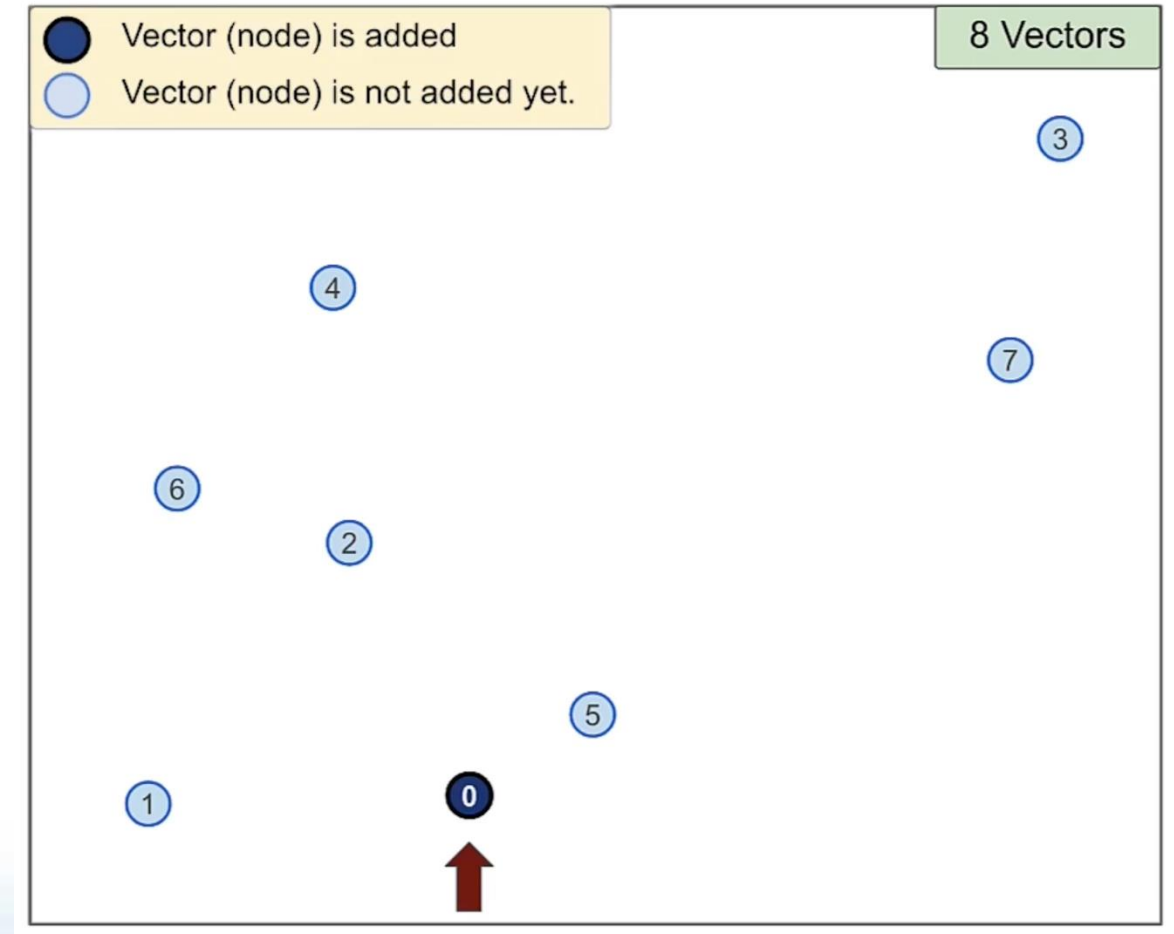
small world



- Observed phenomenon in human social networks where everyone is closely connected.
- **Six Degrees of Separation:** Popularized concept that any two individuals are on average separated by six acquaintance links.
- **High Clustering:** Nodes form tightly-knit groups, with members of groups connecting to one another.
- **Short Path Lengths:** Despite high clustering, only a few steps are needed to connect any two nodes.

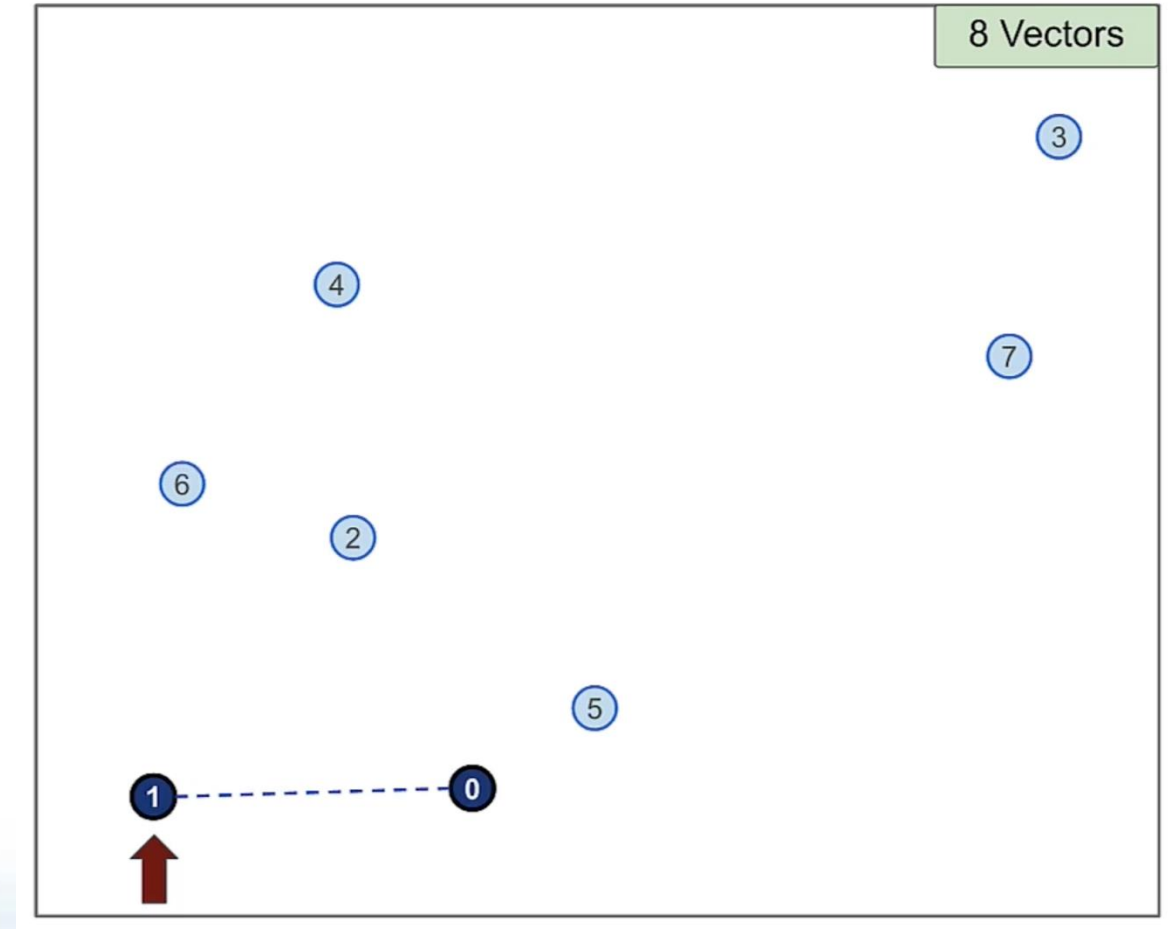


Navigate Small World (Constructing)



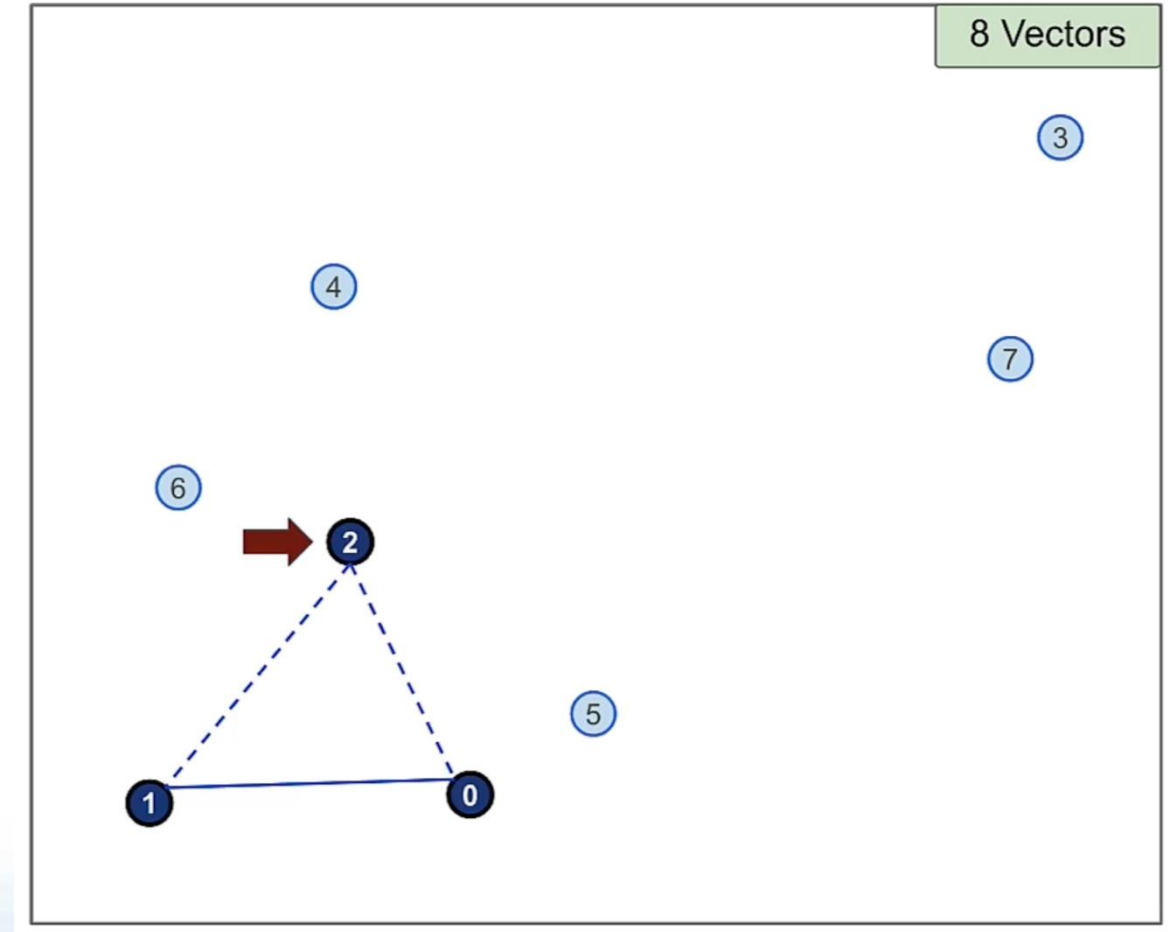


Navigate Small World (Constructing)



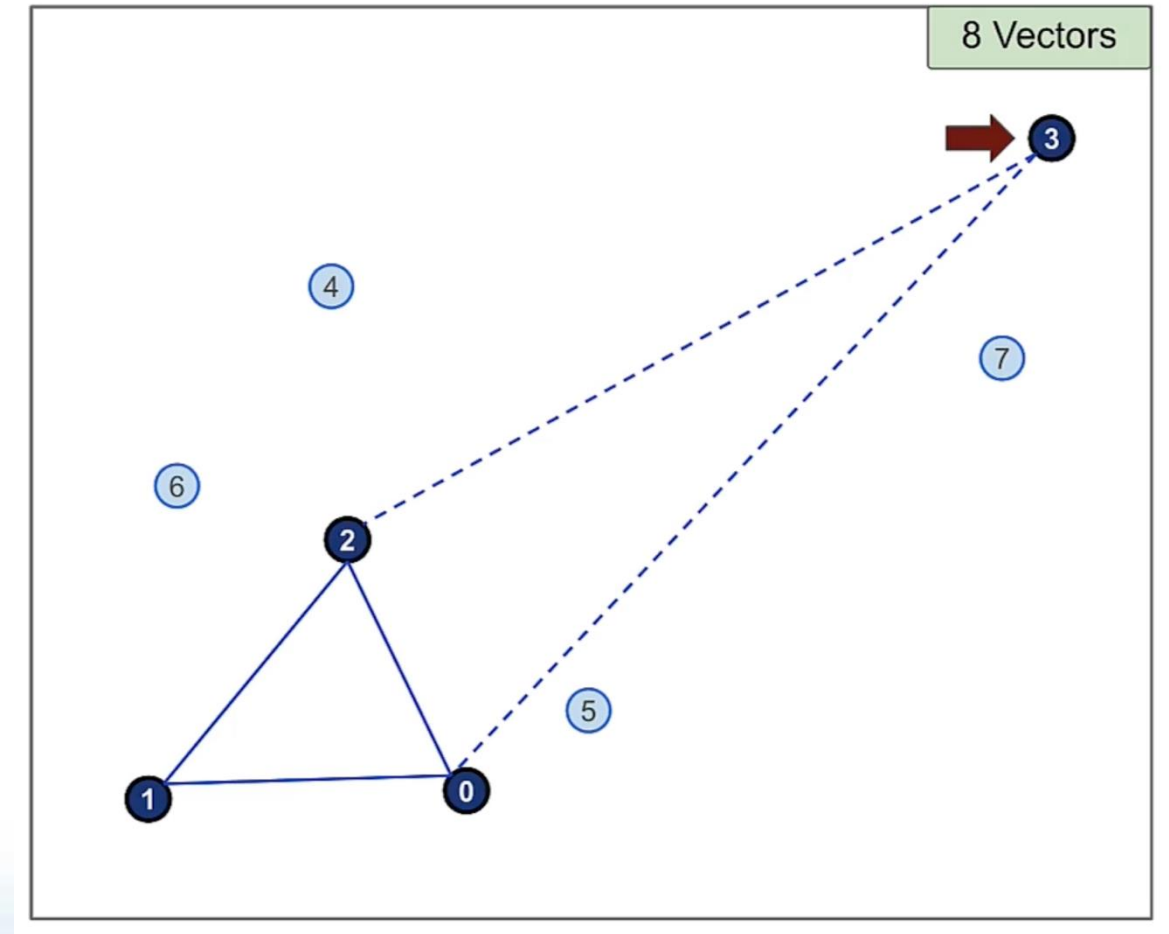


Navigate Small World (Constructing)



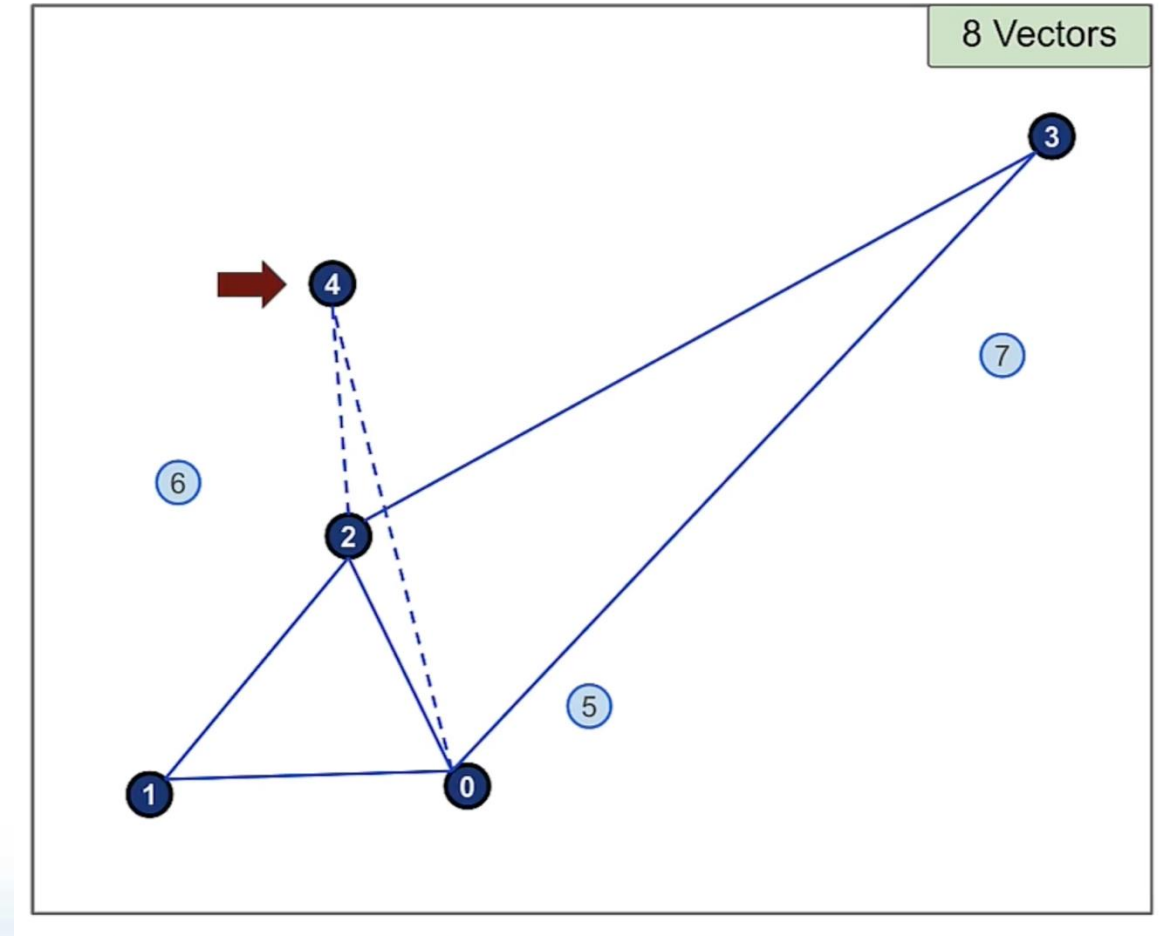


Navigate Small World (Constructing)



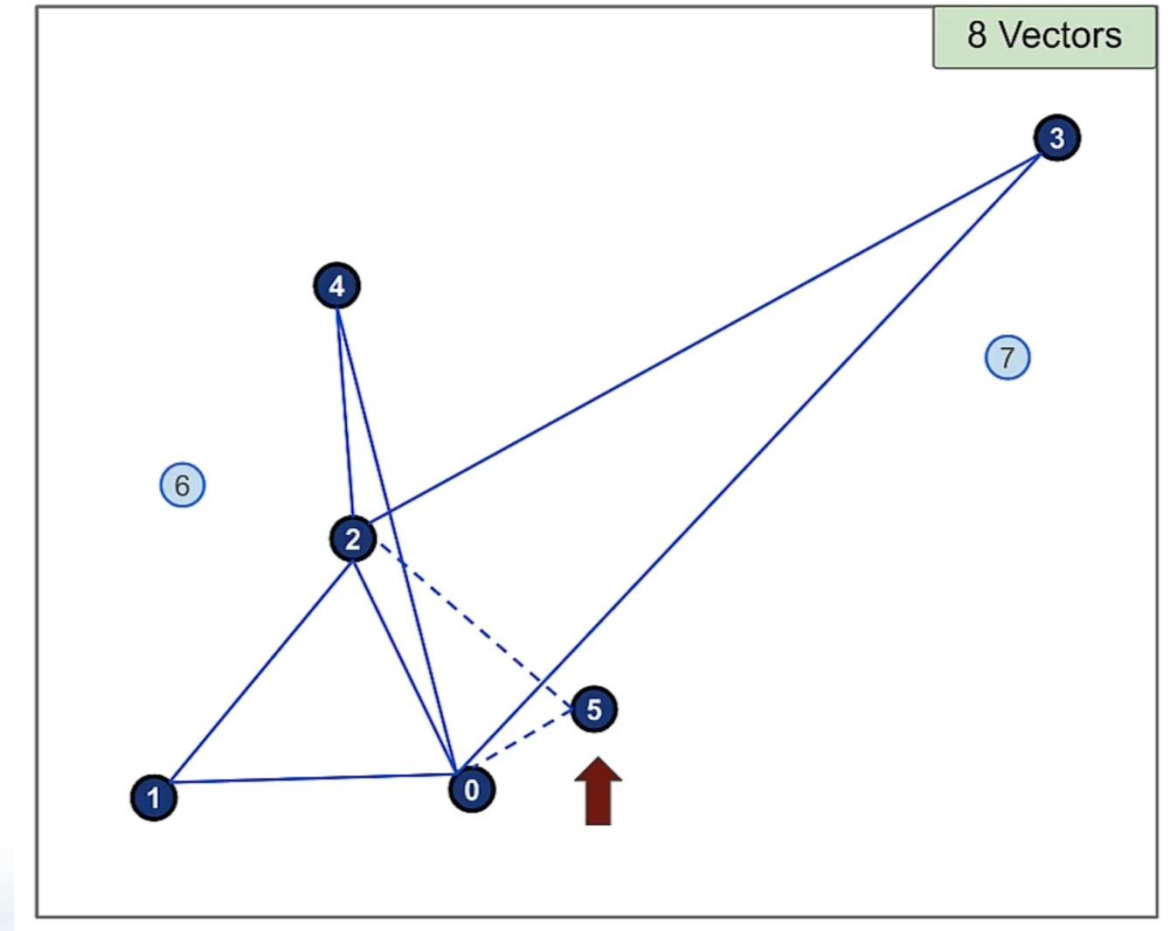


Navigate Small World (Constructing)



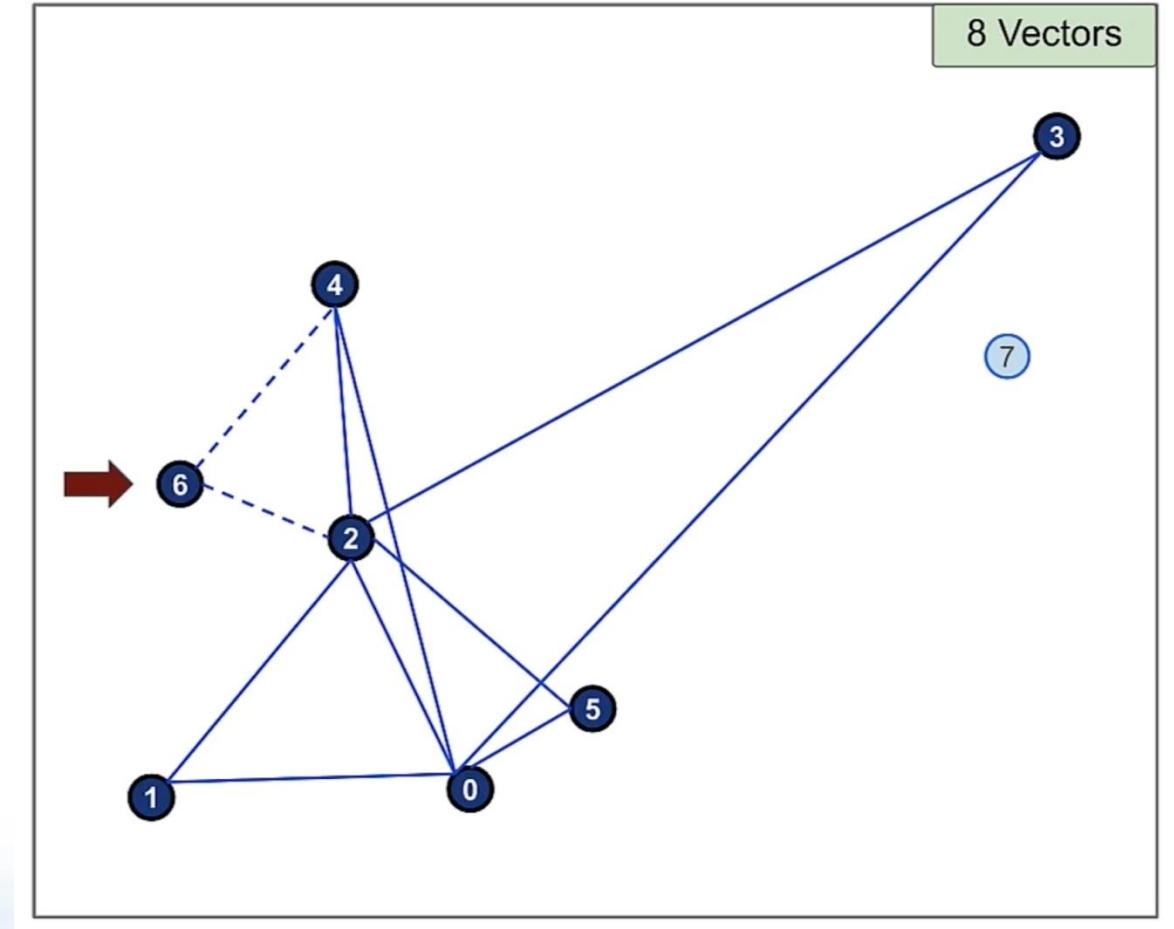


Navigate Small World (Constructing)



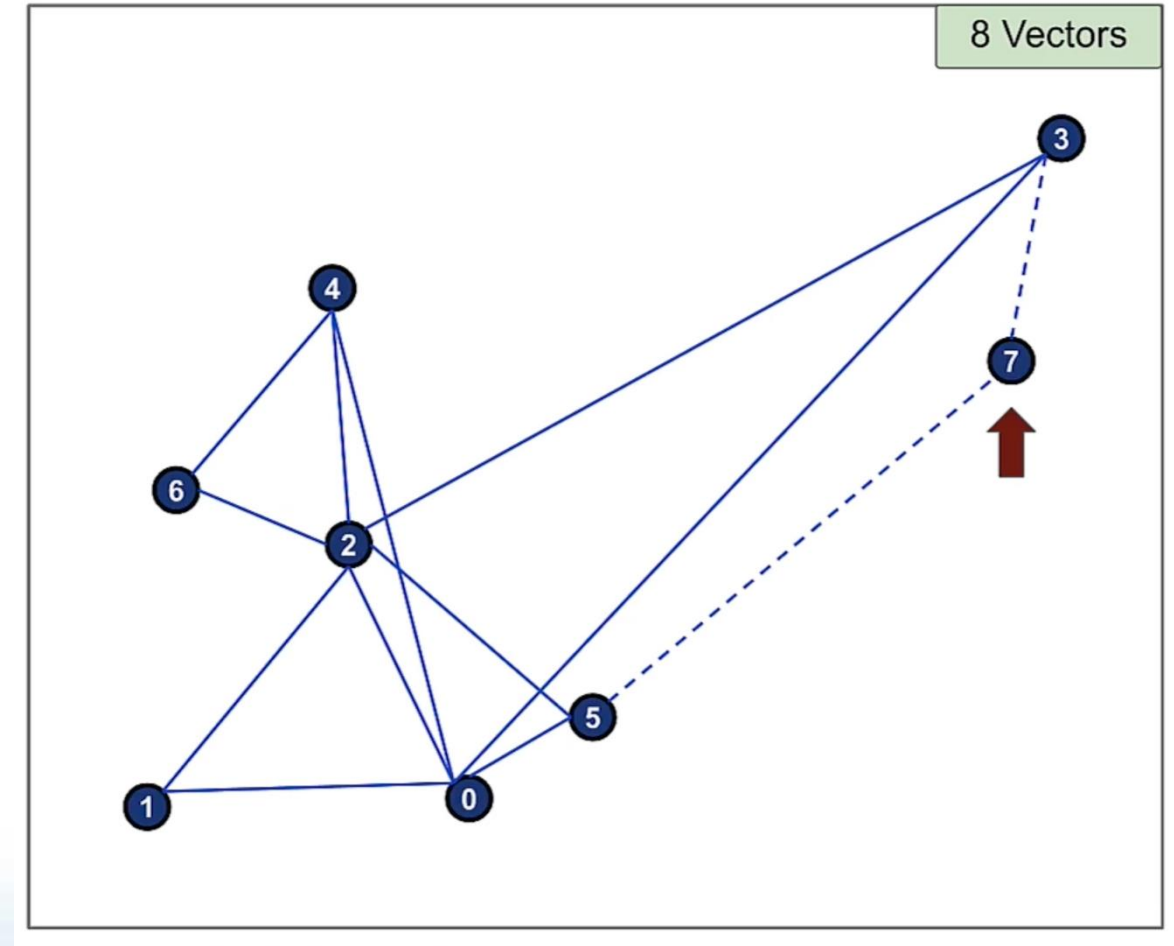


Navigate Small World (Constructing)



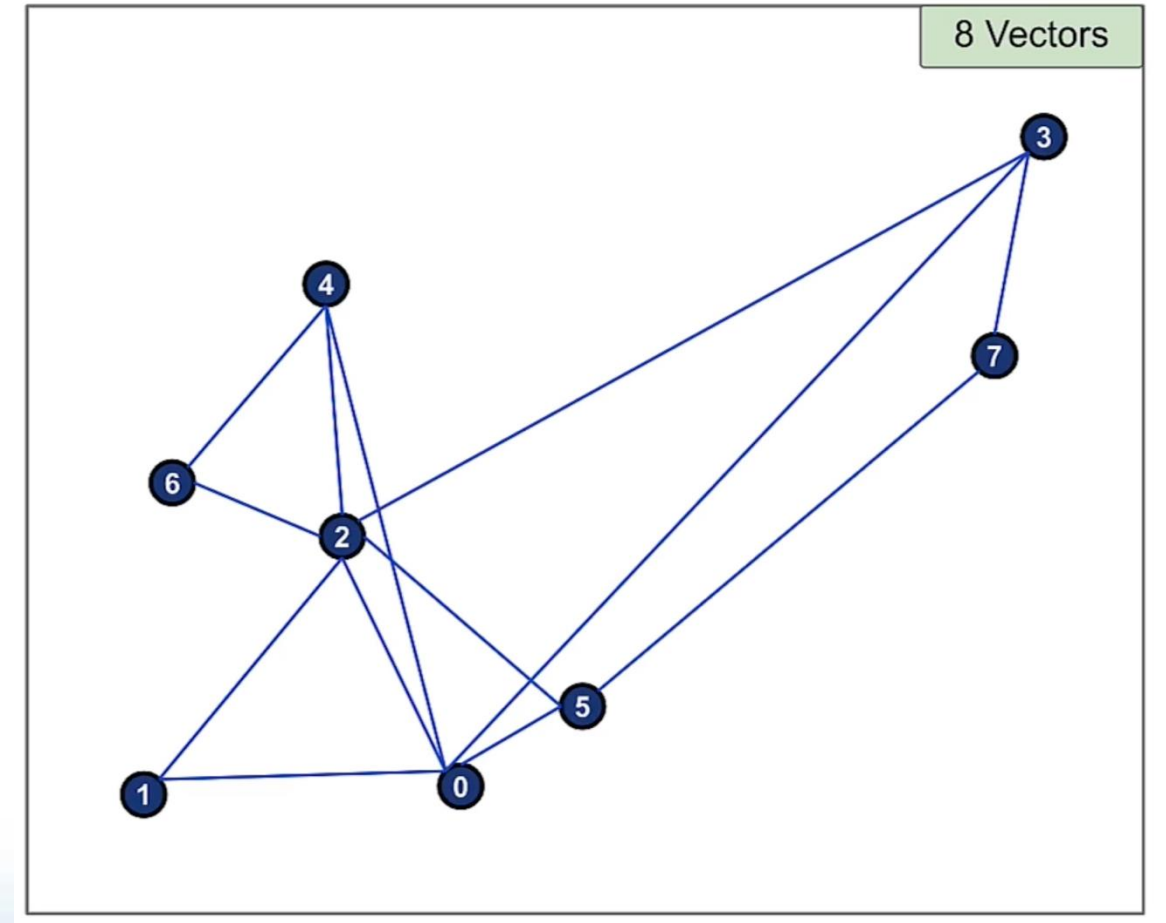


Navigate Small World (Constructing)





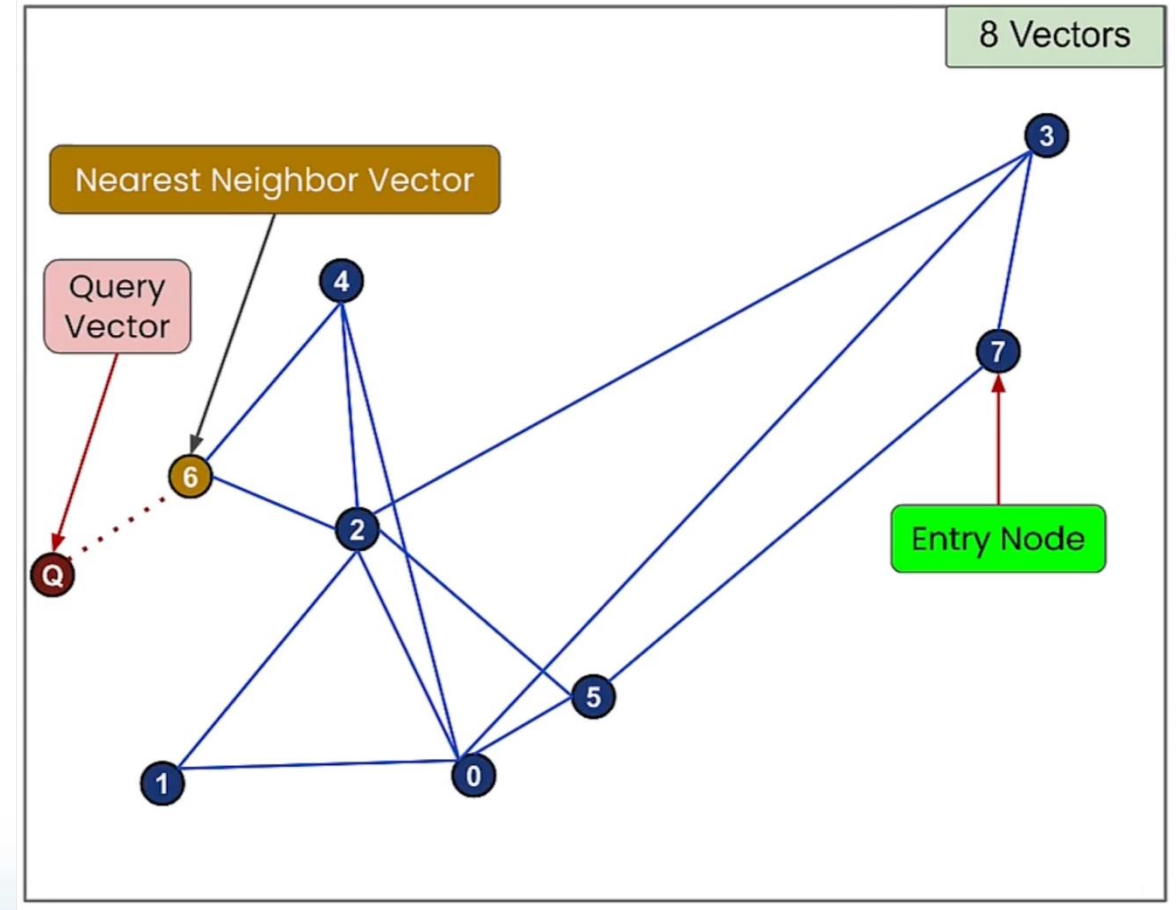
Navigate Small World (Constructing)





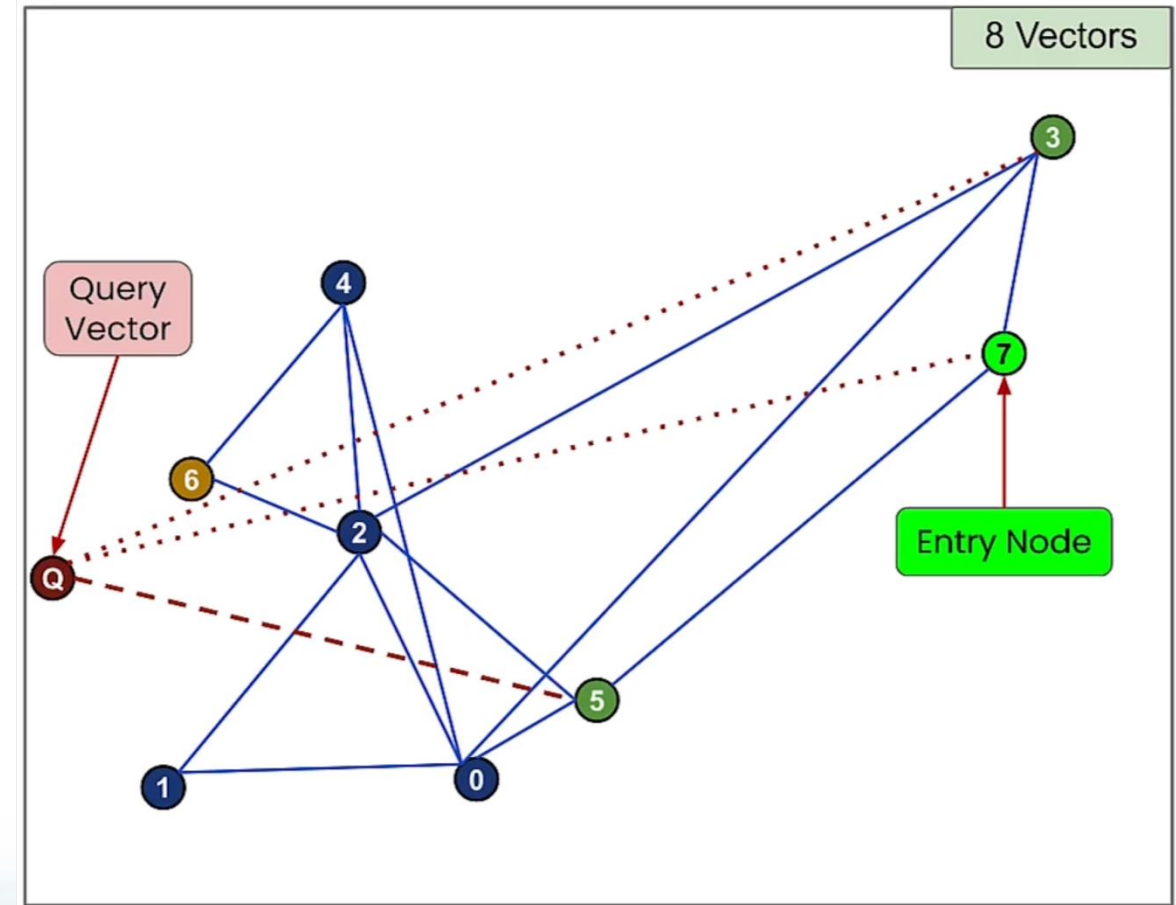
NSW - Search

1: Nearest Neighbor



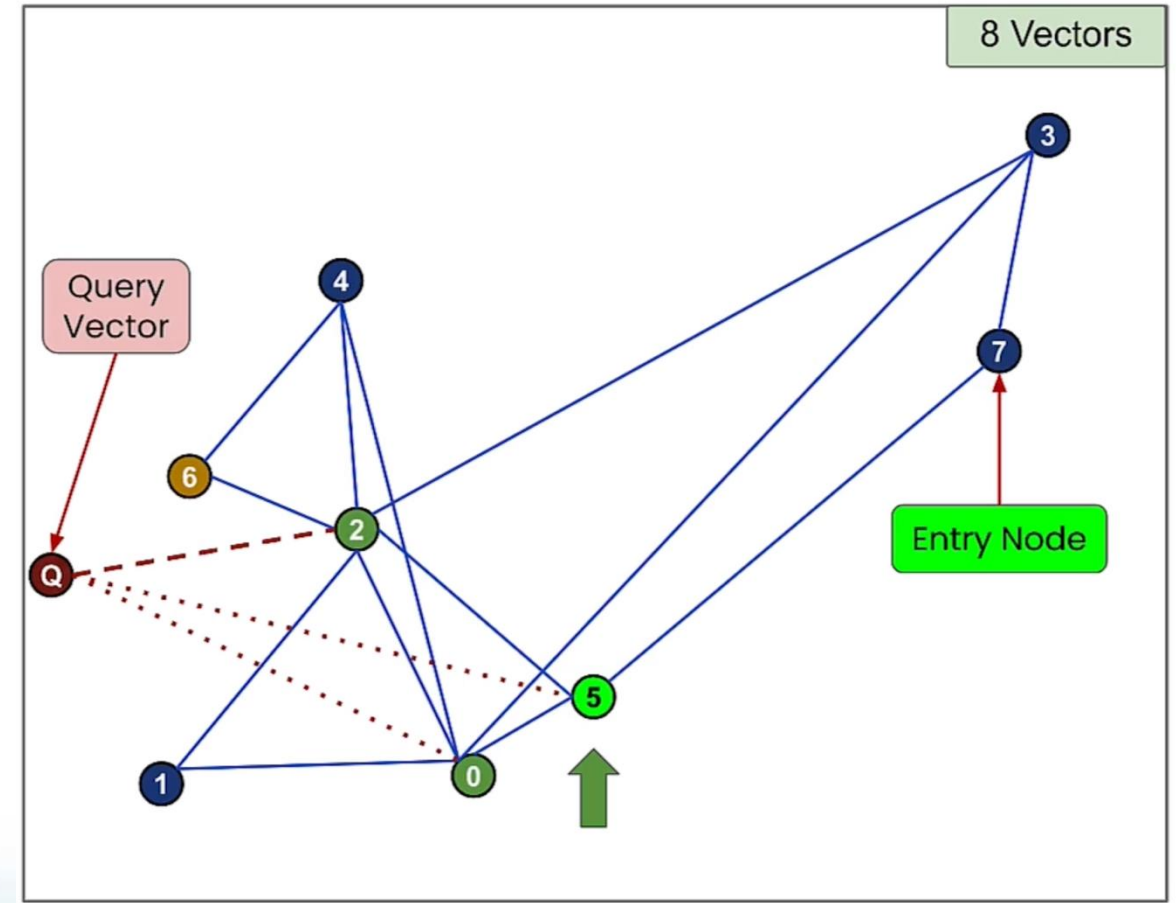


NSW - Search



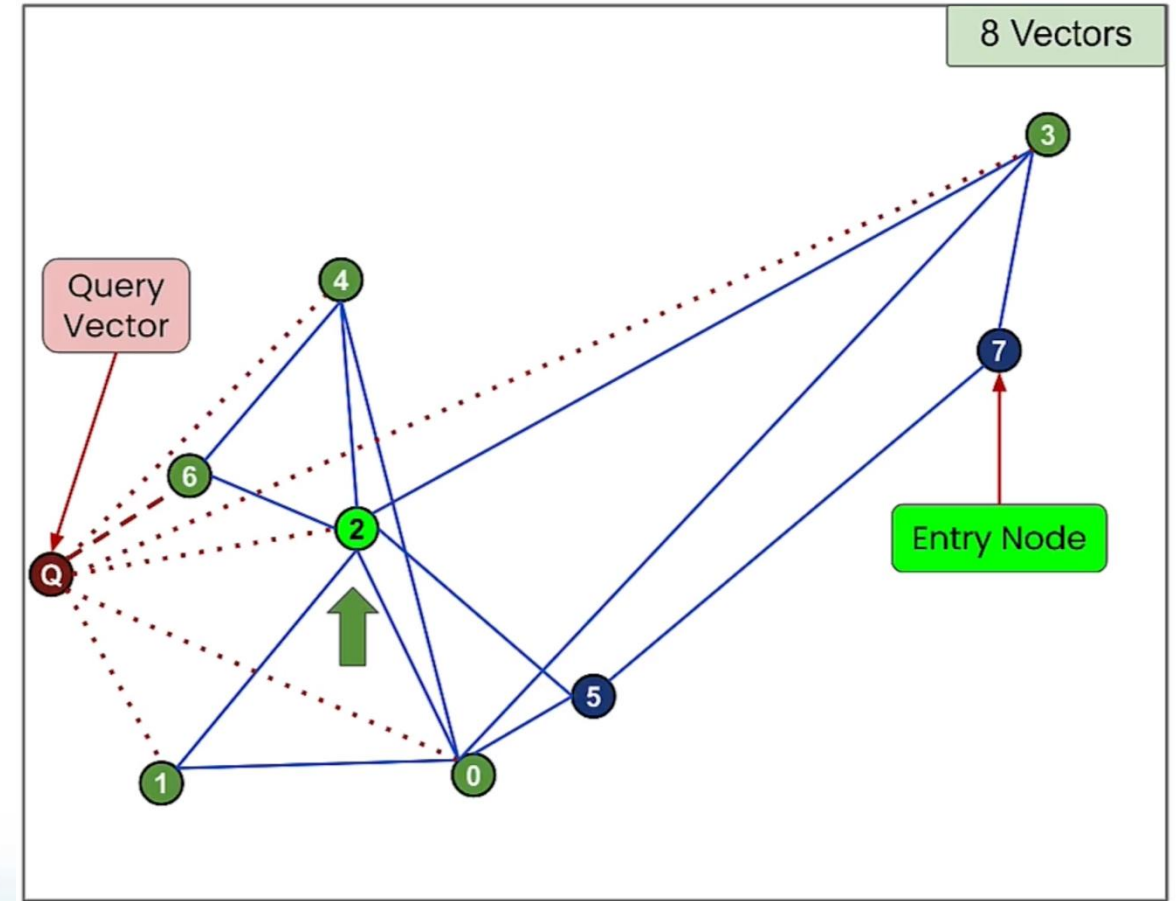


NSW - Search



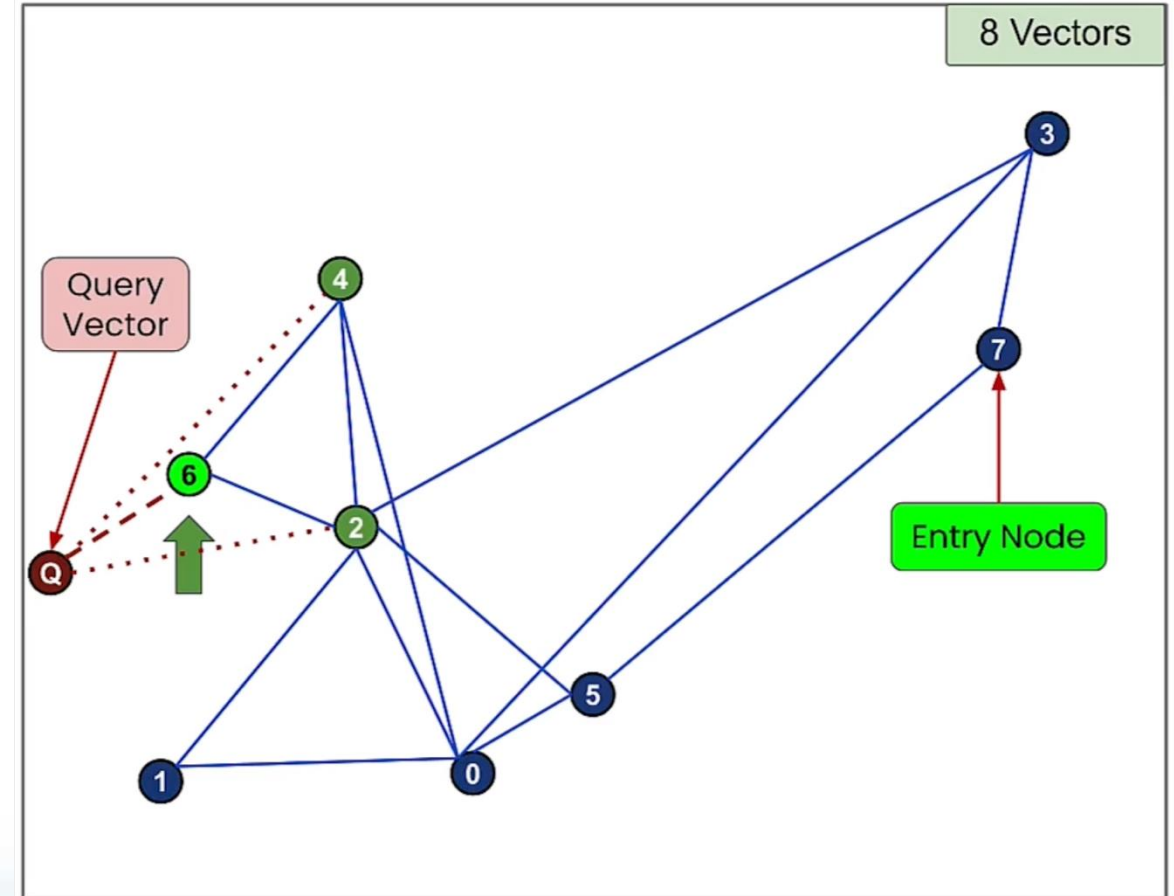


NSW - Search



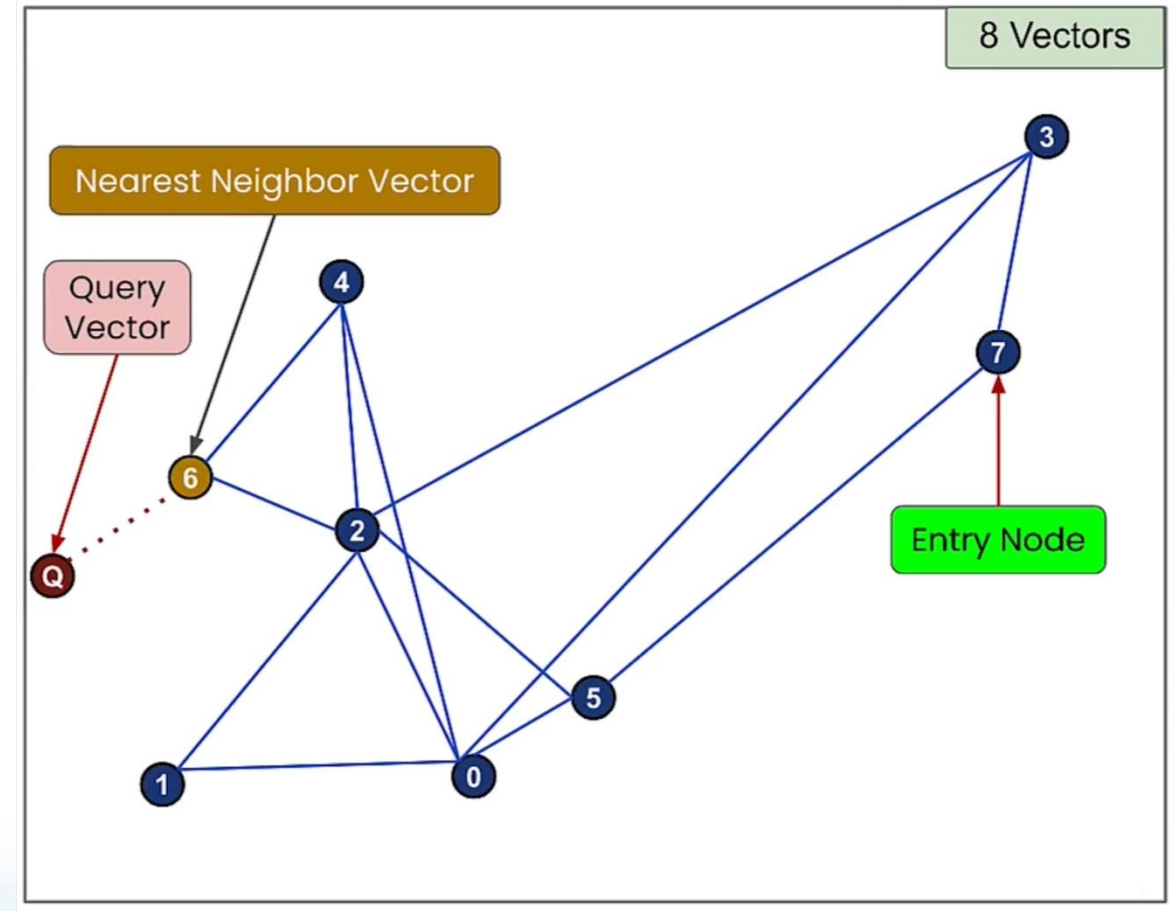


NSW - Search





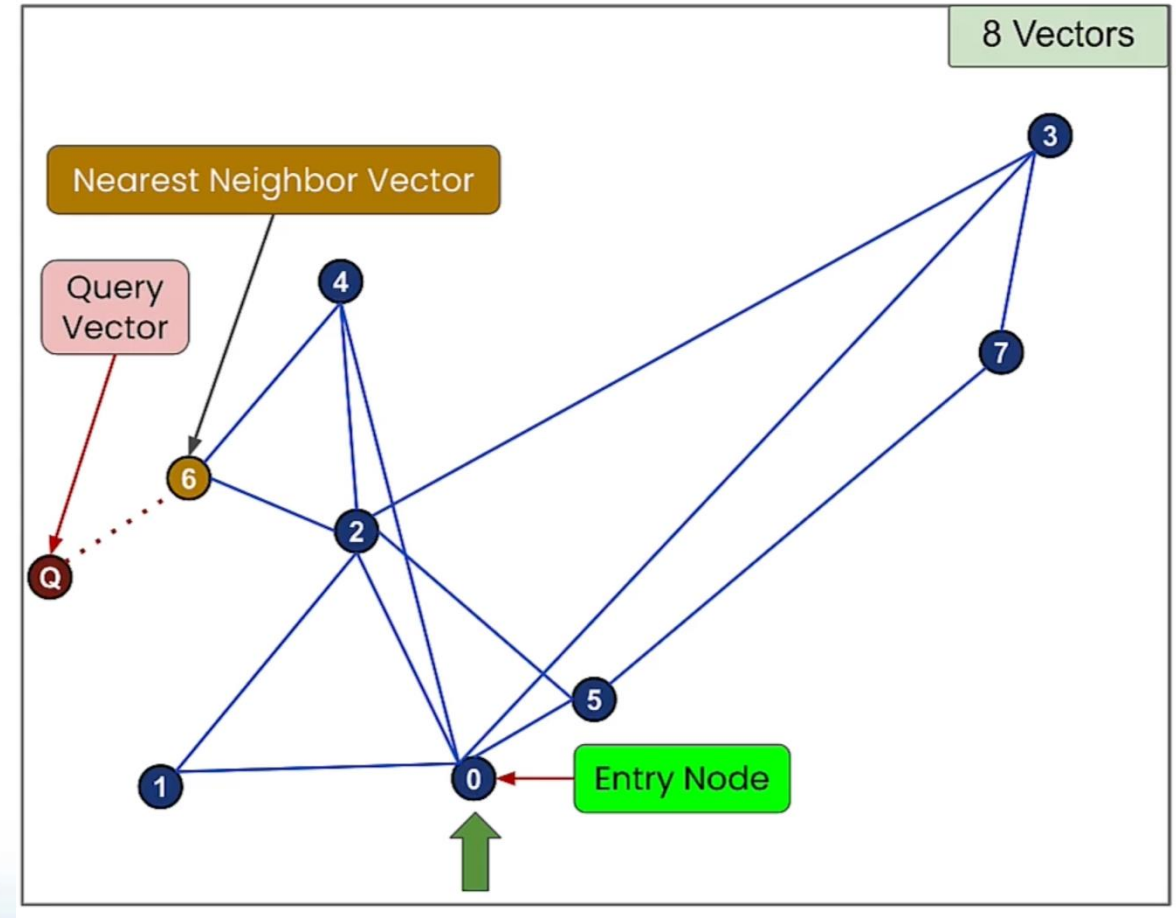
NSW - Search





NSW – Search 2

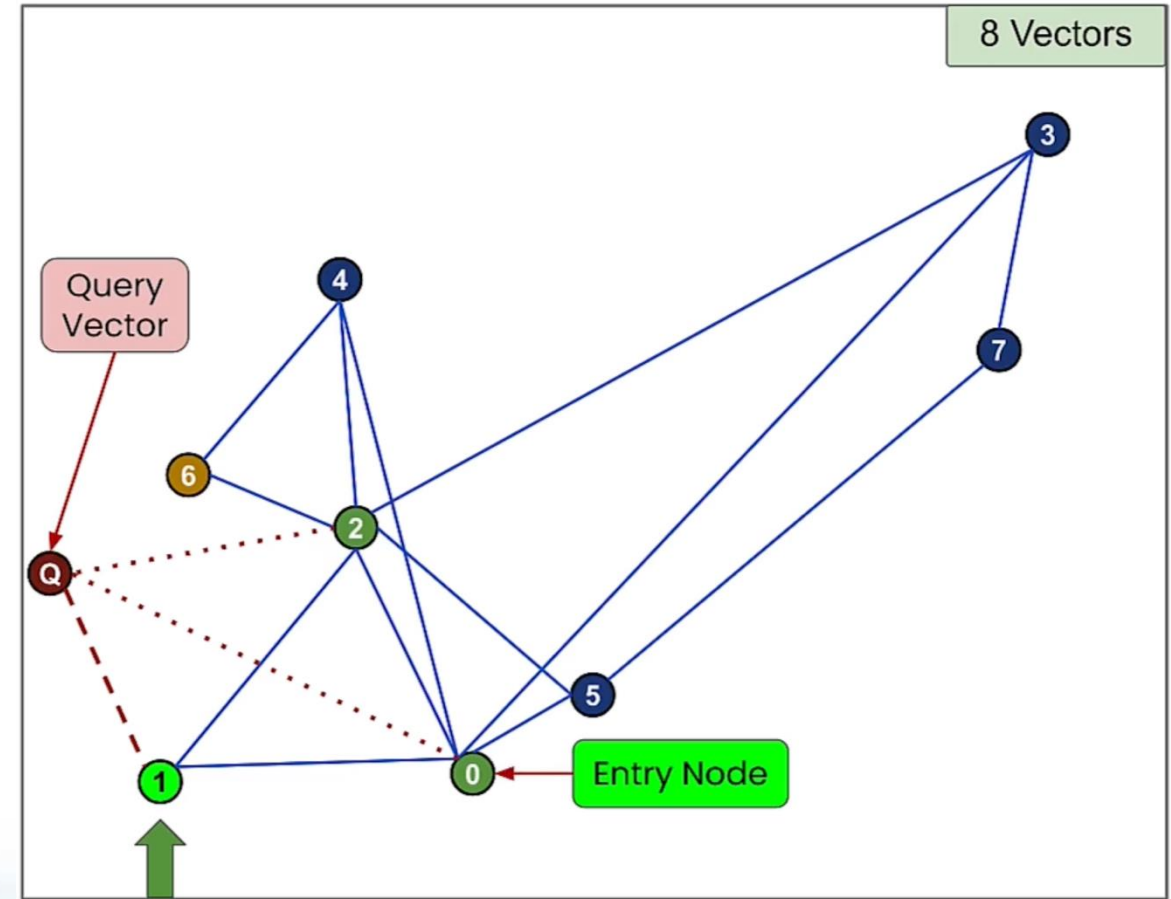
Approximate Nearest Neighbor





NSW – Search 2

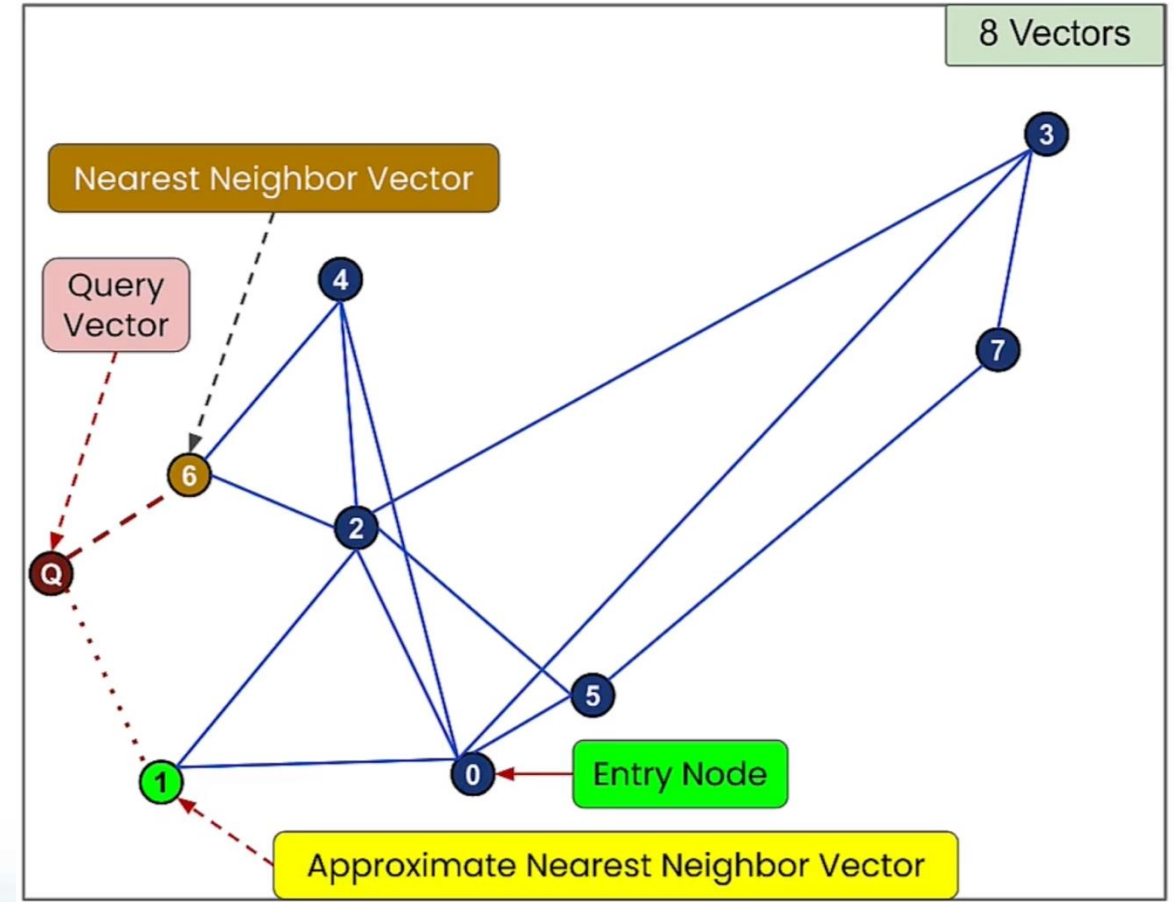
Approximate Nearest Neighbor





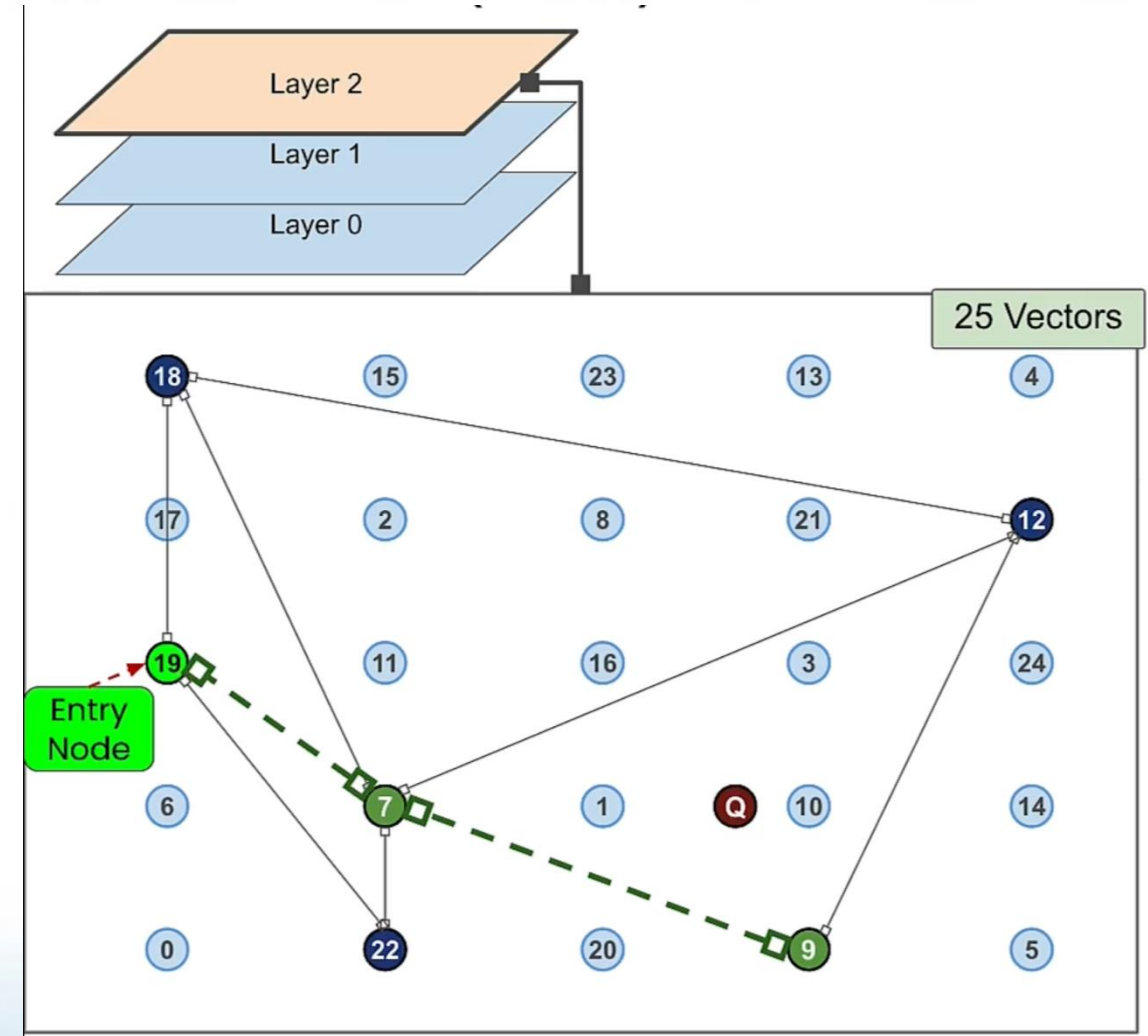
NSW – Search 2

Approximate Nearest Neighbor



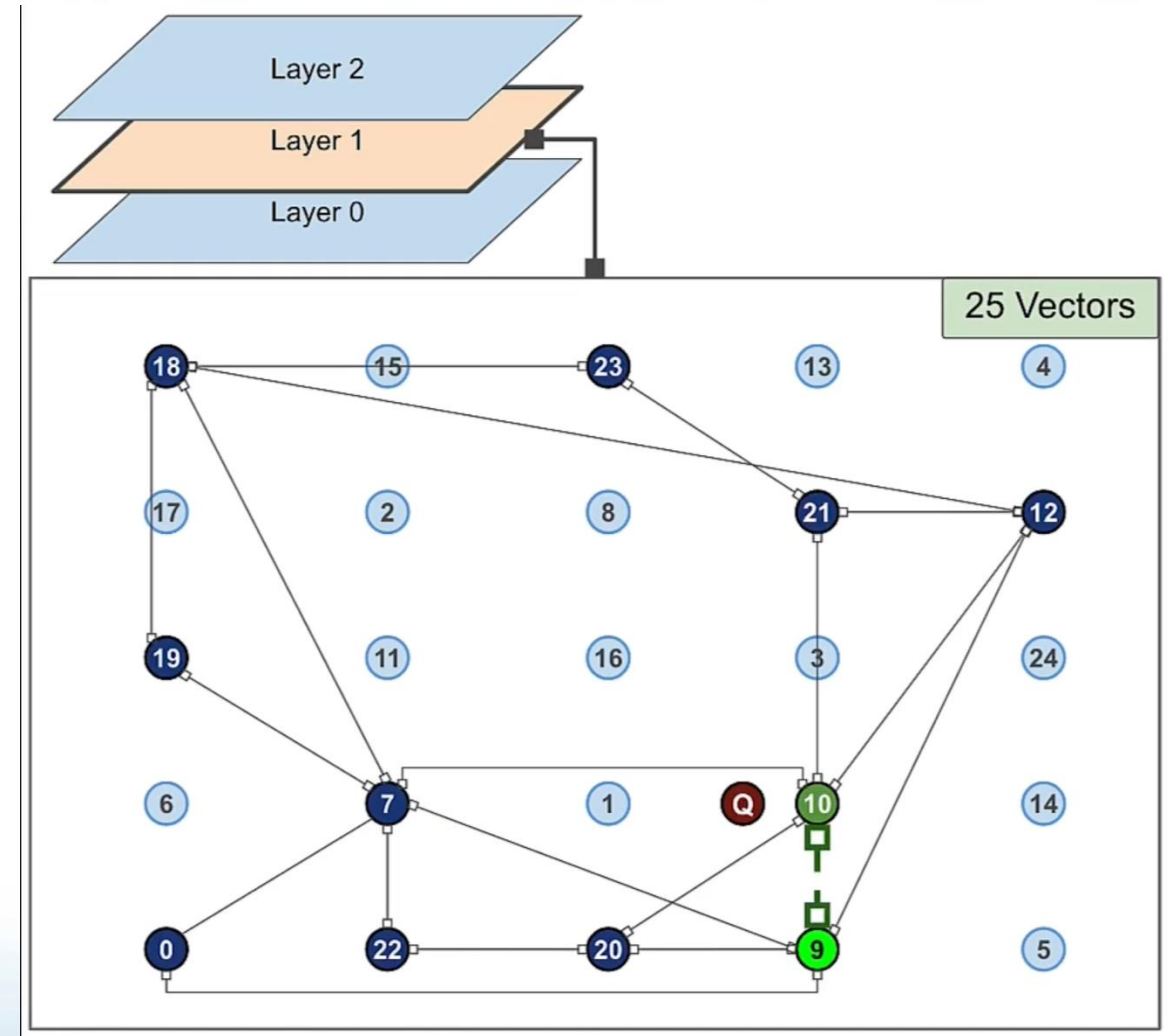


Hierarchical Navigable Small World (HNSW) - Search



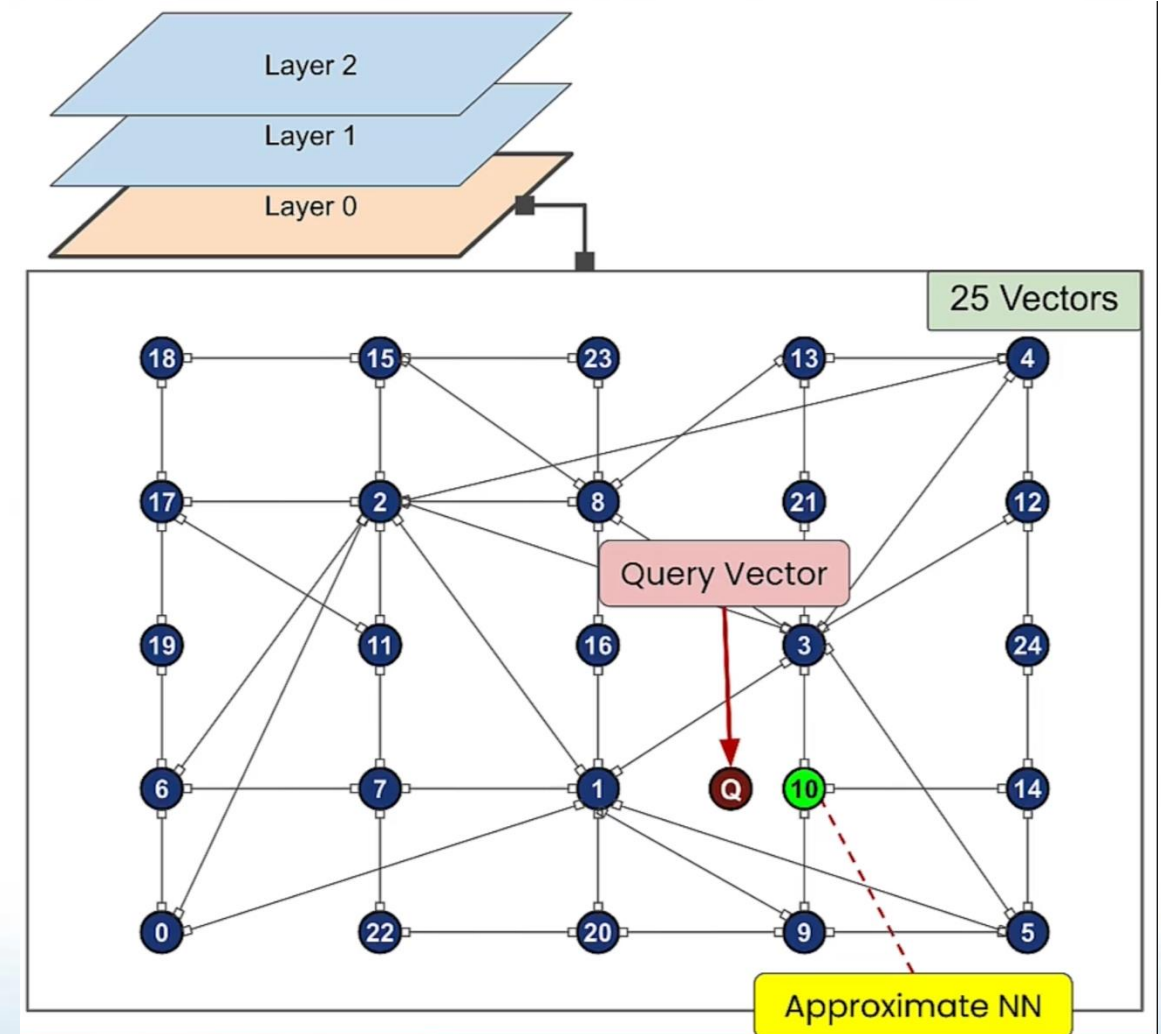


Hierarchical Navigable Small World (HNSW) - Search



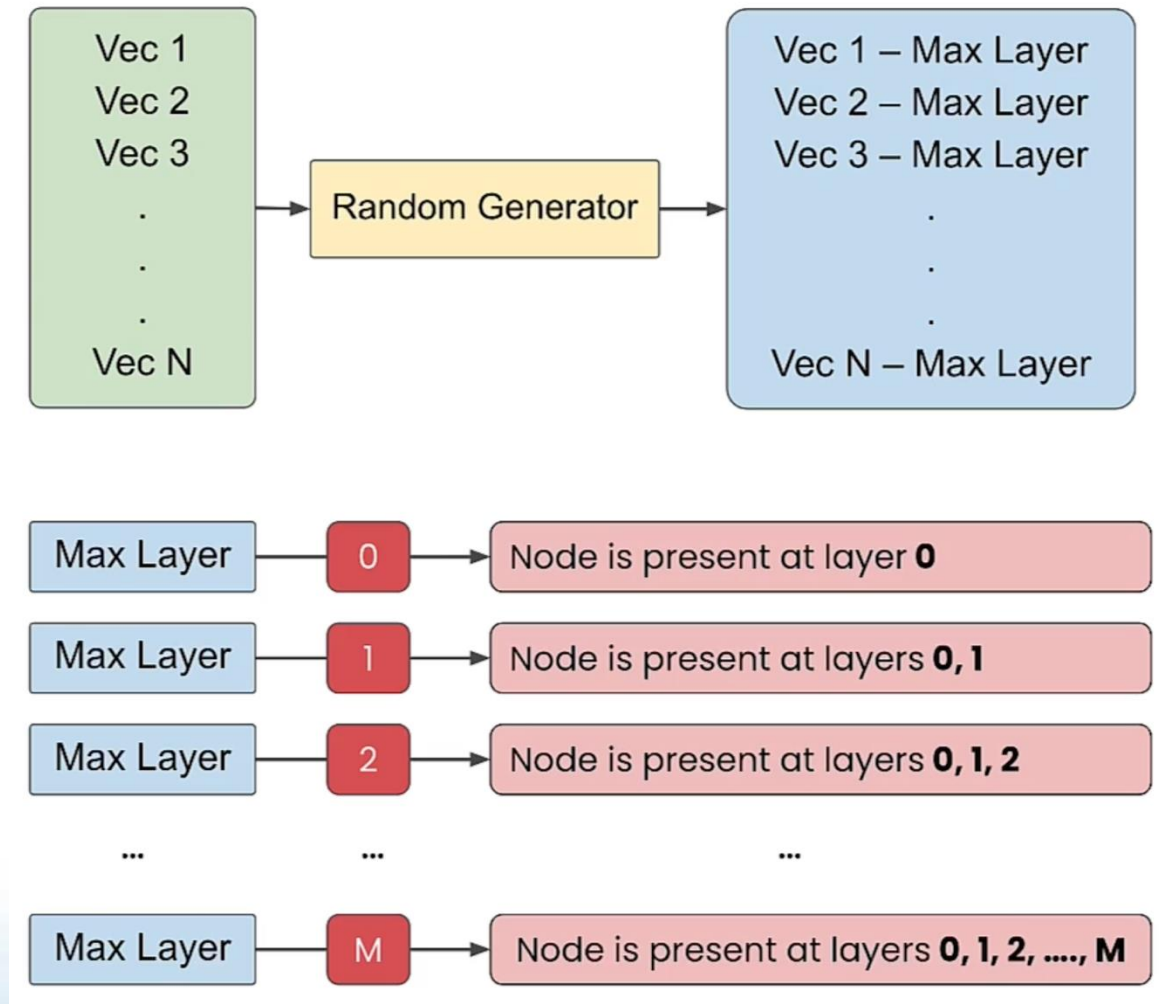


Hierarchical Navigable Small World (HNSW) - Search





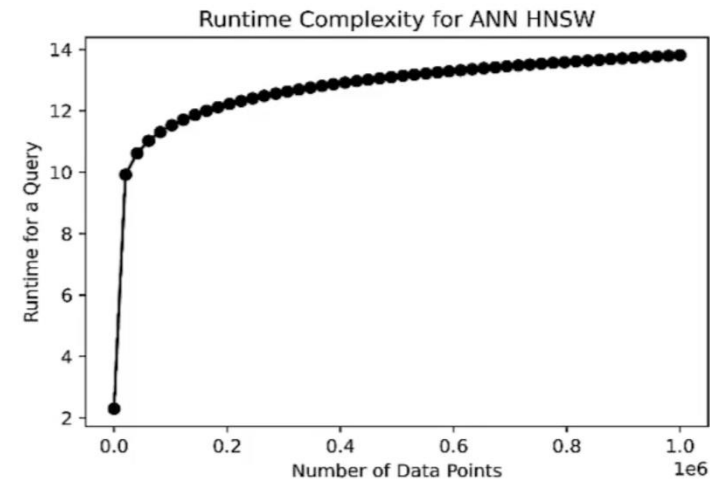
HNSW - Construction





HNSW Runtime

- **Low likelihood in Higher Levels**
 - The likelihood of a vector being found on the higher up levels of our HNSW graph goes down exponentially
- **Query time increases Logarithmically**
 - This means that as the number of datapoints increases the number of comparisons to perform vector search only goes up logarithmically.
- **$O(\log(N))$ runtime complexity**
 - As a results we have a $O(\log(N))$ runtime complexity for the HNSW algorithm

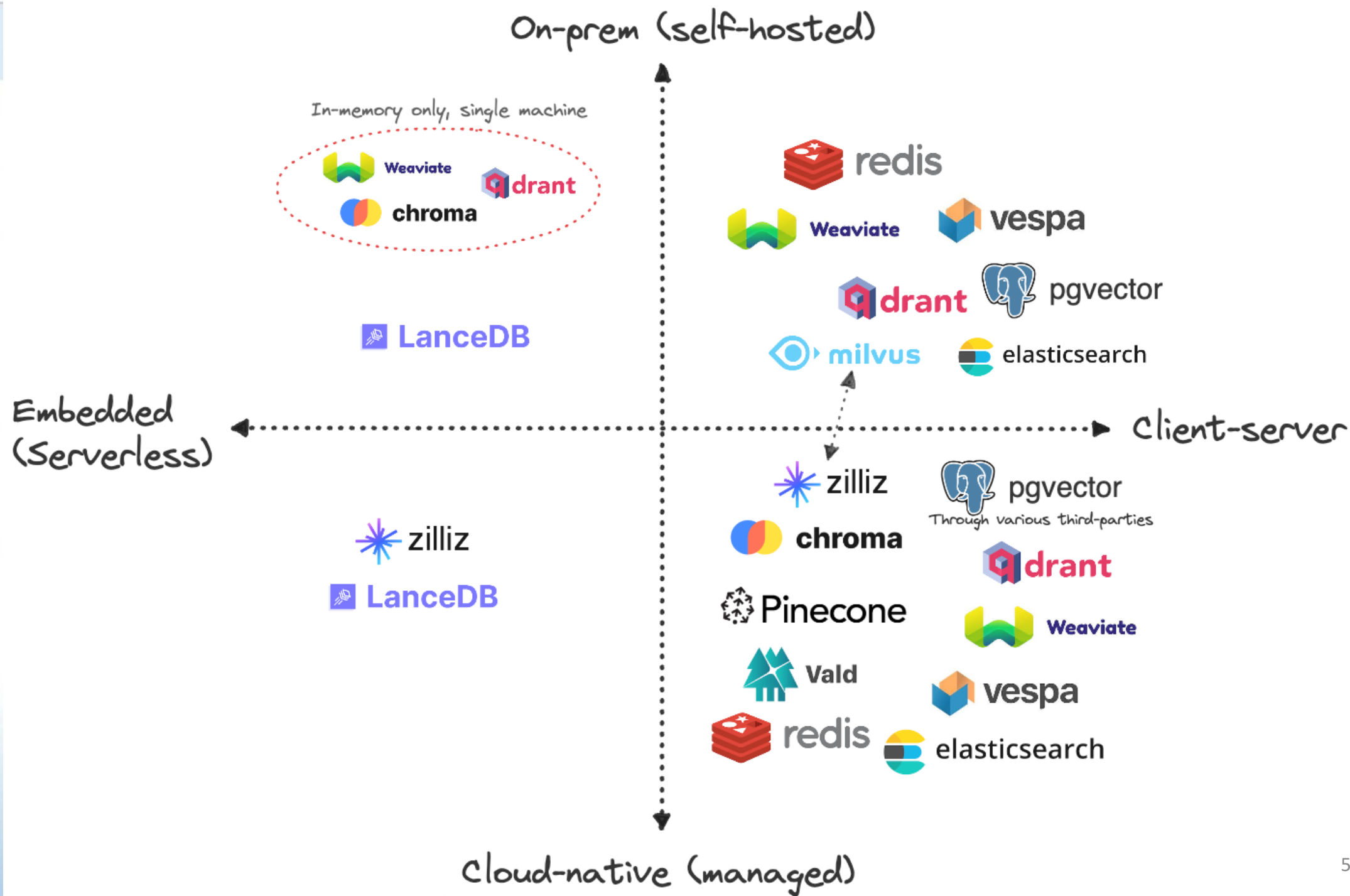




VectorDB

- A vector database is a specialized database designed to store, manage, and query high-dimensional vector data efficiently
 - Optimized for storing and retrieving vector embeddings, which are crucial for AI and machine learning applications
- Why Vector Databases are Useful for Dense Retrieval and RAG
 - Efficient storage and retrieval of high-dimensional data
 - Optimized for similarity search operations
 - Scalability to handle large datasets
 - Low-latency queries for real-time applications
 - Seamless integration with AI and machine learning workflows





Indexing

Hash-based

Spherical
hashing

Spectral
hashing

LSH
(Locally
sensitive
hashing)

Tree-based

NGT
(Neighbourhood Graph
and Tree)

Trinary
projection
trees

DT-ST

Annoy

Graph-based

Scalable
kNN graph

HNSW

Vamana

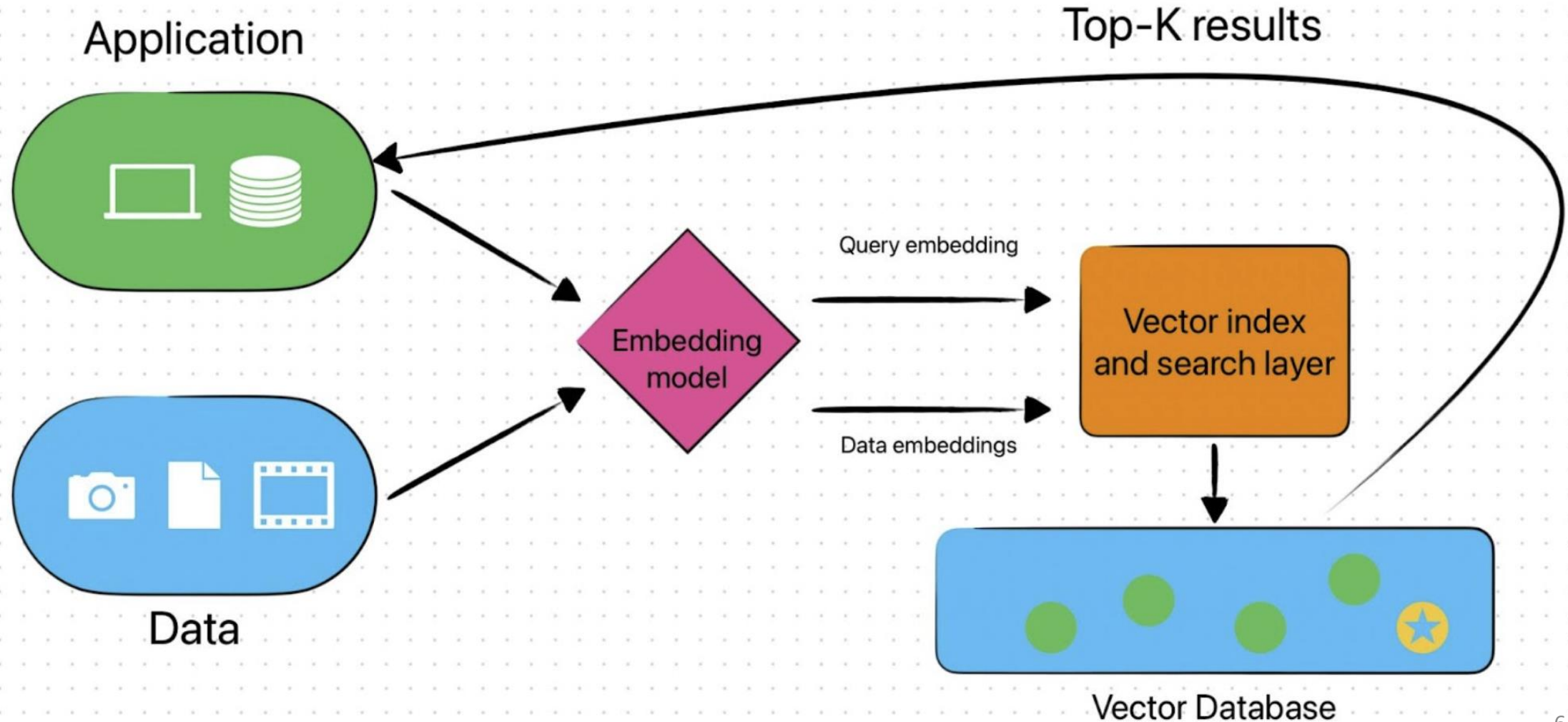
Inverted file

IVMF
(Inverted multi-
index file)

IVF
(Inverted file)



RAG





Sparse vs Dense Search

- **Dense Search (Semantic Search)**
 - Uses vector embedding representation of data to perform search.
(as described in the previous lessons)
 - This types of search allows one to capture and return semantically similar objects.



"Baby dogs"



"Here is content on puppies!"



Dense Search Challenge

- **Product with a Serial Number**

- Searching for seemingly random data (like serial numbers) will also yield poor results.



"BB43300"



"Errrm... Bumble Bee?"

- **String or Word Matching**

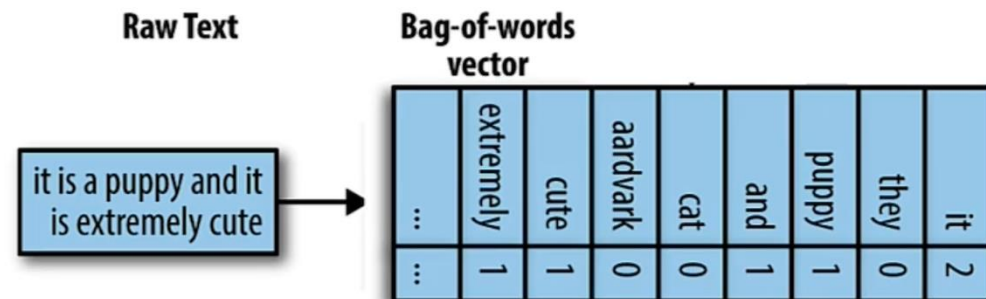
- Here we would be better off doing exact string or word matching to see where which product has the same serial number as the input serial number.

- This is known as **keyword search or sparse search!**



Sparse vs Dense Search

- **Bag of Words**
 - The easiest way to do keyword matching is using Bag of Words - to count how many times a word occurs in the query and the data vector and then return objects with the highest matching word frequencies.
- **This is known as Sparse Search**
 - because the text is embedded into vectors by counting how many times every unique word in your vocabulary occurs in the query and stored sentences.
- **Mostly Zeroes (Sparse Embedding)**
 - Since the likelihood of any given sentence containing every word in your vocabulary is quite low the embeddings is mostly zeroes and thus is known as a sparse embedding.





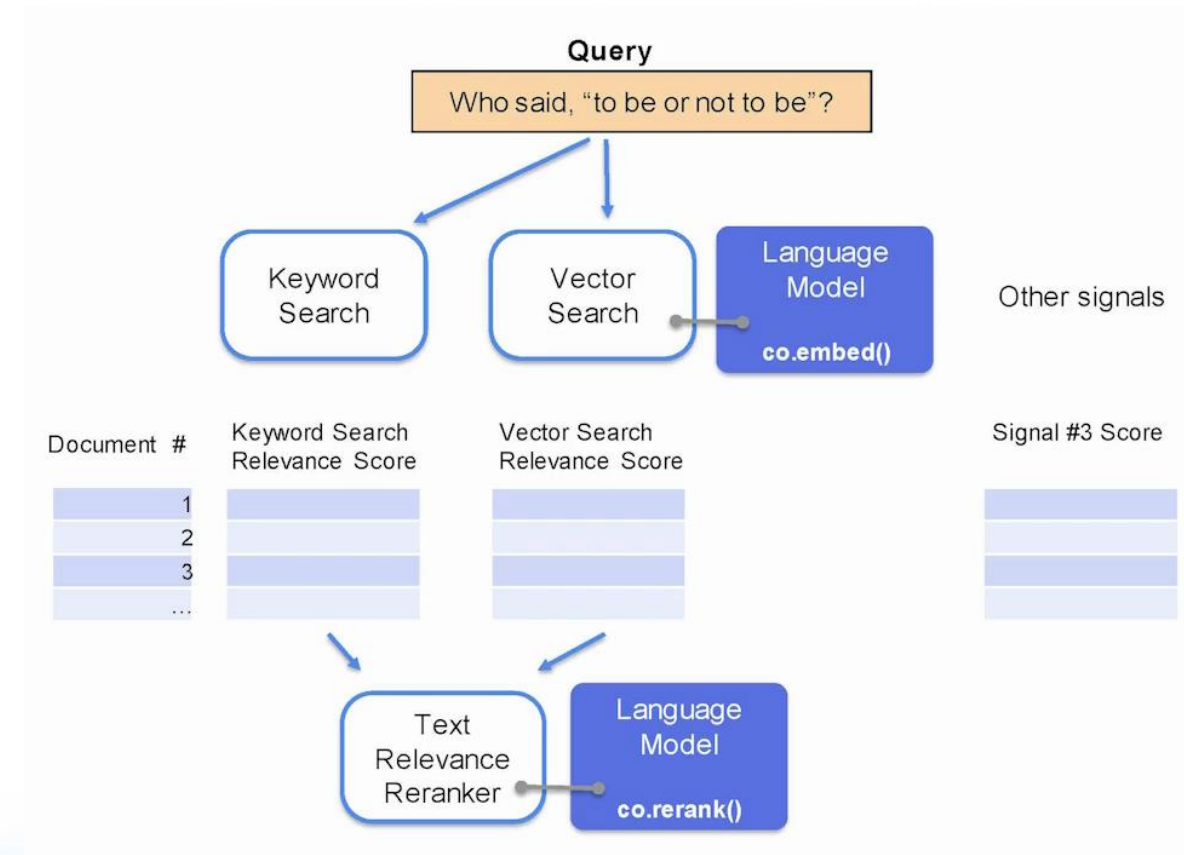
Hybrid Search

- **What is Hybrid Search?**
 - Hybrid search is the process of performing both vector/dense search and keyword/sparse search and then combining the results
- **Combination based on a Scoring System**
 - This combination can be done based on a scoring system that measures how well each objects matches the query using both dense and sparse searches.



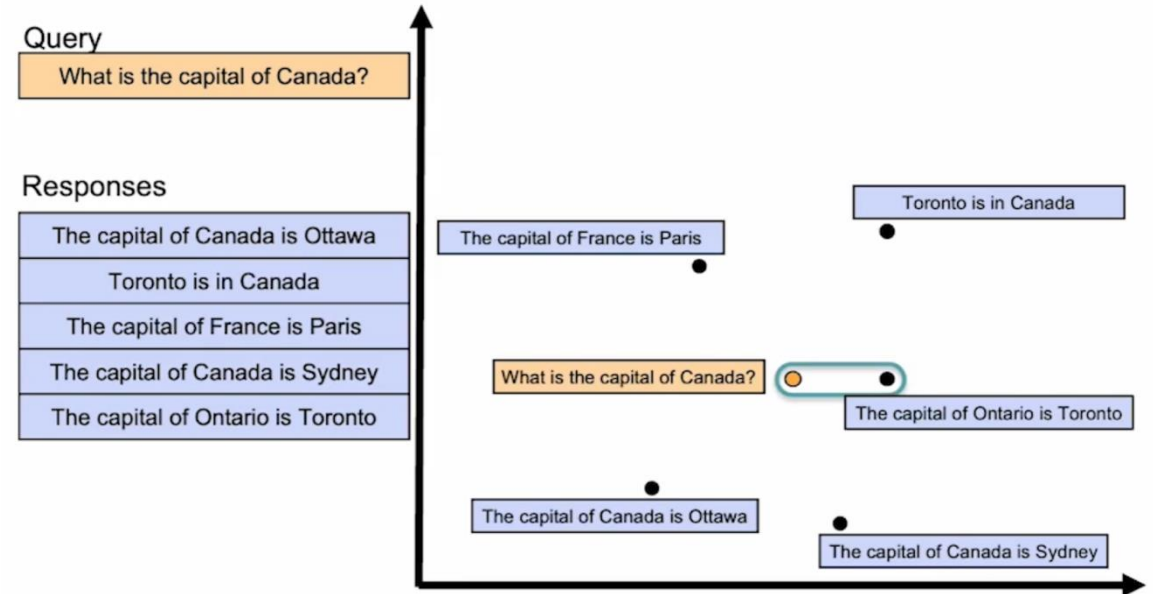


Hybrid Search





Dense Retrieval is not Perfect





Solution: ReRank

Query

What is the capital of Canada?

Top Responses

Europe is a continent
The capital of France is Paris
The grass is green
The sky is blue
Toronto is in Canada
Tomorrow is Sunday
The capital of Canada is Ottawa
The capital of Canada is Sydney
Most apples are red
The capital of Ontario is Toronto

The capital of France is Paris
Toronto is in Canada
The capital of Canada is Ottawa
The capital of Canada is Sydney
The capital of Ontario is Toronto

ReRank

Relevance

0.2
0.3
0.9
0.6
0.5





ReRank is trained on

Many queries with correct answers:

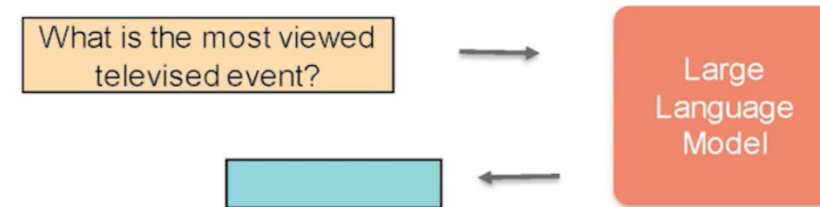
What is the capital of Canada?	The capital of Canada is Ottawa
What is the capital of France?	The capital of France is Paris
.	.
.	.
.	.
What color is the sky?	The sky is blue

Many queries with wrong answers:

What is the capital of Canada?	Toronto is in Canada
What is the capital of France?	The capital of France is Île-de-France
.	.
.	.
.	.
What color is the sky?	The sky is red



Search can help LLMs in multiple ways

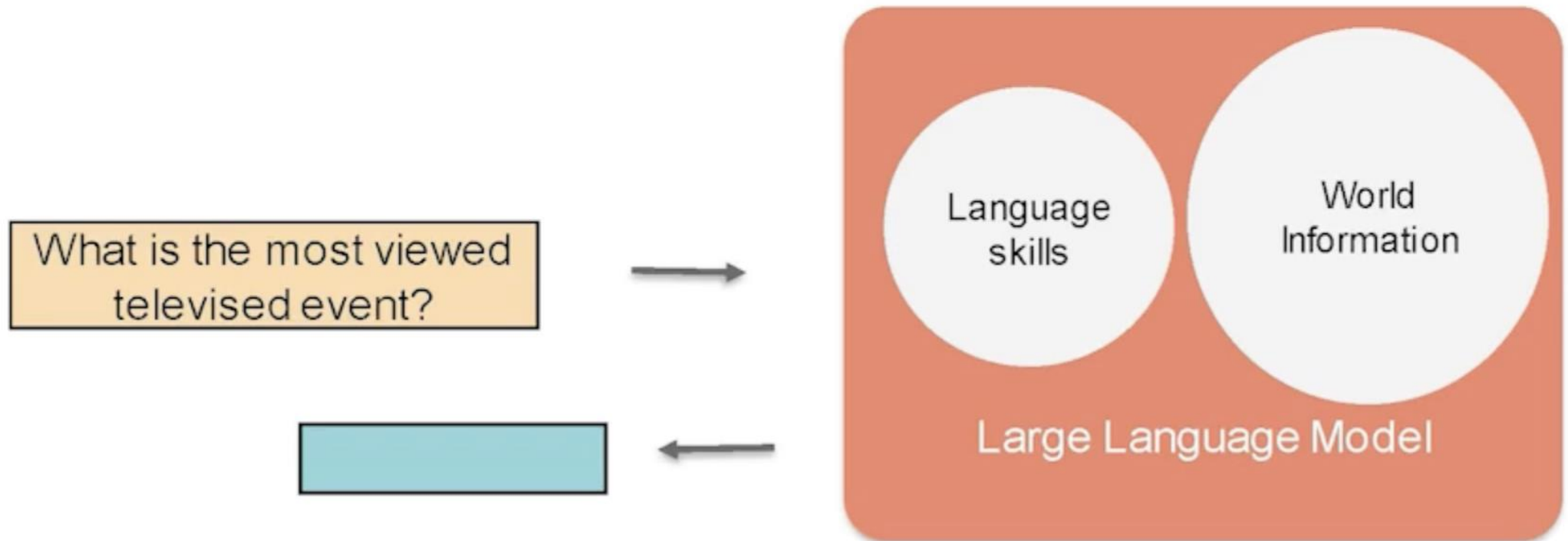


Retrieval can help LLMs with:

- Factual information
- Private information
- Updated information

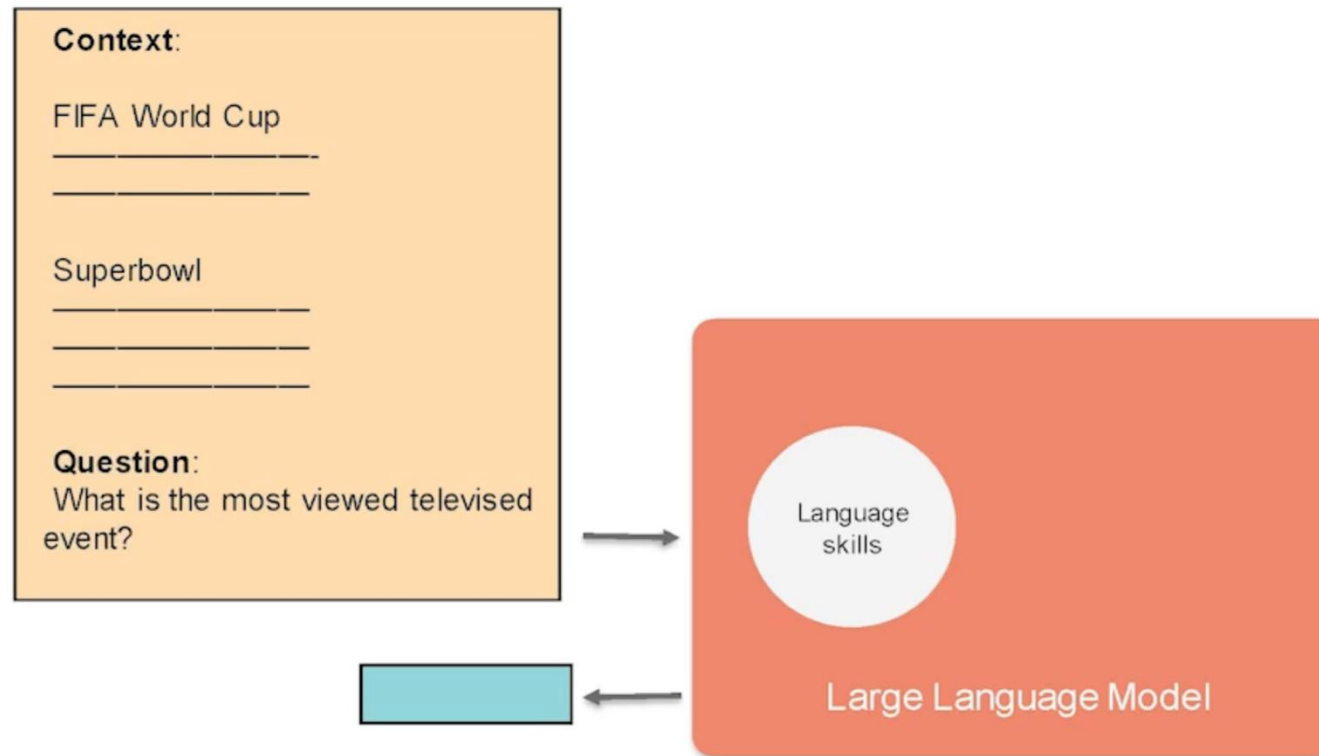


Where is the information stored?





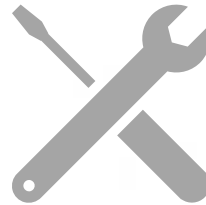
Search can add some context





Agents

Are a combination of LLMs and code
LLMs reason about what steps to take
and call for actions.



Agent loop

Choose a tool to use
Observe the output of the tool
Repeat until a stopping condition is
met

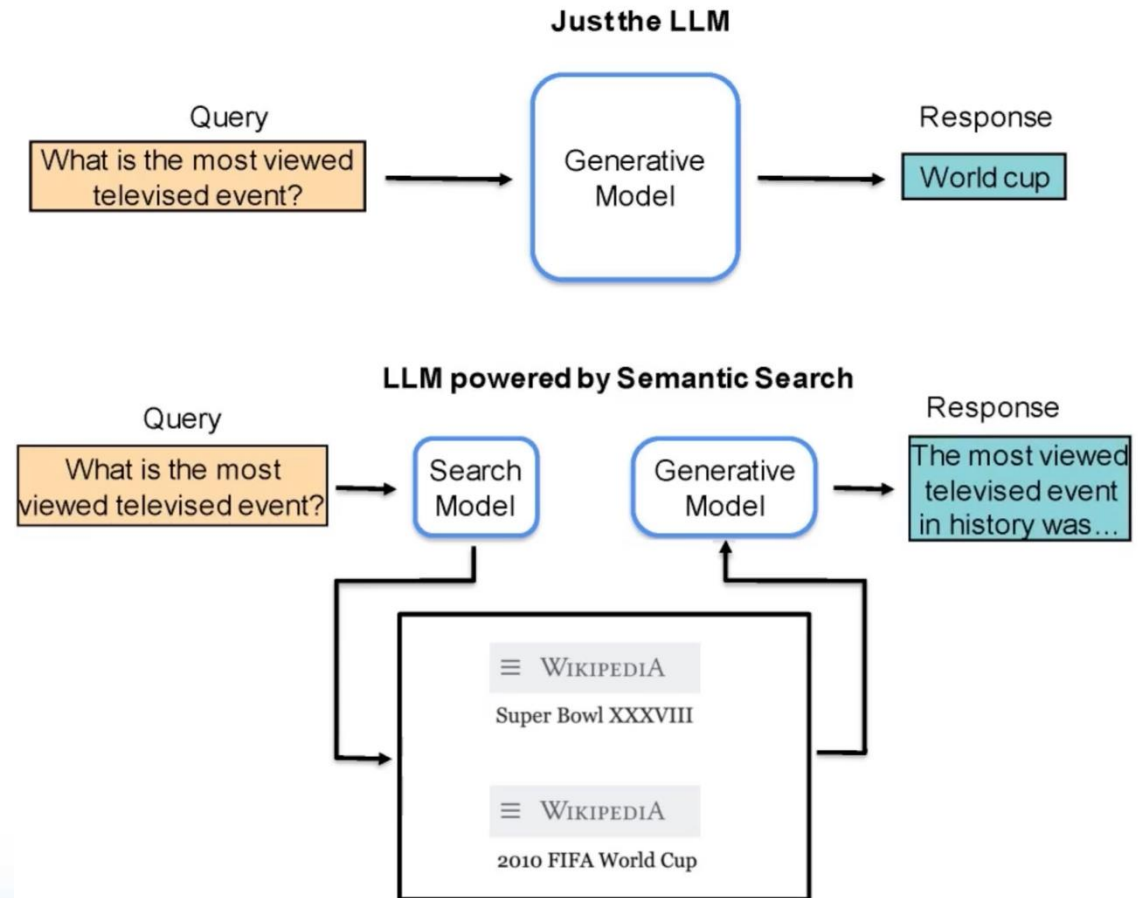


Stopping conditions can be:

LLM determined
Hardcoded rules



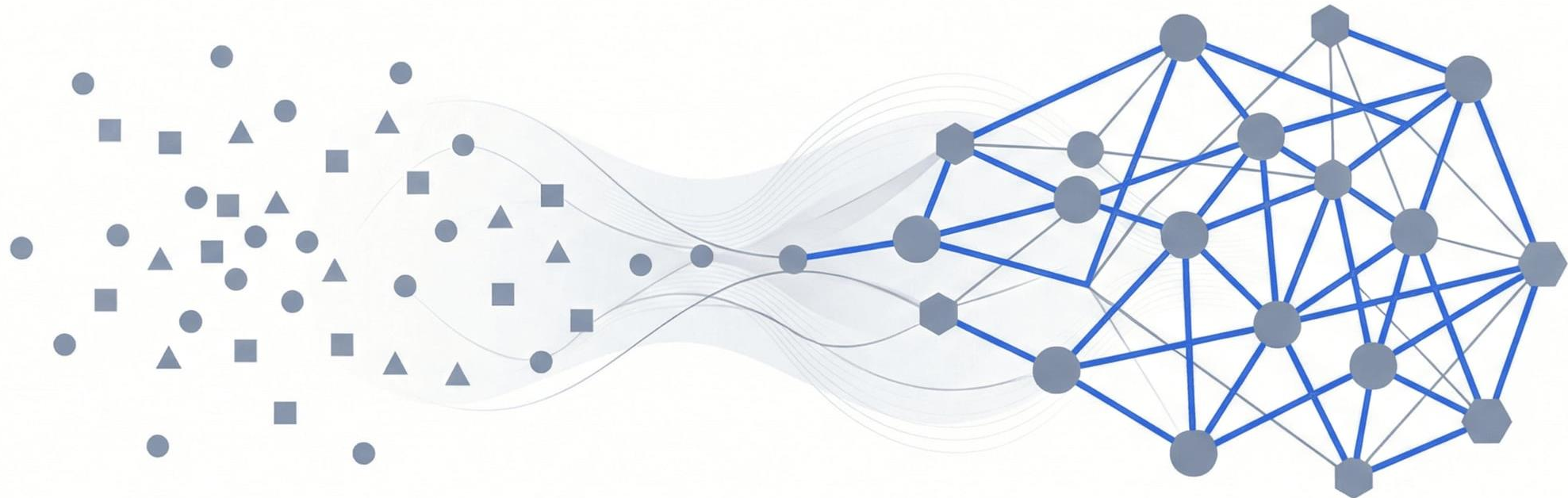
Generating Answers





RAG and GraphRAG: The Evolution of AI-Powered Information Retrieval

From Vector Search to Knowledge Graphs



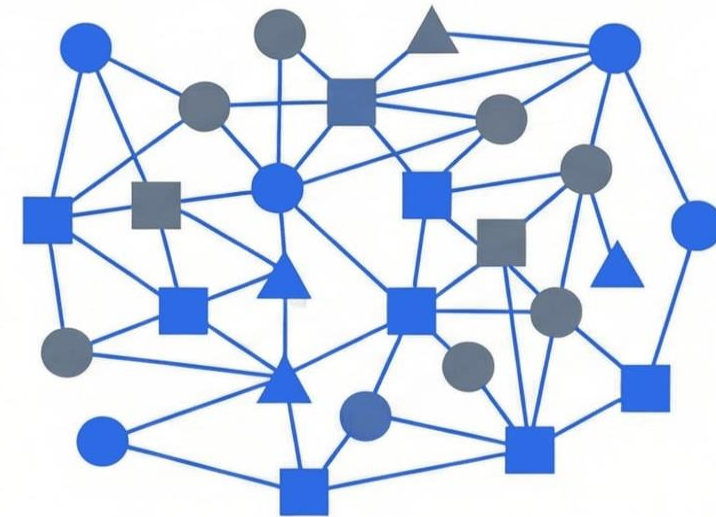


From Vector Search to Knowledge Graphs: The Evolution of RAG

Baseline RAG



GraphRAG

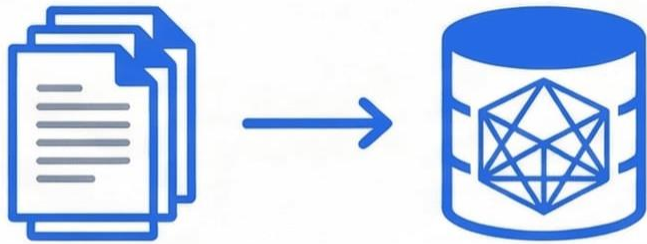


- RAG enhances LLMs by connecting to external knowledge.
- Baseline RAG uses vector search on isolated text chunks.
- GraphRAG leverages knowledge graphs to map relationships.



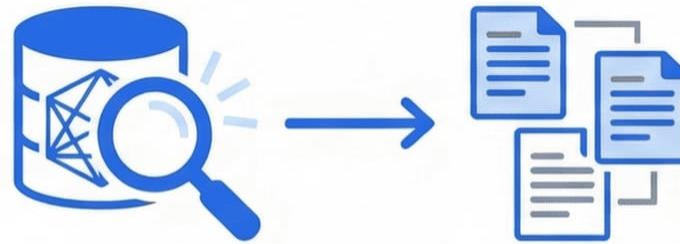
The Mechanics of Baseline RAG: A Three-Phase Process

1. Extraction



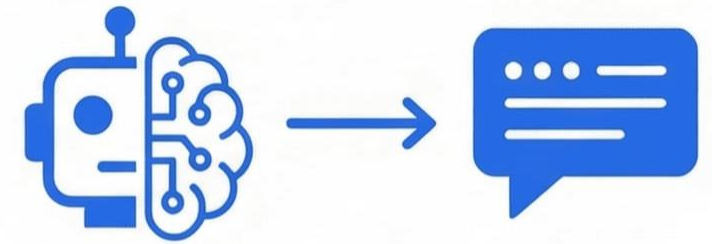
- Chunking & Embedding

2. Retrieval



- Semantic Search

3. Generation



- Context-Aware Response



The Shortcomings of Vector Search in Complex Scenarios

Limitations of Baseline RAG

- **Inability to perform multi-hop reasoning:** Struggles to connect disparate pieces of information.
- **Lack of explainability:** Vector representations are "black boxes," making it hard to trust results.
- **Fragmented answers:** Retrieves isolated chunks without surrounding context.

Multi-hop Failure



Opaque Vector
Embedding

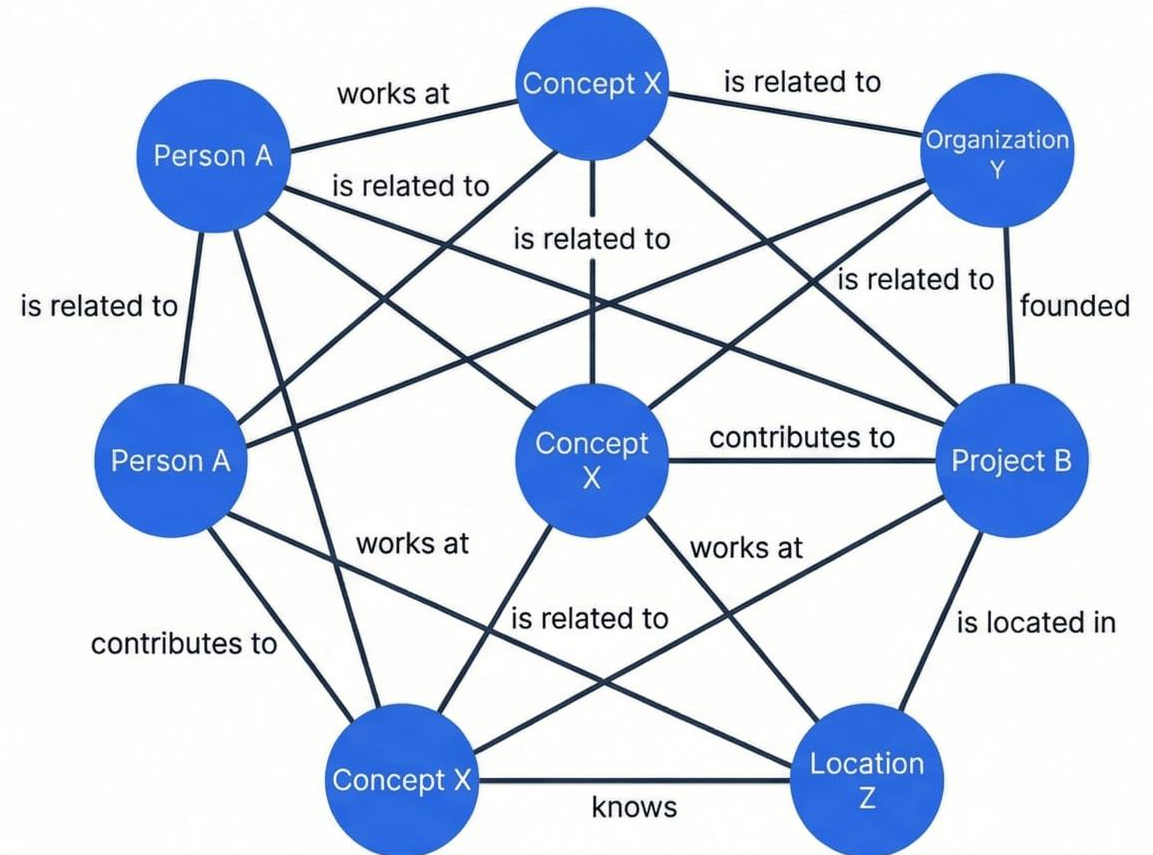
Hidden Internal Representation
(No Explainability)



Introducing GraphRAG: Retrieval Through Relationships

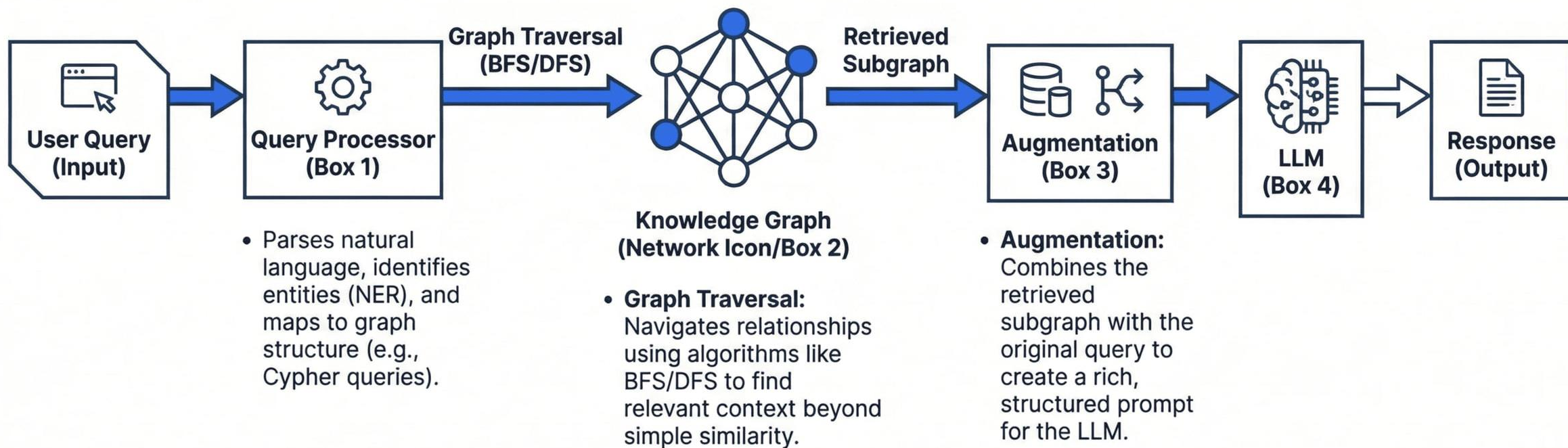
GraphRAG uses knowledge graphs to represent data and its relationships.

- Nodes represent entities (e.g., People, Concepts).
- Edges represent the relationships between them.
- Enables more sophisticated, context-aware retrieval.





The GraphRAG Architecture: A Deeper Dive





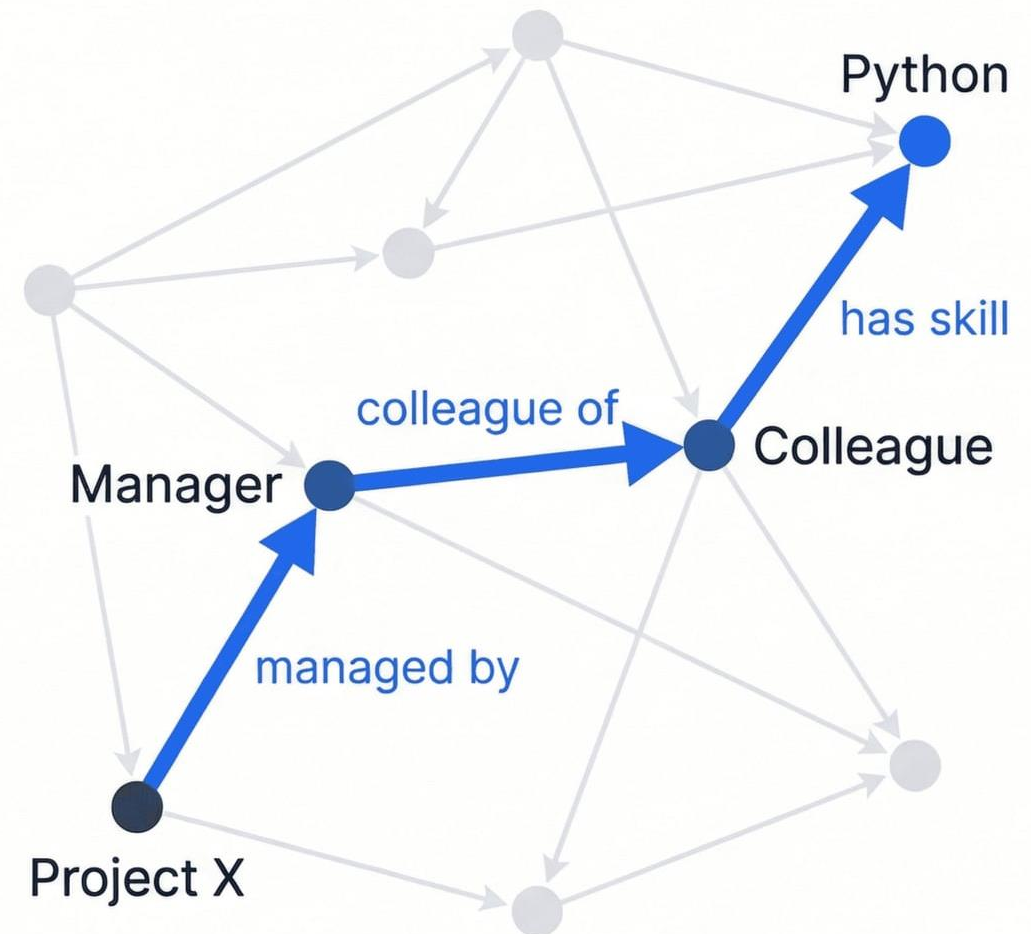
Unlocking Complex Insights with Multi-Hop Reasoning

Multi-hop reasoning connects disparate information.

The process of answering complex questions by traversing multiple nodes and relationships.

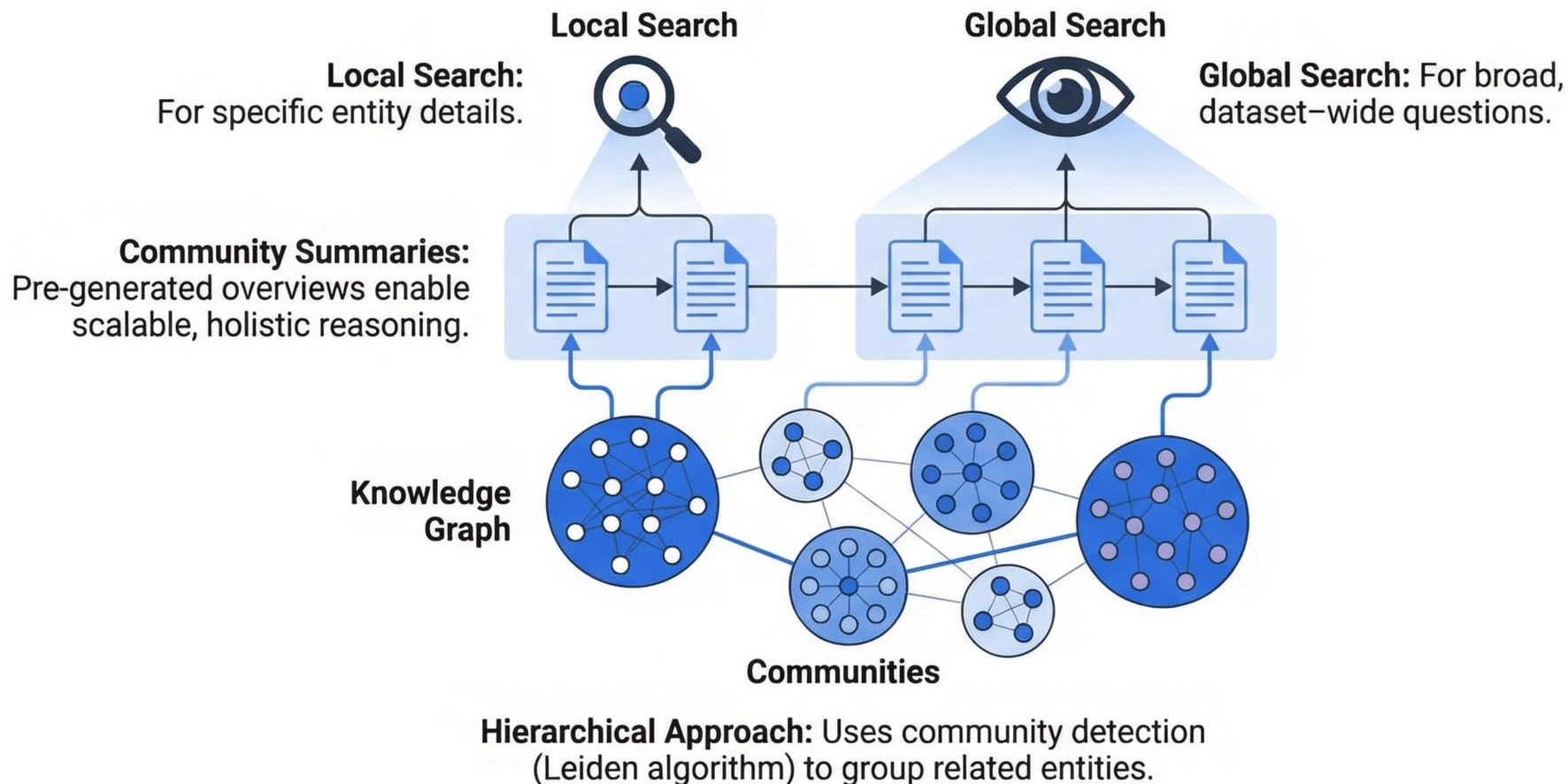
“Which colleagues of the project manager for Project X use Python?”

Enables deep, nuanced understanding impossible with simple vector search.





Microsoft's GraphRAG: Hierarchical Reasoning with Community Detection





The Transformative Benefits of GraphRAG



Accuracy & Relevance

Retrieves precise, contextually relevant information, reducing hallucinations.



Enhanced Explainability

Provides transparent reasoning trails, allowing users to trace how answers were derived.



Deeper Contextual Understanding

Uncovers hidden relationships and insights impossible with simple keyword search.



Conclusion: The Future of AI is Connected

Key Takeaways

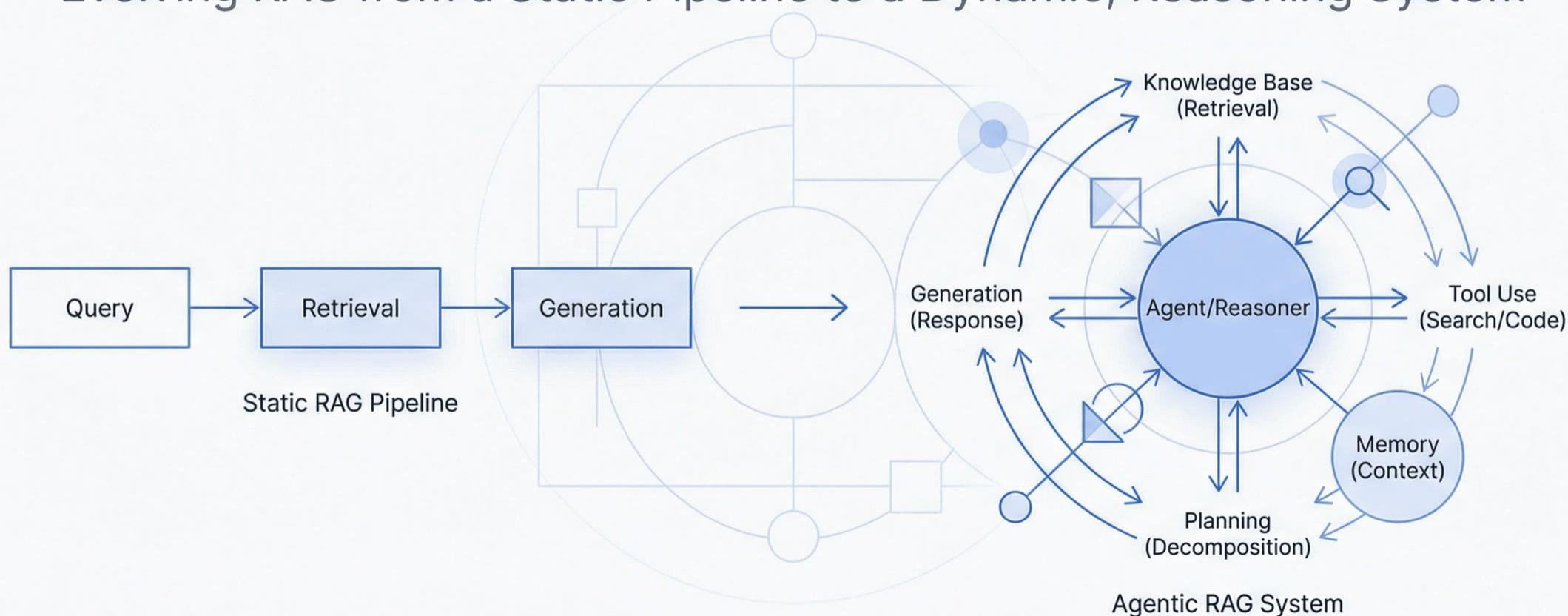
- GraphRAG is a major advancement: Overcomes the limitations of baseline RAG.
- The future is connected: Knowledge graphs are key to unlocking AI's potential.
- Empowering organizations: Builds more powerful, trustworthy, and insightful AI.





Beyond Retrieval: An Introduction to Agentic RAG

Evolving RAG from a Static Pipeline to a Dynamic, Reasoning System



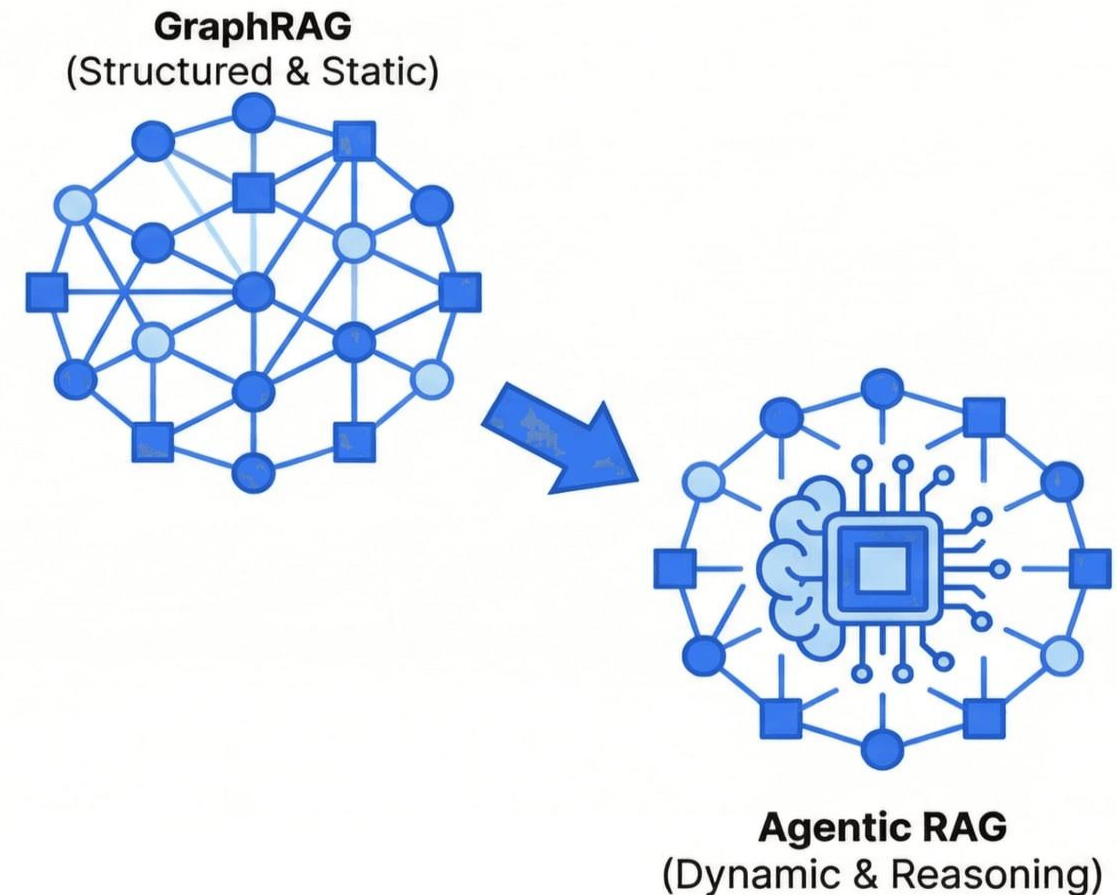


From GraphRAG to Agentic RAG

Evolving the Pipeline: Adding Autonomy and Reasoning to Knowledge Graph Retrieval

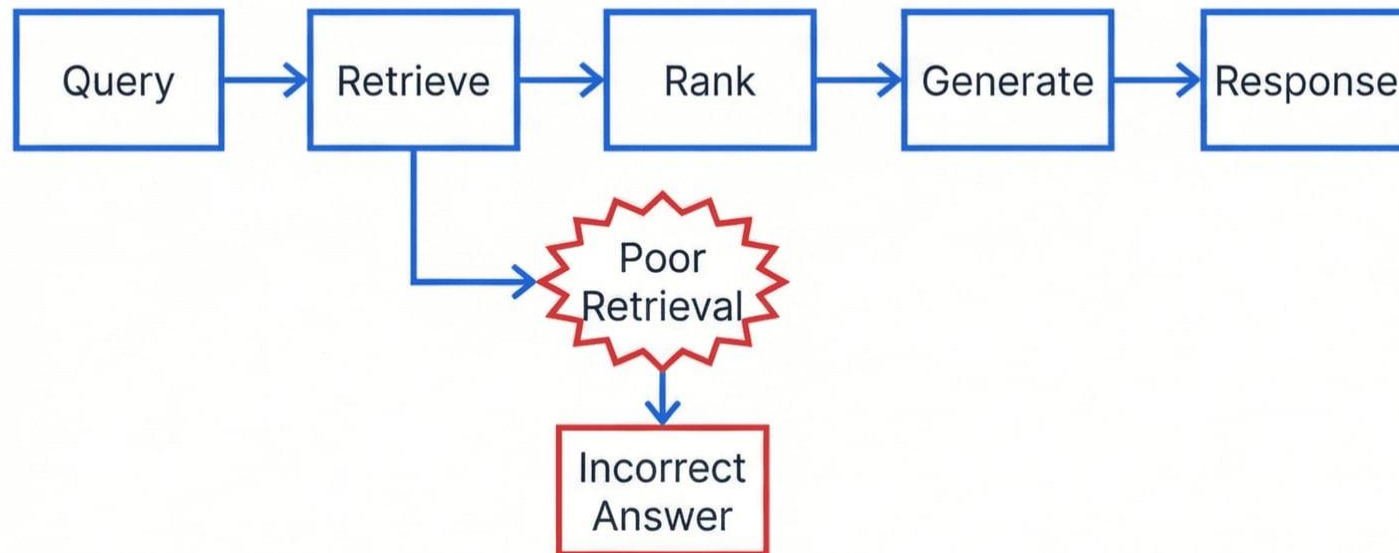
**We've Explored Knowledge Graphs;
Now Let's Add a "Brain"**

- **GraphRAG Recap:** Leverages structured knowledge graphs for multi-hop reasoning and contextual understanding.
- **Limitation of Static Pipelines:** Even with graphs, the workflow is a fixed "retrieve-then-generate" path.
- **The Next Step:** Introducing autonomy and reasoning into the retrieval process itself.
- **Core Idea:** Shifting from a passive pipeline to an active, problem-solving system.





Traditional RAG is a One-Way Street: It Can't Course-Correct



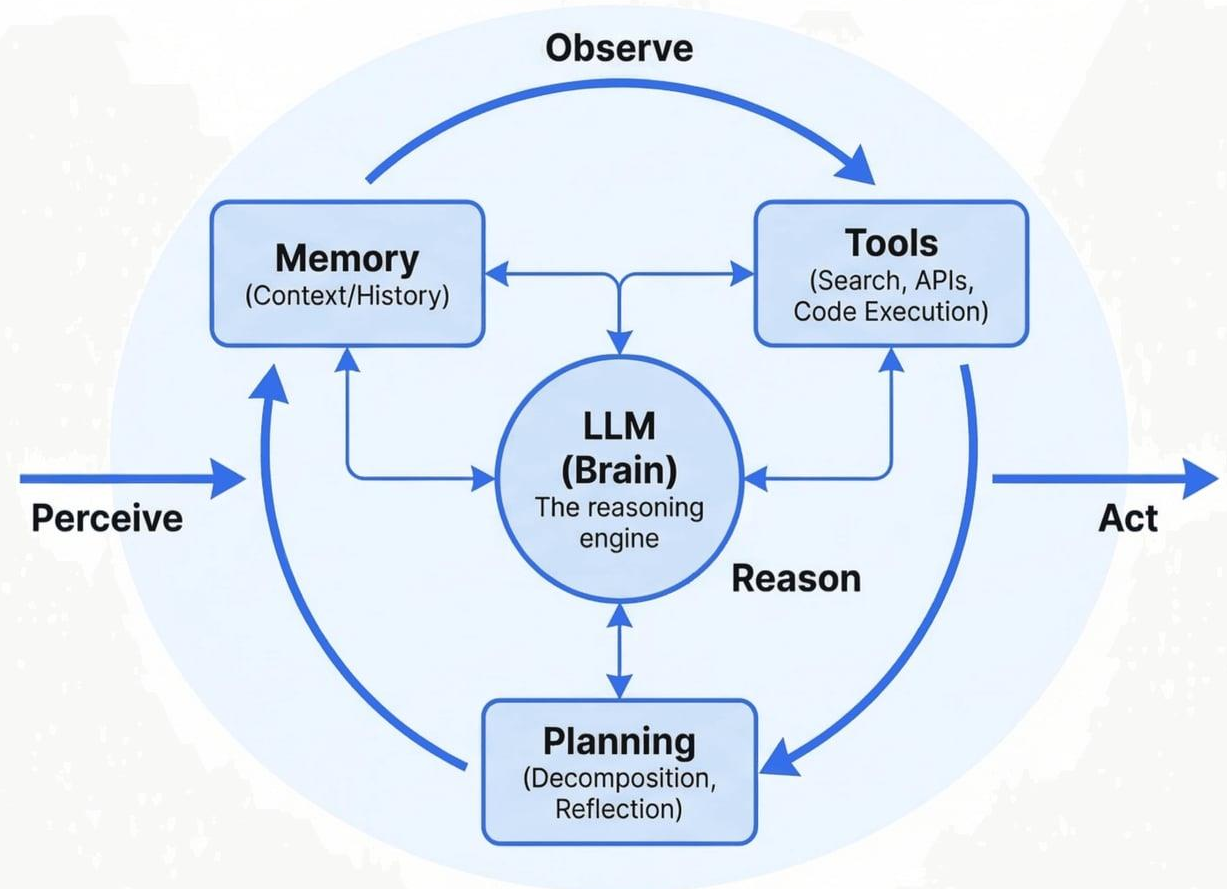
- **Linear Pipeline:** Fixed retrieve-rank-generate sequence.
- **No Self-Correction:** System cannot evaluate retrieval quality.
- **Vulnerable:** “Garbage in, garbage out” — poor context leads to failure.
- **Rigid:** Fails to handle ambiguous queries or adapt.



What is an AI Agent?

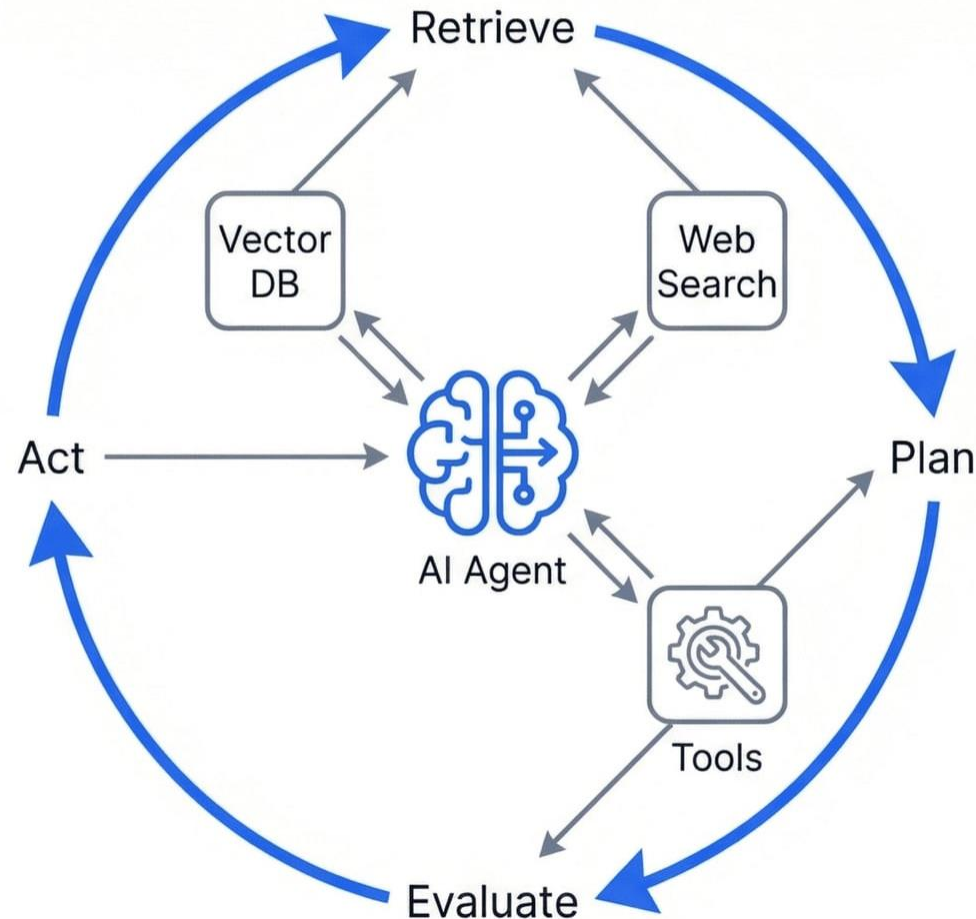
An Agent is More Than a Model: It's a System That Perceives, Reasons, and Acts

- **Core Components:** An agent consists of an LLM for reasoning, tools for acting, and memory for context.
- **The "Reason-Act" Loop:** Agents operate in an iterative cycle: Think -> Act -> Observe.
- **Autonomous Problem-Solving:** Agents can break down complex goals into smaller steps.
- **Beyond Text Generation:** Agents interact with external systems to execute tasks.





Agentic RAG Turns Retrieval into an Active, Iterative Dialogue



- **Agent-Driven Retrieval:** An AI agent orchestrates the entire pipeline.
- **Iterative Process:** The agent retrieves, evaluates, and re-retrieves in a loop.
- **Dynamic Tool Use:** Intelligently selects tools (Vector Search, Web, APIs).
- **From Lookup to Workflow:** Transforms static fetching into adaptive problem-solving.



Agentic RAG vs. Traditional RAG: A New Paradigm

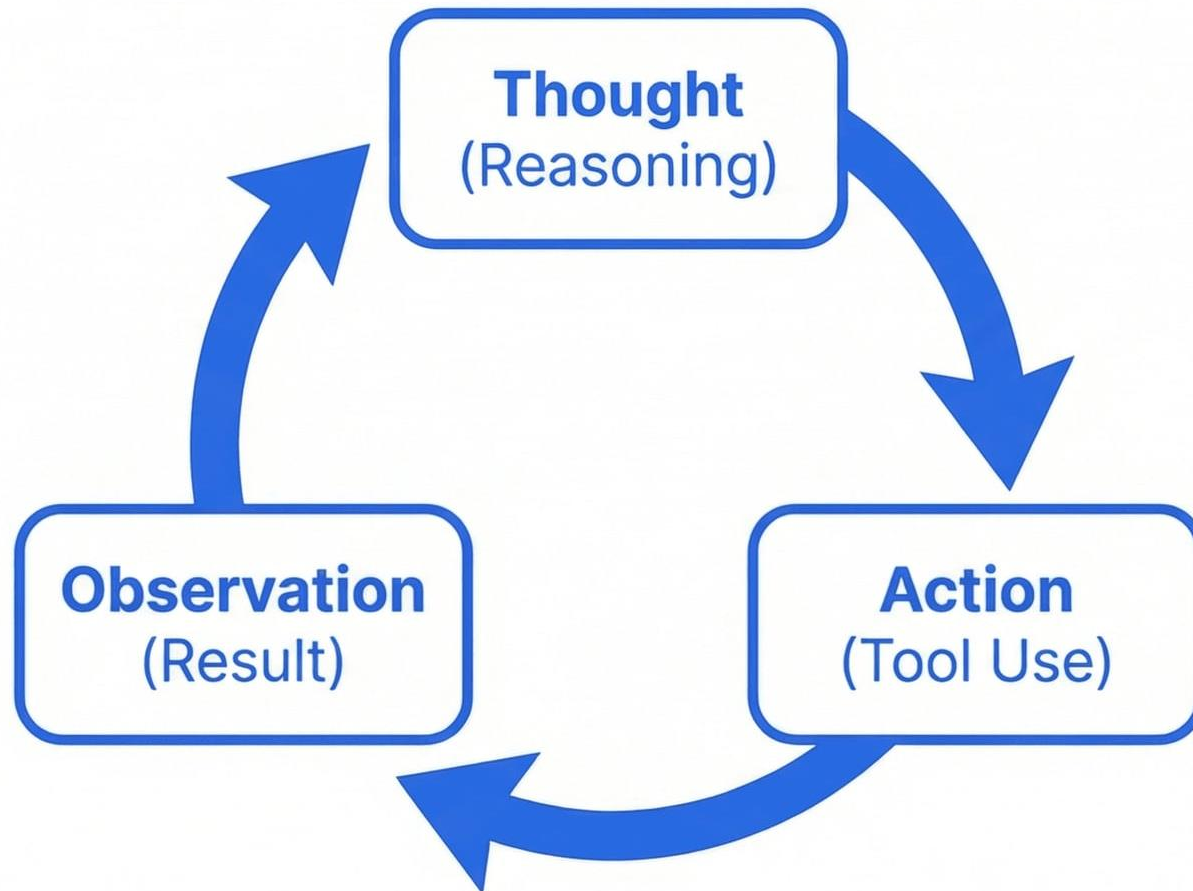
The Leap from a Fixed Assembly Line to a Robotic Workshop

	Traditional RAG	Agentic RAG
Workflow	Linear, one-shot (Retrieve -> Generate)	Iterative, multi-step (Reason -> Act -> Observe)
Retrieval Strategy	Static; single source	Dynamic; multiple tools/sources
Query Handling	Processes as-is	Analyzes, decomposes, rewrites
Self-Correction	None	Yes; evaluates and re-retrieves
Adaptability	Low; brittle	High; context-aware



The Agentic Workflow: The ReAct Framework

The Core Engine of Agentic RAG is the 'Reason-Act' Loop

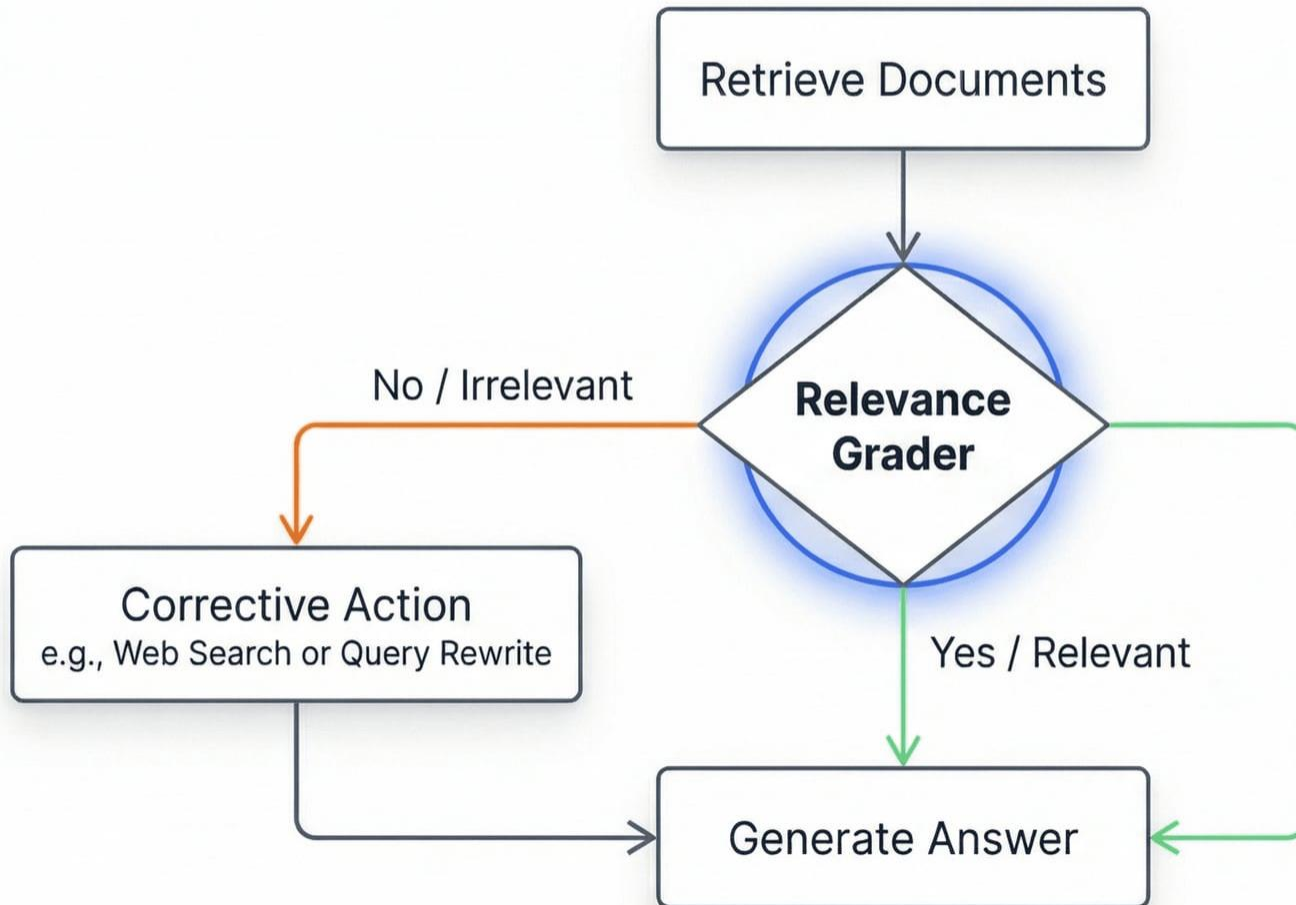


Key Points:

- **ReAct Framework:** Design pattern alternating between Reasoning and Acting.
- **Iterative Refinement:** Builds understanding step-by-step.
- **Transparency:** Externalized reasoning helps debugging.
- **Stateful:** Maintains memory of past actions.



CRAG Teaches the System to Ask: “Is This Information Actually Useful?”



Key Points

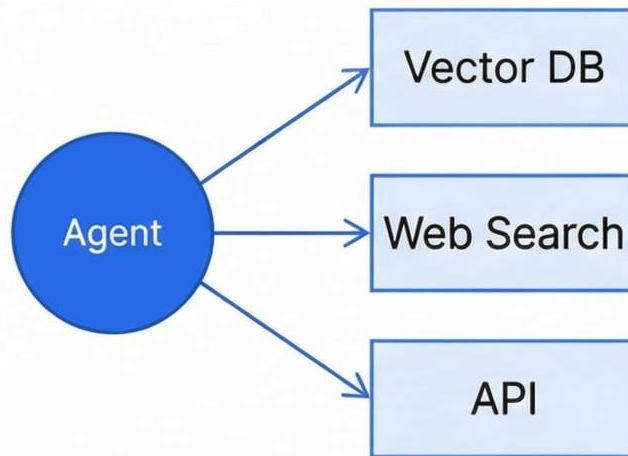
- **Self-Grading:** Internal component evaluates document relevance.
- **Corrective Action:** Triggers new search if quality is low.
- **Dynamic Source Switching:** Switches to web search if needed.
- **Increased Robustness:** Prevents errors from poor context.



Agentic Architectures

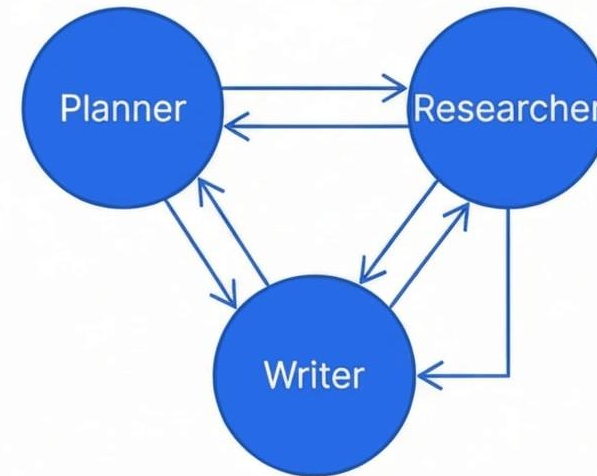
Agentic RAG Can Be a Single Router or a Team of Specialists

Single-Agent Router



A single decision-maker directs queries to the best tool.

Multi-Agent System



Specialized agents collaborate to solve complex problems.

- **Scalability:** Multi-agent systems handle complex workflows better.
- **Specialization:** Agents can focus on specific tasks (planning, execution).



Benefits and Future Outlook

Agentic RAG Paves the Way for Truly Autonomous Information Systems

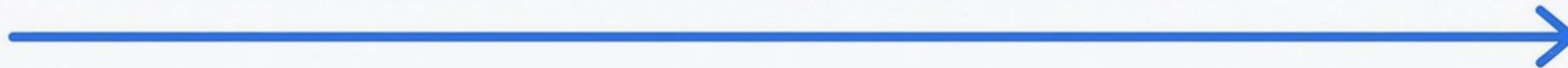
Benefits

- ✓ Improved Accuracy & Relevance
- ✓ Greater Adaptability
- ✓ Reduced Hallucinations

Challenges

- Increased System Complexity
- Potential Latency
- Higher Operational Costs

Moving from simple chatbots to autonomous information robots.





Conclusion

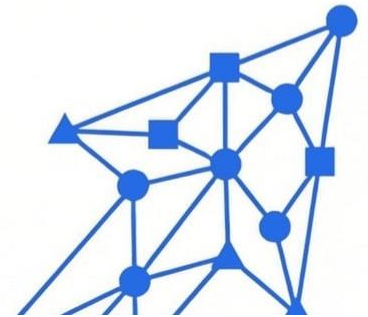
From Static Lookups to Dynamic Reasoning

Paradigm Shift: Moving from instruction-following to autonomous problem-solving.

Key Enabler: The AI agent, with its ability to reason, plan, and use tools.

Core Benefit: More accurate, reliable, and adaptable information systems.

Future of RAG: An intelligent, dynamic, and indispensable component of advanced AI.





Function Call

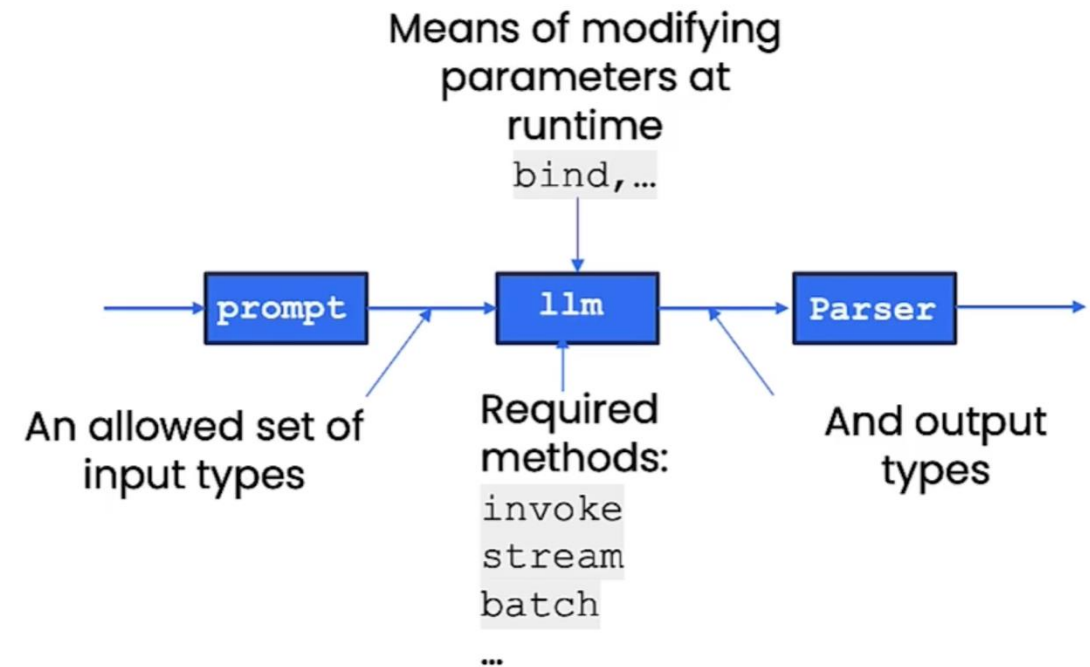
- OpenAI has fine tuned GPT-3.5-turbo and GPT-4 models to
 - Accept additional arguments through which users can pass in descriptions of functions.
 - If it is relevant, return the name of the function to use, along with a JSON object with the appropriate input parameters.
- Notebook example:
 - https://drive.google.com/file/d/1U-Cb_ED3anGlfUvqm19ZKvVdBTBJb4kM/view?usp=sharing



LangChain Expression Language (LCEL)

LangChain composes chains of **components**

LCEL and the runnable protocol define:



Composition can now use the Linux pipe syntax:

```
Chain = prompt | llm | OutputParser
```



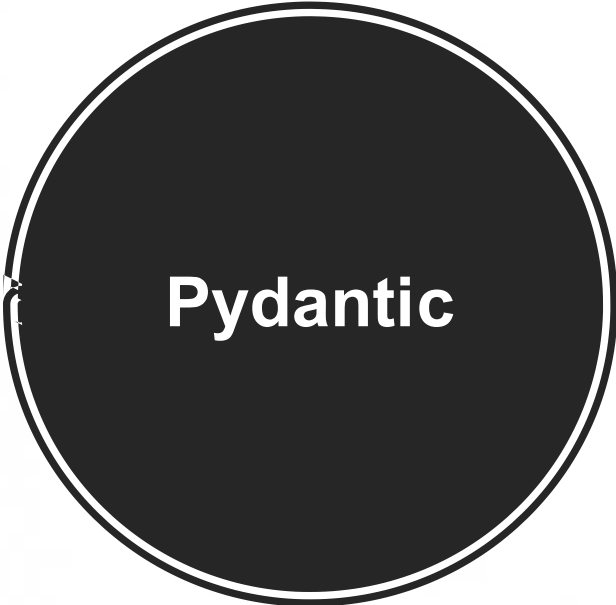

Interface

- Components implement “**Runnable**” protocol
- Common methods include:
 - `invoke` [`ainvoke`]
 - `stream` [`astream`]
 - `batch` [`abatch`]
- Common properties
 - `input_schema`, `output_schema`
- Common I/O

Component	Input Type	Output Type
Prompt	Dictionary	Prompt Value
Retriever	Single String	List of Documents
LLM	String, list of messages or Prompt Value	String
ChatModel	String, list of messages or Prompt Value	ChatMessage
Tool	String/Dictionary	Tool dependent
Output Parser	Output of LLM or ChatModel	Parser dependent



- **Runnables support:**
 - Async, Batch and Streaming Support
 - Fallbacks
 - Parallelism
 - LLM calls can be time consuming
 - Any components that can be run in parallel are!
 - Logging is built in

A large black circle with a white border, containing the word "Pydantic" in white text.

Pydantic

Pydantic is a 'data validation library' for python.

- Works with python type annotations. But rather than static type checking, they are actively used at runtime for data validation and conversion.
- Provides built-in methods to serialize/deserialize models to/from JSON, dictionaries, etc.
- LangChain leverages Pydantic to create JSON Scheme describing function.



Pydantic

```
class pUser(BaseModel):
    name: str
    age: int
    email: str

foo_p = pUser(name="Jane", age=32, email="jane@gmail.com")

foo_p.name

'Jane'

foo_p = pUser(name="Jane", age="bar", email="jane@gmail.com")

-----
ValidationError                                Traceback (most recent call last)
Cell In[20], line 1
----> 1 foo_p = pUser(name="Jane", age="bar", email="jane@gmail.com")

File ~/Documents/GitHub/SC-langchain-openai/.venv_LC3/lib/python3.9/site-packages/pydantic/main.py:341, in pydantic.main.BaseModel.__init__()
Validation error: 1 validation error for pUser
age
  value is not a valid integer (type=type_error.integer)
```

In normal python you would create a class like this:

```
class User:
    def __init__(self, name: str, age: int, email: str):
        self.name = name
        self.age = age
        self.email = email
```

Using Pydantic, this is:

```
from pydantic import BaseModel,

class User(BaseModel):
    name: str
    age: int
    email: EmailStr
```




Pydantic with LangChain for OpenAI function calls

Using multiple functions

Even better, we can pass a set of function and let the LLM decide which to use based on the question context.

```
class ArtistSearch(BaseModel):  
    """Call this to get the names of songs by a particular artist"""  
    artist_name: str = Field(description="name of artist to look up")  
    n: int = Field(description="number of results")
```

```
functions = [  
    convert_pydantic_to_openai_function(WeatherSearch),  
    convert_pydantic_to_openai_function(ArtistSearch),  
]
```

```
model_with_functions = model.bind(functions=functions)
```

```
model_with_functions.invoke("what is the weather in sf?")
```

```
AIMessage(content='', additional_kwargs={'function_call': {'name': 'WeatherSearch', 'arguments': '{\n  "airport_code": "SFO"\n}'}})
```

```
model_with_functions.invoke("what are three songs by taylor swift?")
```

```
AIMessage(content='', additional_kwargs={'function_call': {'name': 'ArtistSearch', 'arguments': '{\n  "artist_name": "Taylor Swift",\n  "n": 3\n}'}})
```



Tagging and Extraction Using OpenAI functions

```
import os
import openai

from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
openai.api_key = os.environ['OPENAI_API_KEY']

from typing import List
from pydantic import BaseModel, Field
from langchain.utils.openai_functions import convert_pydantic_to_openai_f

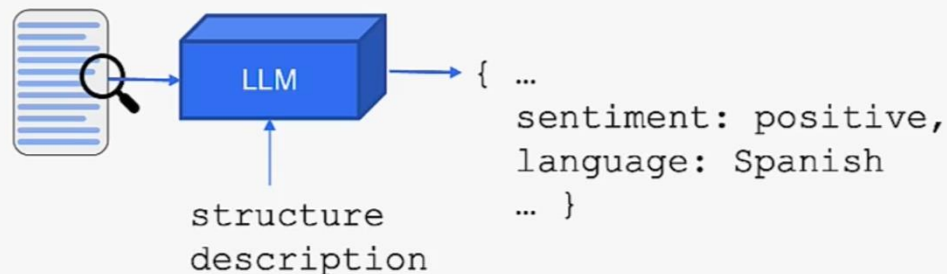
class Tagging(BaseModel):
    """Tag the piece of text with particular info."""
    sentiment: str = Field(description="sentiment of text, should be `pos` or `neg`")
    language: str = Field(description="language of text (should be ISO 639-1 code)")

convert_pydantic_to_openai_function(Tagging)

{
  'name': 'Tagging',
  'description': 'Tag the piece of text with particular info.',
  'parameters': {
    'title': 'Tagging',
    'description': 'Tag the piece of text with particular info.',
    'type': 'object',
    'properties': {
      'sentiment': {
        'title': 'Sentiment',
        'description': 'sentiment of text, should be `pos`, `neg`, or `neutral`',
        'type': 'string',
        'language': {
          'title': 'Language',
          'description': 'language of text (should be ISO 639-1 code)',
          'type': 'string'
        }
      },
      'language': {
        'title': 'Language',
        'description': 'language of text (should be ISO 639-1 code)',
        'type': 'string'
      }
    },
    'required': ['sentiment', 'language']
  }
}
```

Tagging

- We have seen the LLM, given a function description, select arguments from the input text generate a structured output forming a function call
- More generally, the LLM can *evaluate* the input text and generate structured output.





```
from langchain.prompts import ChatPromptTemplate
from langchain.chat_models import ChatOpenAI

model = ChatOpenAI(temperature=0)

tagging_functions = [convert_pydantic_to_openai_function(Tagging)]

prompt = ChatPromptTemplate.from_messages([
    ("system", "Think carefully, and then tag the text as instructed"),
    ("user", "{input}")
])

model_with_functions = model.bind(
    functions=tagging_functions,
    function_call={"name": "Tagging"}
)

tagging_chain = prompt | model_with_functions

tagging_chain.invoke({"input": "I love langchain"})

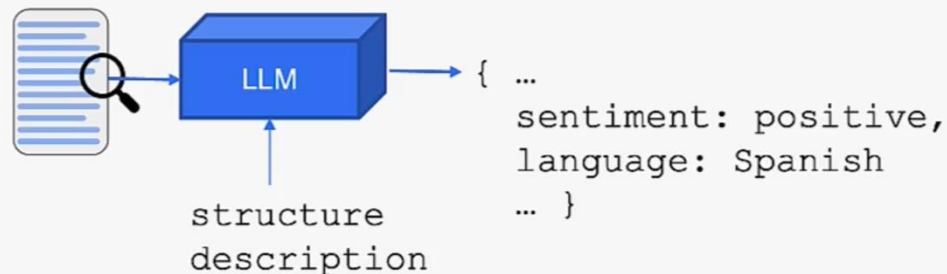
AIMessage(content='', additional_kwargs={'function_call': {'name': 'Tagging', 'arguments': '{\n  "sentiment": "pos",\n  "language": "en"\n}'}})

tagging_chain.invoke({"input": "non mi piace questo cibo"})

AIMessage(content='', additional_kwargs={'function_call': {'name': 'Tagging', 'arguments': '{\n  "sentiment": "neg",\n  "language": "it"\n}'}})
```

Tagging

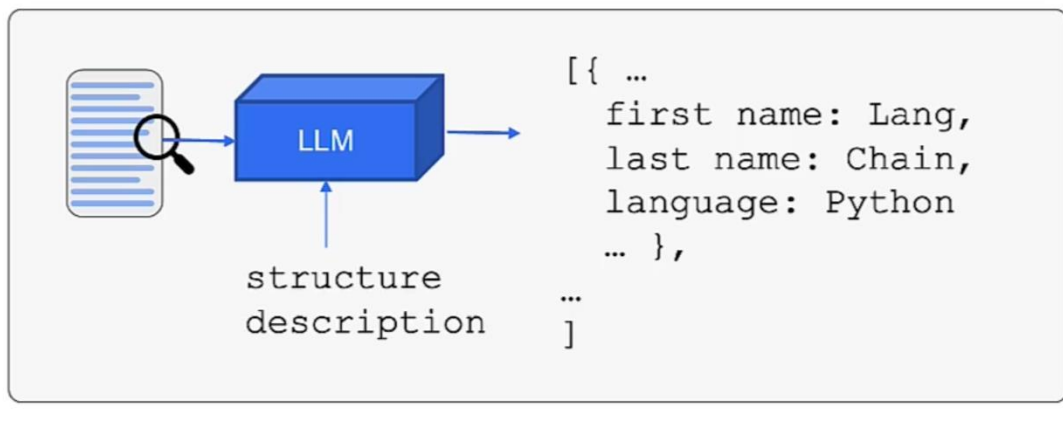
- We have seen the LLM, given a function description, select arguments from the input text generate a structured output forming a function call
- More generally, the LLM can *evaluate* the input text and generate structured output.





Extraction

- When given an input JSON schema, the LLM has been fine tuned to find and fill in the parameters of that schema.
- The capability is not limited to function schema.
- This can be used for general purpose extraction.



Extraction

Extraction is similar to tagging, but used for extracting multiple pieces of information.

```
from typing import Optional  
class Person(BaseModel):  
    """Information about a person."""  
    name: str = Field(description="person's name")  
    age: Optional[int] = Field(description="person's age")
```

```
class Information(BaseModel):  
    """Information to extract."""  
    people: List[Person] = Field(description="List of info about people")
```

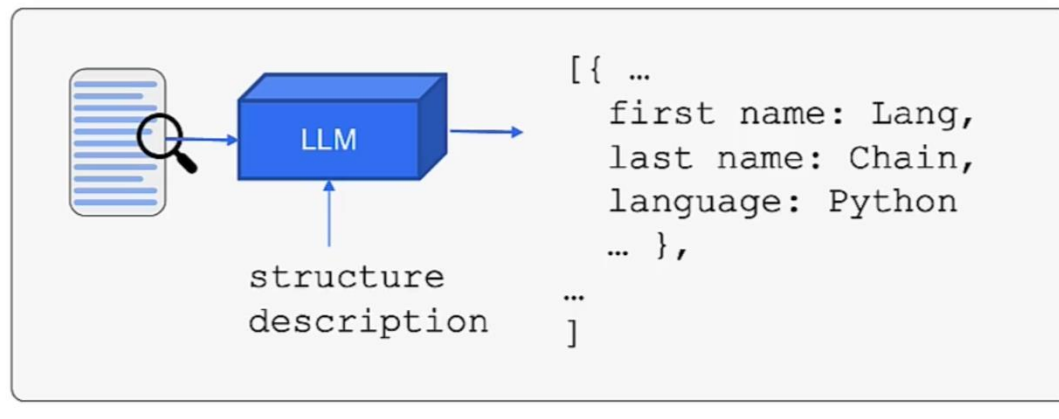
```
convert_pydantic_to_openai_function(Information)
```

```
{'name': 'Information',  
 'description': 'Information to extract.',  
 'parameters': {'title': 'Information',  
 'description': 'Information to extract.',  
 'type': 'object',  
 'properties': {'people': {'title': 'People',  
 'description': 'List of info about people',  
 'type': 'array',  
 'items': {'title': 'Person',  
 'description': 'Information about a person.',  
 'type': 'object',  
 'properties': {'name': {'title': 'Name',  
 'description': 'person's name',  
 'type': 'string'},  
 'age': {'title': 'Age',  
 'description': 'person's age',  
 'type': 'integer'}}},  
 'required': ['name']}}},  
 'required': ['people']}}
```




Extraction

- When given an input JSON schema, the LLM has been fine tuned to find and fill in the parameters of that schema.
- The capability is not limited to function schema.
- This can be used for general purpose extraction.



```
'type': 'object',
'properties': {'name': {'title': 'Name',
'description': "person's name",
'type': 'string'},
'age': {'title': 'Age',
'description': "person's age",
'type': 'integer'}},
'required': ['name']},
'required': ['people']}]
```

```
extraction_functions = [convert_pydantic_to_openai_function(Information)]
extraction_model = model.bind(functions=extraction_functions, function_ca
```

```
extraction_model.invoke("Joe is 30, his mom is Martha")
```

```
AIMessage(content='', additional_kwargs={'function_call': {'name': 'Info
rmation', 'arguments': '{\n  "people": [\n    {\n      "name": "Joe",\n    "age": 30\n    },\n    {\n      "name": "Martha",\n      "age": 0\n    }
  ]\n}'}})
```

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "Extract the relevant information, if not explicitly provi
    ("human", "{input}")
])
```

```
extraction_chain = prompt | extraction_model
```

```
extraction_chain.invoke({"input": "Joe is 30, his mom is Martha"})
```

```
AIMessage(content='', additional_kwargs={'function_call': {'name': 'Info
rmation', 'arguments': '{\n  "people": [\n    {\n      "name": "Joe",\n    "age": 30\n    },\n    {\n      "name": "Martha"\n    }\n  ]\n}'}})
```

```
extraction_chain = prompt | extraction_model | JsonOutputFunctionsParser()
```

```
extraction_chain.invoke({"input": "Joe is 30, his mom is Martha"})
```




Tools and Routing

- Functions and services an LLM can utilize to extend its capabilities are named “tools” in LangChain
- LangChain has many tools available
 - Search tools
 - Math tools
 - SQL tools
 - ...

```
import requests
from pydantic import BaseModel, Field
import datetime

# Define the input schema
class OpenMeteoInput(BaseModel):
    latitude: float = Field(..., description="Latitude of the location to")
    longitude: float = Field(..., description="Longitude of the location")

@tool(args_schema=OpenMeteoInput)
def get_current_temperature(latitude: float, longitude: float) -> dict:
    """Fetch current temperature for given coordinates."""

    BASE_URL = "https://api.open-meteo.com/v1/forecast"

    # Parameters for the request
    params = {
        'latitude': latitude,
        'longitude': longitude,
        'hourly': 'temperature_2m',
        'forecast_days': 1,
    }

    # Make the request
    response = requests.get(BASE_URL, params=params)

    if response.status_code == 200:
        results = response.json()
    else:
        raise Exception(f"API Request failed with status code: {response.status_code}")

    current_utc_time = datetime.datetime.utcnow()
    time_list = [datetime.datetime.fromisoformat(time_str.replace('Z', '+')) for time_str in results['hourly']['time']]
    temperature_list = results['hourly']['temperature_2m']

    closest_time_index = min(range(len(time_list)), key=lambda i: abs(time_list[i] - current_utc_time))
    current_temperature = temperature_list[closest_time_index]
```



```
import wikipedia
@tool
def search_wikipedia(query: str) -> str:
    """Run Wikipedia search and get page summaries."""
    page_titles = wikipedia.search(query)
    summaries = []
    for page_title in page_titles[: 3]:
        try:
            wiki_page = wikipedia.page(title=page_title, auto_suggest=False)
            summaries.append(f"Page: {page_title}\nSummary: {wiki_page.summary}")
        except (
            self.wiki_client.exceptions.PageError,
            self.wiki_client.exceptions.DisambiguationError,
        ):
            pass
    if not summaries:
        return "No good Wikipedia Search Result was found"
    return "\n\n".join(summaries)
```

```
search_wikipedia.name
```

```
'search_wikipedia'
```

```
search_wikipedia.description
```

```
'search_wikipedia(query: str) -> str - Run Wikipedia search and get page summaries.'
```

```
format_tool_to_openai_function(search_wikipedia)
```

```
{'name': 'search_wikipedia',
 'description': 'search_wikipedia(query: str) -> str - Run Wikipedia search and get page summaries.',
 'parameters': {'title': 'search_wikipediaSchemaSchema',
 'type': 'object',
 'properties': {'query': {'title': 'Query', 'type': 'string'}}},
 'schema': {'title': 'search_wikipediaSchemaSchema',
 'type': 'object',
 'properties': {'query': {'title': 'Query', 'type': 'string'}}}}
```

```
functions = [
    format_tool_to_openai_function(f) for f in [
        search_wikipedia, get_current_temperature
    ]
]
model = ChatOpenAI(temperature=0).bind(functions=functions)
```

```
model.invoke("what is the weather in sf right now")
```

```
AIMessage(content='', additional_kwargs={'function_call': {'name': 'get_current_temperature', 'arguments': '{\n  "latitude": 37.7749,\n  "longitude": -122.4194\n}'}})
```

```
model.invoke("what is langchain")
```

```
AIMessage(content='', additional_kwargs={'function_call': {'name': 'search_wikipedia', 'arguments': '{\n  "query": "langchain"\n}'}})
```

```
from langchain.prompts import ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are helpful but sassy assistant"),
    ("user", "{input}"),
])
chain = prompt | model
```

```
chain.invoke({"input": "what is the weather in sf right now"})
```

```
AIMessage(content='', additional_kwargs={'function_call': {'name': 'get_current_temperature', 'arguments': '{\n  "latitude": 37.7749,\n  "longitude": -122.4194\n}'}})
```





```
chain.invoke({"input": "what is the weather in sf right now"})
```

```
AIMessage(content='', additional_kwargs={'function_call': {'name': 'get_current_temperature', 'arguments': '{\n  "latitude": 37.7749,\n  "longitude": -122.4194\n}'}})
```

```
from langchain.agents.output_parsers import OpenAIFunctionsAgentOutputParser
```

```
chain = prompt | model | OpenAIFunctionsAgentOutputParser()
```

```
result = chain.invoke({"input": "what is the weather in sf right now"})
```

```
type(result)
```

```
langchain.schema.agent.AgentActionMessageLog
```

```
result.tool
```

```
'get_current_temperature'
```

```
result.tool_input
```

```
{'latitude': 37.7749, 'longitude': -122.4194}
```

```
get_current_temperature(result.tool_input)
```

```
'The current temperature is 22.9°C'
```

```
{ "output": "hello: how can i assist you today: " }
```

```
from langchain.schema.agent import AgentFinish
def route(result):
    if isinstance(result, AgentFinish):
        return result.return_values['output']
    else:
        tools = {
            "search_wikipedia": search_wikipedia,
            "get_current_temperature": get_current_temperature,
        }
        return tools[result.tool].run(result.tool_input)
```

```
chain = prompt | model | OpenAIFunctionsAgentOutputParser() | route
```

```
result = chain.invoke({"input": "What is the weather in san francisco right now"})
```

```
result
```

```
'The current temperature is 22.9°C'
```

```
result = chain.invoke({"input": "What is langchain?"})
```

```
result
```

```
'Page: LangChain\nSummary: LangChain is a framework designed to simplify the creation of applications using large language models (LLMs). As a language model integration framework, LangChain\'s use-cases largely overlap with those of language models in general, including document analysis and summarization, chatbots, and code analysis.\n\nPage: Prompt engineering\nSummary: Prompt engineering is the process of structuring text that can be interpreted and understood by a generative AI model. A prompt is natural language text describing the task that an AI should perform. A prompt for a text-to-text model can be a query such as "what is Fermat\'s little theorem?", a command such as "write a poem about leaves falling", a short statement of feedback (for example, "too verbose", "too formal", "rephrase again", "omit this word") or a longer statement including context.'
```